FAUST Quick Reference
(version 2.88.0)

GRAME
Centre National de Création Musicale

September 4, 2026

Contents

1 Introduction **9**

1.1 Design Principles 9

1.2 Signal Processor Semantics 10

2 Compiling and Installing FAUST **13**

2.1 Organization of the distribution 13

2.2 Compilation 14

2.3 Installation 15

2.4 Compilation of the examples 15

3 FAUST Syntax **17**

3.1 FAUST program 18

3.1.1 A Simple Program 18

3.2 Statements 18

3.2.1 Declarations 19

3.2.2 Imports 19

3.2.3 Documentation 20

3.3 Definitions 22

3.3.1 Simple Definitions 22

3.3.2 Function Definitions 22

3.3.3 Definitions with pattern matching 23

3.3.4 Variants . . . 24
3.4 Expressions . . . 24
3.4.1 Constant Numerical Expressions . . . 25
3.4.2 Diagram Expressions . . . 25
3.4.3 Infix Notation . . . 34
3.4.4 Prefix Notation . . . 37
3.4.5 Time expressions . . . 38
3.4.6 Environment expressions . . . 39
3.4.7 Foreign expressions . . . 46
3.4.8 Applications and Abstractions . . . 50
3.4.9 Equivalent expressions . . . 53
3.5 Primitives . . . 54
3.5.1 Numbers . . . 54
3.5.2 Route Primitive . . . 55
3.5.3 Waveform Primitive . . . 57
3.5.4 Soundfile Primitive . . . 57
3.5.5 C-equivalent primitives . . . 58
3.5.6 `math.h`-equivalent primitives . . . 59
3.5.7 Delay, Table, Selector primitives . . . 60
3.5.8 User Interface Elements . . . 61
3.5.9 Widget Modulation . . . 66

4 Invoking the FAUST Compiler 69
4.1 Structure of the generated code . . . 69
4.2 Compilation options . . . 72

5 Embedding the FAUST Compiler Using libfaust 75
5.1 Dynamic compilation chain . . . 75
5.2 LLVM . . . 75
5.3 Compiling in memory . . . 76
5.4 Saving/restoring the factory . . . 78
5.5 Additional functions . . . 79

5.6 Using the libfaust library 79
5.7 Use case examples 79

6 Architecture files 81
6.1 Audio architecture modules 81
6.2 UI architecture modules 83
6.2.1 Active widgets 84
6.2.2 Passive widgets 84
6.2.3 Widgets layout 84
6.2.4 Metadata 86
6.3 Developing a new architecture file 87

7 OSC support 93
7.1 A simple example 93
7.2 Automatic port allocation 95
7.3 Discovering OSC applications 96
7.4 Discovering the OSC interface of an application 96
7.5 Widget's OSC address 97
7.6 Controlling the application via OSC 98
7.7 Turning transmission ON 98
7.8 Filtering OSC messages 99
7.9 Using OSC aliases 100
7.10 OSC cheat sheet 102
7.11 DSP with polyphonic support 102

8 HTTP support 105
8.1 A Simple Example 105
8.2 JSON Description of the User Interface 107
8.3 Querying the State of the Application 108
8.4 Changing the value of a widget 109
8.5 Proxy control access to the Web server 109
8.6 HTTP cheat sheet 109

9 MIDI support **113**

9.1 Describing MIDI messages in the DSP source code 113

9.2 Description of the possible standard MIDI messages 113

9.3 A Simple Example 114

9.4 MIDI Synchronization 116

10 Polyphonic support **117**

10.1 Polyphony-ready DSP code 117

10.2 Using the mydsp_poly class 117

10.3 Controlling the polyphonic instrument 119

10.4 Deploying the polyphonic instrument 120

10.5 Polyphonic instrument with a global output effect 120

10.5.1 Integrated global output effect 120

10.5.2 Integrated global output effect and libfaust 121

11 Controlling the Code Generation **123**

11.1 Vector code generation 123

11.2 Parallel code generation 126

11.2.1 The OpenMP code generator 126

11.2.2 Adding OpenMP directives 128

11.2.3 Example of parallel OpenMP code 129

11.2.4 The scheduler code generator 132

11.2.5 Example of parallel scheduler code 133

12 Mathematical Documentation **137**

12.1 Goals of the mathdoc 137

12.2 Installation requirements 137

12.3 Generating the mathdoc 138

12.3.1 Invoking the -mdoc option 138

12.3.2 Invoking faust2mathdoc 139

12.3.3 Online examples 139

12.4 Automatic documentation 139

12.5 Manual documentation 140
12.5.1 Six tags 140
12.5.2 The mdoc top-level tags 140
12.5.3 An example of manual mathdoc 141
12.5.4 The -stripmdoc option 143
12.6 Localization of mathdoc files 143
12.7 Summary of the mathdoc generation steps 147

13 Acknowledgments **149**

Chapter 1

Introduction

FAUST (*Functional Audio Stream*) is a functional programming language specifically designed for real-time signal processing and synthesis. FAUST targets high-performance signal processing applications and audio plug-ins for a variety of platforms and standards.

1.1 Design Principles

Various principles have guided the design of FAUST:

- FAUST is a *specification language*. It aims at providing an adequate notation to describe *signal processors* from a mathematical point of view. FAUST is, as much as possible, free from implementation details.
- FAUST programs are fully compiled, not interpreted. The compiler translates FAUST programs into equivalent target languages such as C/C++, LLVM IR, WebAssembly, and others, taking care of generating the most efficient code. The result can generally compete with, and sometimes even outperform, C++ code written by seasoned programmers.
- The generated code works at the sample level. It is therefore suited to implement low-level DSP functions like recursive filters. Moreover the code can be easily embedded. It is self-contained and does not depend on any DSP library or runtime system. It has a very deterministic behavior and a constant memory footprint.
- The semantics of FAUST are simple and well defined. This is not just of academic interest. It allows the FAUST compiler to be *semantically driven*. Instead of compiling a program literally, it compiles the mathematical function it denotes.

This feature is useful, for example, to promote component reuse while preserving optimal performance.

- FAUST is a textual language but nevertheless block-diagram oriented. It actually combines two approaches: *functional programming* and *algebraic block-diagrams*. The key idea is to view block-diagram construction as function composition. For that purpose, FAUST relies on a *block-diagram algebra* of five composition operations (`: , ~ <: :>`).
- Thanks to the notion of *architecture*, FAUST programs can be easily deployed on a large variety of audio platforms and plugin formats without any change to the FAUST code.

1.2 Signal Processor Semantics

A FAUST program describes a *signal processor*. The role of a *signal processor* is to transform a (possibly empty) group of *input signals* in order to produce a (possibly empty) group of *output signals*. Most audio equipment can be modeled as *signal processors*. They have audio inputs, audio outputs, as well as control signals interfaced with sliders, knobs, vu-meters, etc.

More precisely:

FAUST considers two types of signals: *integer signals* ($s : \mathbb{Z} \to \mathbb{Z}$) and *floating point signals* ($s : \mathbb{Z} \to \mathbb{Q}$). Exchanges with the outside world are, by convention, made using floating point signals. The full range is represented by sample values between -1.0 and +1.0.

- A *signal* s is a discrete function of time $s : \mathbb{Z} \to \mathbb{R}$. The value of a signal s at time t is written $s(t)$. The values of signals are usually needed starting from time 0. But to take into account *delay operations*, negative times are possible and are always mapped to zeros. Therefore for any FAUST signal s we have $\forall t < 0, s(t) = 0$. In operational terms this corresponds to assuming that all delay lines are signals initialized with 0s.
- The set of all possible signals is $\mathbb{S} = \mathbb{Z} \to \mathbb{R}$.
- A group of n signals (a n-tuple of signals) is written $(s_1, \ldots, s_n) \in \mathbb{S}^n$. The *empty tuple*, single element of $\mathbb{S}^0$, is denoted $()$.
- A *signal processor* p is a function from n-tuples of signals to m-tuples of signals $p : \mathbb{S}^n \to \mathbb{S}^m$. The set $\mathbb{P} = \bigcup_{n,m} \mathbb{S}^n \to \mathbb{S}^m$ is the set of all possible signal processors.

As an example, let's express the semantics of the FAUST primitive `+`. Like any FAUST expression, it is a signal processor. Its signature is $\mathbb{S}^2 \to \mathbb{S}$. It takes two input signals X_0 and X_1 and produces an output signal Y such that $Y(t) = X_0(t) + X_1(t)$.

Numbers are signal processors too. For example the number 3 has signature $\mathbb{S}^0 \to \mathbb{S}$. It takes no input signals and produces an output signal Y such that $Y(t) = 3$.

Chapter 2

Compiling and Installing FAUST

The FAUST source distribution `faust-2.88.0.tar.gz` can be downloaded from GitHub (https://github.com/grame-cncm/faust/releases).

2.1 Organization of the distribution

The first step is to decompress the downloaded archive.

```
tar xzf faust-.tar.gz
```

The resulting `faust-2.88.0/` folder should contain the following elements:

`architecture/`	FAUST libraries and architecture files
`benchmark`	tools to measure the efficiency of the generated code
`compiler/`	sources of the FAUST compiler
`examples/`	examples of FAUST programs
`syntax-highlighting/`	support for syntax highlighting for several editors
`documentation/`	FAUST's documentation, including this manual
`tools/`	tools to produce audio applications and plugins
`COPYING`	license information
`Makefile`	Makefile used to build and install FAUST
`README`	instructions on how to build and install FAUST

2.2 Compilation

FAUST has no dependencies outside standard libraries. Therefore, the compilation should be straightforward. There is no configuration phase; to compile the FAUST compiler simply do:

```
cd faust-2.88.0/
make
```

If the compilation is successful you can test the compiler before installing it:

```
[cd faust-2.88.0/]
./build/bin/faust -v
```

It should output:

```
FAUST Version 2.88.0
Embedded backends:
   DSP to AssemblyScript
   DSP to C
   DSP to C++
   DSP to Cmajor
   DSP to Codebox
   DSP to CSharp
   DSP to DLang
   DSP to Java
   DSP to NNX
   DSP to Linen
   DSP to Julia
   DSP to JSFX
   DSP to old C++
   DSP to Rust
   DSP to SDF3
   DSP to VHDL
   DSP to WebAssembly (wast/wasm)
Copyright (C) 2002-2026, GRAME - Centre National de
   Creation Musicale. All rights reserved.
```

You can then try to compile one of the examples:

```
[cd faust-2.88.0/]
./build/bin/faust examples/generator/noise.dsp
```

It should produce some C++ code on the standard output.

2.3 Installation

You can install FAUST with:

```
[cd faust-2.88.0/]
sudo make install
```

or

```
[cd faust-2.88.0/]
su
make install
```

depending on your system.

2.4 Compilation of the examples

Once FAUST is correctly installed, you can look at the provided examples in the `examples/` folder.

You can use any of the `faust2...` scripts installed on your system (go to `/tools/faust2appls` to get an exhaustive list) to compile the FAUST programs available in this folder. For example, if you're a Mac user and you want to turn `filtering/vcfWahLab.dsp` into a standalone CoreAudio application with a Qt interface, just run:

```
faust2caqt filtering/vcfWahLab.dsp
```

Chapter 3

Faust Syntax

This section describes the syntax of Faust. Figure 3.1 gives an overview of the various concepts and where they are defined in this section.

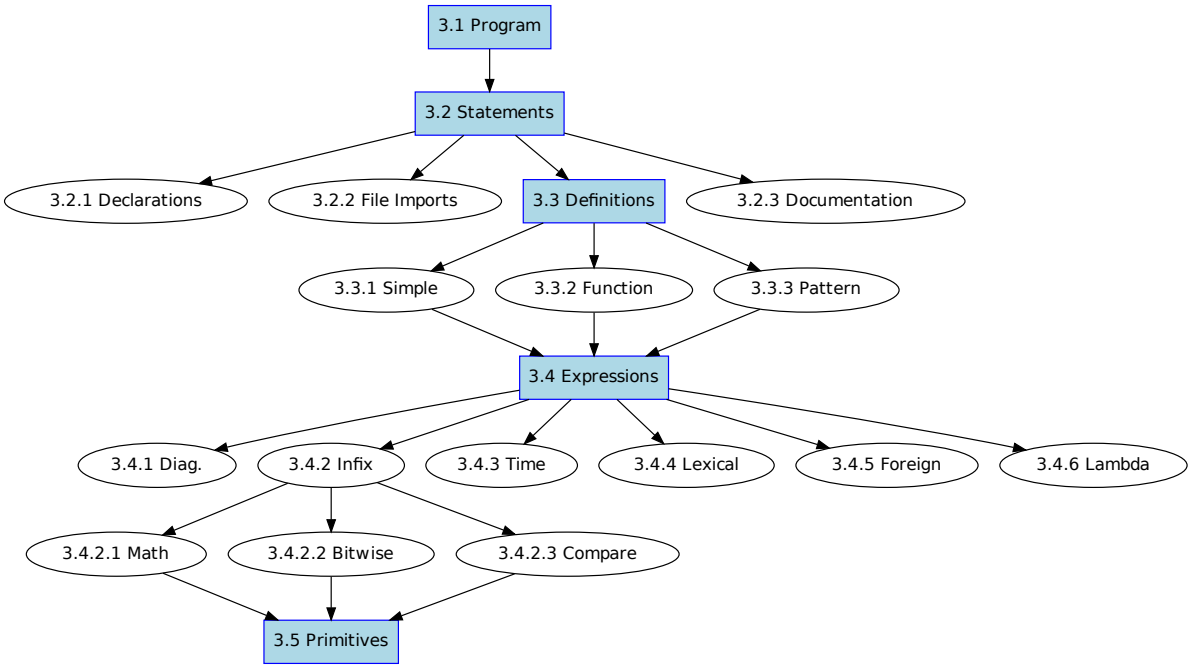


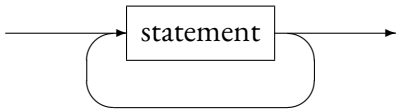
Figure 3.1: Overview of Faust syntax

As we will see, *definitions* and *expressions* play a central role.

3.1 FAUST program

A FAUST program is essentially a list of *statements*. These statements can be *declarations*, *imports*, *definitions*, and *documentation tags*, with optional C++-style (//... and /*...*/) comments.

program



3.1.1 A Simple Program

Here is a short FAUST program that implements a simple noise generator. It exhibits various kinds of statements: two *declarations*, an *import*, a *comment*, and a *definition*. We will look at *documentation* statements later on (3.2.3).

```
declare name        "noise";
declare copyright   "(c)GRAME 2006";

import("music.lib");

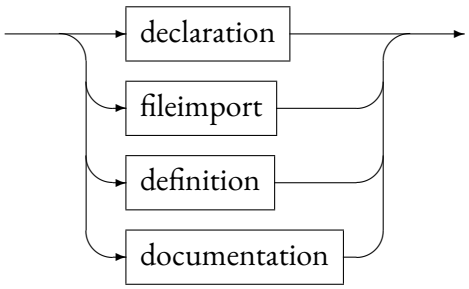
// noise level controlled by a slider
process = noise * vslider("volume", 0, 0, 1, 0.1);
```

The keyword `process` is the equivalent of `main` in C/C++. Any FAUST program, to be valid, must at least define `process`.

3.2 Statements

The *statements* of a FAUST program are of four kinds: *metadata declarations*, *file imports*, *definitions*, and *documentation*. All statements except documentation end with a semicolon (`;`).

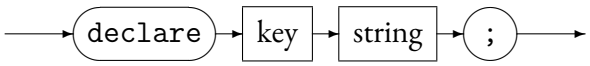
statement



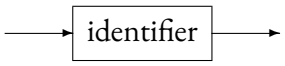
3.2.1 Declarations

Metadata declarations (for example `declare name "noise";`) are optional and typically used to document a FAUST project.

declaration



key



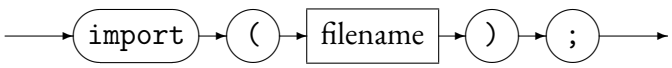
Contrary to regular comments, these declarations will appear in the C++ code generated by the compiler. A good practice is to start a FAUST program with some standard declarations:

```
declare name "MyProgram";
declare author "MySelf";
declare copyright "MyCompany";
declare version "1.00";
declare license "BSD";
```

3.2.2 Imports

File imports allow you to import definitions from other source files.

fileimport



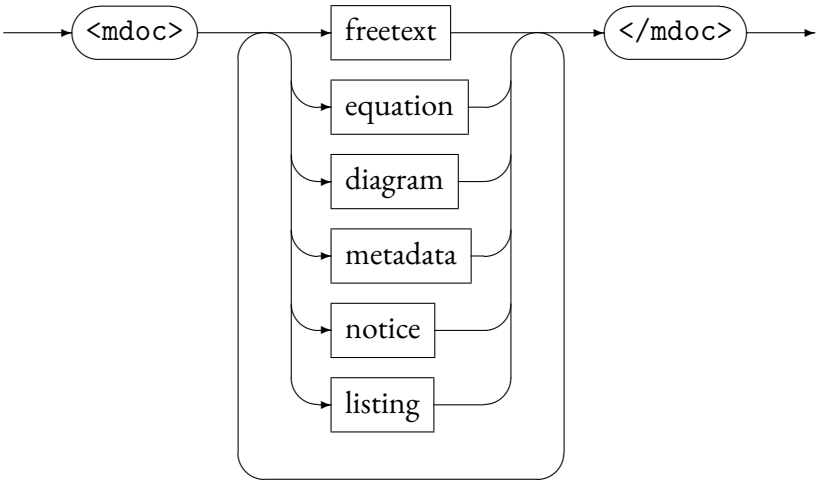
For example `import("maths.lib");` imports the definitions of the `maths.lib` library, a set of additional mathematical functions provided as foreign functions.

3.2.3 Documentation

Documentation statements are optional and typically used to control the generation of the mathematical documentation of a FAUST program. This documentation system is detailed in Chapter 12. In this section we essentially describe the syntax of documentation statements.

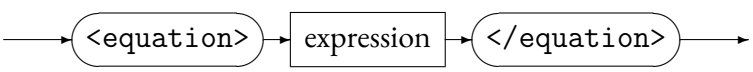
A documentation statement starts with an opening `<mdoc>` tag and ends with a closing `</mdoc>` tag. Free text content, typically in LaTeX format, can be placed between these two tags.

documentation



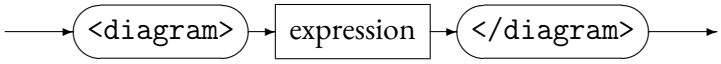
Moreover, optional sub-tags can be inserted in the text content itself to request the generation, at the insertion point, of mathematical *equations*, graphical *block-diagrams*, FAUST source code *listings*, and explanatory *notices*.

equation



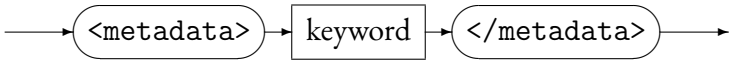
The generation of the mathematical equations of a FAUST expression can be requested by placing this expression between an opening `<equation>` and a closing `</equation>` tag. The expression is evaluated within the lexical context of the FAUST program.

diagram



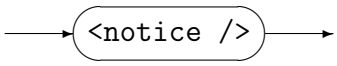
Similarly, the generation of the graphical block-diagram of a FAUST expression can be requested by placing this expression between an opening `<diagram>` and a closing `</diagram>` tag. The expression is evaluated within the lexical context of the FAUST program.

metadata



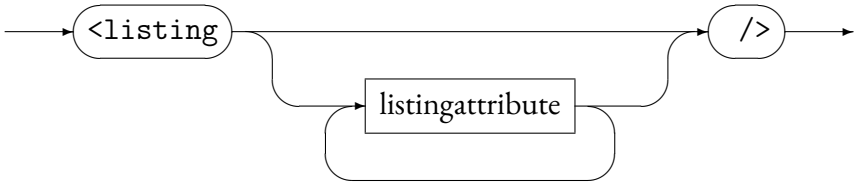
The `<metadata>` tags allow you to reference FAUST metadata (cf. declarations) by calling the corresponding keyword.

notice

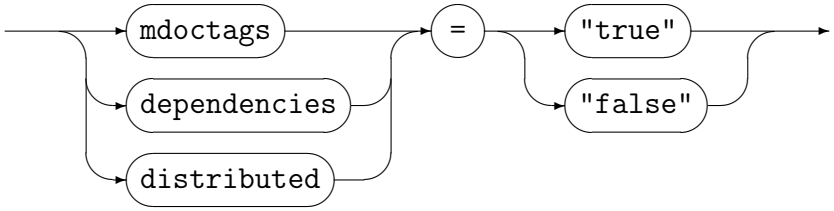


The `<notice />` empty-element tag is used to generate the conventions used in the mathematical equations.

listing



listingattribute



The `<listing />` empty-element tag is used to generate the listing of the FAUST program. Its three attributes `mdoctags`, `dependencies`, and `distributed` respectively enable or disable `<mdoc>` tags, other file dependencies, and the distribution of interleaved FAUST code between `<mdoc>` sections.

3.3 Definitions

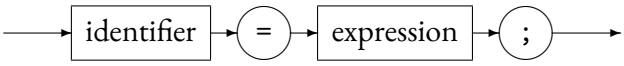
A *definition* associates an identifier with an expression it stands for.

Definitions are essentially a convenient shortcut that avoids typing long expressions. During compilation, more precisely during the evaluation stage, identifiers are replaced by their definitions. It is therefore always equivalent to use an identifier or its definition directly. Please note that multiple definitions of the same identifier are not allowed, unless it is a pattern-matching-based definition.

3.3.1 Simple Definitions

The syntax of a simple definition is:

definition



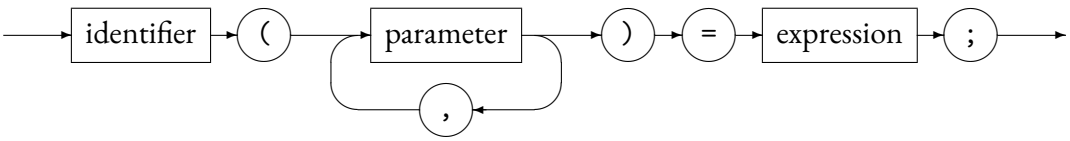
For example here is the definition of `random`, a simple pseudo-random number generator:

```
random = +(12345) ~ *(1103515245);
```

3.3.2 Function Definitions

Definitions with formal parameters correspond to function definitions.

fdefinition



For example the definition of `linear2db`, a function that converts linear values to decibels, is:

```
linear2db(x) = 20*log10(x);
```

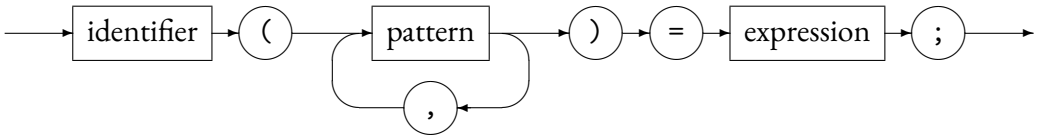
Please note that this notation is only a convenient alternative to the direct use of *lambda-abstractions* (also called anonymous functions). The following is an equivalent definition of `linear2db` using a lambda-abstraction:

```
linear2db = \(x).(20*log10(x));
```

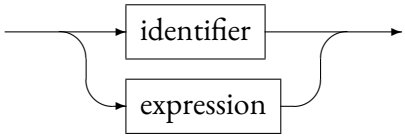
3.3.3 Definitions with pattern matching

Moreover, formal parameters can also be full expressions representing patterns.

pdefinition



pattern



This powerful mechanism allows you to algorithmically create and manipulate block-diagram expressions. Let's say that you want to describe a function to duplicate an expression several times in parallel:

```
duplicate(1,x) = x;
duplicate(n,x) = x, duplicate(n-1,x);
```

Please note that this last definition is a convenient alternative to the more verbose :

```
duplicate = case {
              (1,x) => x;
              (n,x) => duplicate(n-1,x);
            };
```

Here is another example to count the number of elements of a list. Please note that we simulate lists using parallel composition: (1,2,3,5,7,11). The main limitation of this approach is that there is no empty list. Moreover, lists of only one element are represented by this element:

```
count((x,xs)) = 1+count(xs);
count(x) = 1;
```

If we now write `count(duplicate(10,666))` the expression will evaluate to `10`.

Please note that the order of pattern matching rules matters. The more specific rules must precede the more general rules. When this order is not respected, as in :

```
count(x) = 1;
count((x,xs)) = 1+count(xs);
```

the first rule will always match and the second rule will never be called.

Please note that numeric arguments in pattern matching rules are typically *constant numerical expressions*, so they can be the result of more complex expressions involving computations done at compile time.

3.3.4 Variants

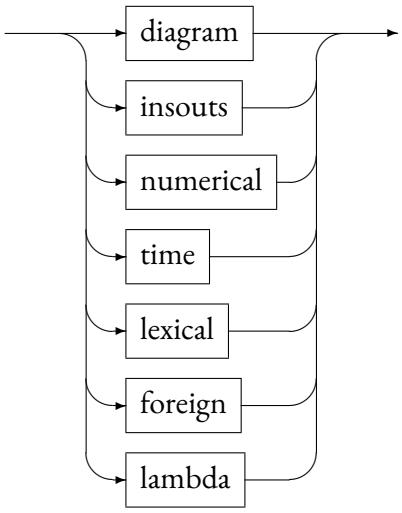
Some statements (imports, definitions) can be preceded by a *variantlist*, composed of variants that can be *singleprecision*, *doubleprecision*, *quadprecision*, or *fixedpointprecision*. This allows some imports and definitions to be effective only for one (or several) specific float-precision option in the compiler (that is, either `-single`, `-double`, `-quad`, or `-fx` respectively). A typical use case is the definition of floating point constants in the maths.lib library with the following lines:

```
singleprecision MAX = 3.402823466e+38;
doubleprecision MAX = 1.7976931348623158e+308;
```

3.4 Expressions

Despite its textual syntax, FAUST is conceptually a block-diagram language. FAUST expressions represent DSP block-diagrams and are assembled from primitive ones using various *composition* operations. More traditional *numerical* expressions in infix notation are also possible. Additionally, FAUST provides time-based expressions, such as delays, expressions related to lexical environments, expressions to interface with foreign functions, and lambda expressions.

expression



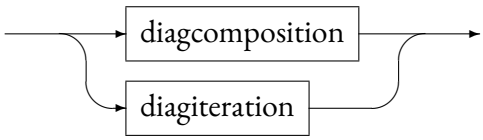
3.4.1 Constant Numerical Expressions

Some language primitives (like *rdtable*, *rwtable*, *hslider* etc.) take constant numbers as some of their parameters. This is the case also for expressions using pattern matching techniques. Those numbers can be directly given in the code, but can also be computed by more complex expressions which have to produce numbers known at compile time. We will refer to them as *constant numerical expressions* in the documentation.

3.4.2 Diagram Expressions

Diagram expressions are assembled from primitive ones using either binary composition operations or high-level iterative constructions.

diagramexp



SVG block-diagram representation

Since a FAUST expression *is* a block-diagram, the compiler can draw it. The `-svg` option generates an SVG drawing of the `process` expression (`-ps` generates the same drawing in PostScript):

```
faust -svg foo.dsp
```

The drawings are written in a `foo-svg/` folder, whose entry point is `process.svg`. The block-diagrams of this document are produced this way, and the Faust IDE displays them too. Figure 3.2 shows the drawing of the following program, which uses most of the graphical elements described in this section:

```
smooth(c) = *(1-c) : + ~ *(c);
gain = hslider("gain", 0.5, 0, 1, 0.01);
process = _ <: (smooth(0.9) * gain), (*(-1) : @(10));
```

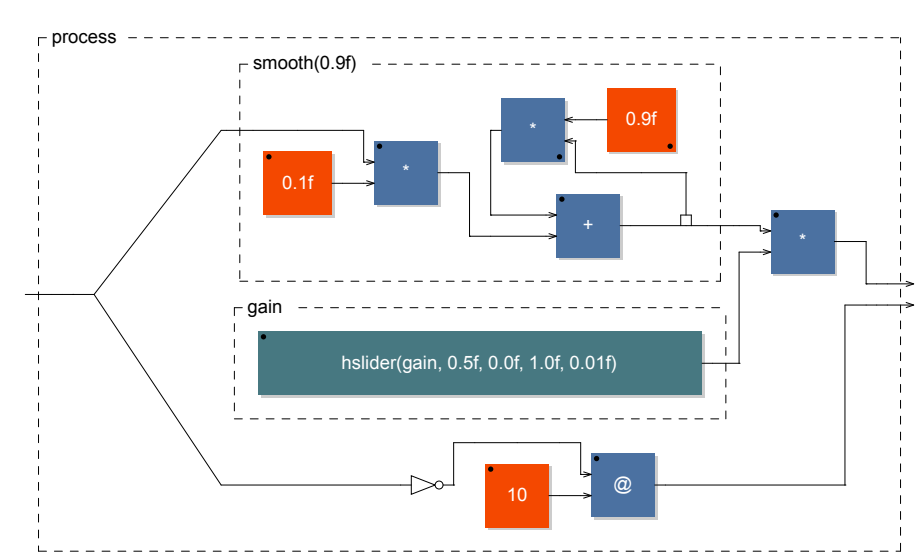


Figure 3.2: SVG drawing of the smooth and gain example

Reading a diagram

- Signals flow **from left to right**. The diagram is surrounded by a dashed frame labelled with the name of the drawn expression (here `process`). Its inputs enter on the left side of the frame and its outputs leave on the right side, marked with arrows.
- Each primitive block is a **colored box** labelled with its name. Its inputs are on its left side, marked by small arrows, and its outputs on its right side, both numbered from top to bottom. A **black dot** in a corner of the box marks its first input, like the notch of an integrated circuit: it is useful to read boxes drawn in the reverse direction (see recursion below).

- The color of a box gives its kind, as described in Table 3.1.
- **Wires** are plain lines: the identity function `_` is drawn as a simple wire, and the cut `!` as a wire that stops. In a split composition, the wires fan out from one output to several inputs. In a merge composition, several wires converge to one input, where their signals are summed. A `route` is drawn as its crossing wires (`-drf` draws a box around them).
- Each **named definition** used in the expression is surrounded by a dashed frame labelled with its name and the values of its arguments (here `smooth(0.9f)` and `gain`), unless it only contains wires. The groups of the user interface (`hgroup`, `vgroup`, `tgroup`) are drawn the same way.
- In a recursive composition `A~B`, the feedback block `B` is placed **above** `A` and drawn **from right to left**: its input arrows point to the left and its black dot is in the bottom right corner. Each output of `A` going back to `B` carries a small **square sign** on the wire, which stands for the implicit one-sample delay of the feedback loop. In Figure 3.2, `*(0.9f)` is the feedback block of `+`.

Appearance	Represents
orange box	a number, with no input and one output
blue box	a primitive, a foreign function or constant, a waveform
teal box	a user interface element, or a soundfile
green box `[n] x`	the parameter `x` of an unapplied abstraction
dark blue box	a folded sub-diagram, drawn in a separate file
white triangle	an inverter, i.e. `*(-1)`

Table 3.1: Boxes of the SVG block-diagrams

Note that the diagram shows the expression **after evaluation**: definitions, abstractions and pattern matching are applied, iterations are expanded, and constant numerical sub-expressions are already computed. With the `-double` option, numbers are displayed without the `f` suffix.

Folding large diagrams A diagram with more than 25 boxes (by default) is *folded*: instead of being expanded in place, its named sub-diagrams are drawn as dark blue boxes, and each of them is drawn in its own file of the `foo-svg/` folder. Clicking on a folded box opens its diagram, and clicking on the background of a sub-diagram goes back to the diagram that uses it. This makes it possible to browse large programs, including the code taken from the libraries, one level at a time. Figure 3.3 shows the top-level drawing of the following program:

```
import("stdfaust.lib");
process = dm.zita_light;
```

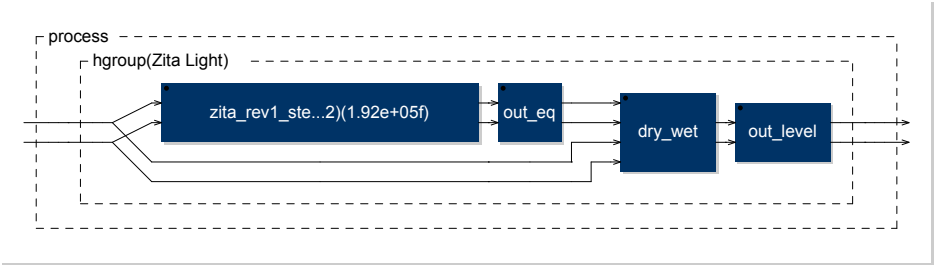


Figure 3.3: A folded block-diagram

Drawing options The compiler options of Table 3.2 control the drawing.

Option	Effect
`-svg`	draw the block-diagram in SVG files
`-ps`	draw the block-diagram in PostScript files
`-f` *n*	number of boxes beyond which a diagram is folded (default 25)
`-fc` *n*	minimal complexity of a folded sub-diagram (default 2)
`-sd`	simplify the block-diagram before drawing it
`-sn`	use simple names, without the values of the arguments
`-mns` *n*	maximal size of the labels (default 40 characters)
`-drf`	draw a box around the wires of a `route`
`-blur`	add a blurred shadow to the boxes
`-sc`	generate SVG files that scale to the size of their window

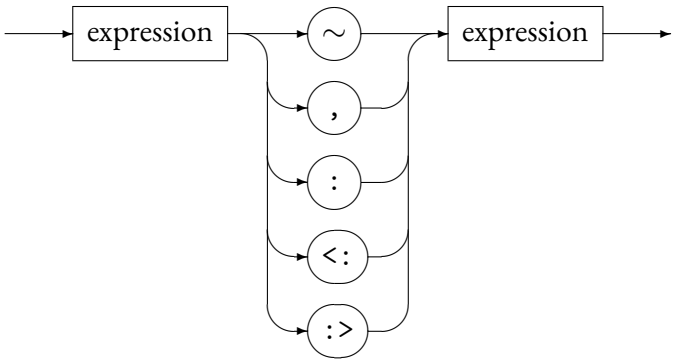
Table 3.2: Options controlling the SVG block-diagrams

For instance, with `-sd`, `10,20,30,40 :> _` is drawn as a single box `100`, and with `-sn`, the frame `smooth(0.9f)` is labelled `smooth`.

Diagram composition operations

Five binary *composition operations* are available to combine block-diagrams: *recursion*, *parallel*, *sequential*, *split* and *merge* composition. One can think of each of these composition operations as a particular way to connect two block diagrams.

diagcomposition



To describe precisely how these connections are made, we have to introduce some notation. The number of inputs and outputs of a block diagram A are denoted $\mathrm{inputs}(A)$ and $\mathrm{outputs}(A)$. The inputs and outputs themselves are respectively denoted $[0]A$, $[1]A$, $[2]A$, ... and $A[0]$, $A[1]$, $A[2]$, etc.

For each composition operation between two block-diagrams A and B we will describe the connections $A[i] \rightarrow [j]B$ that are created and the constraints on their relative numbers of inputs and outputs.

The priority and associativity of these five operations are given in Table 3.3. Please note that a higher priority value means a higher priority in the evaluation order. There is a companion Table 3.5 that gives the associativity of each numerical operator in infix expressions.

Syntax	**Pri.**	**Assoc.**	**Description**
expression ~ *expression*	4	left	recursive composition
expression , *expression*	3	right	parallel composition
expression : *expression*	2	right	sequential composition
expression <: *expression*	1	right	split composition
expression :> *expression*	1	right	merge composition

Table 3.3: Block-diagram composition operation priorities

Parallel Composition The *parallel composition* `(A,B)` (Figure 3.4) is probably the simplest one. It places the two block diagrams one on top of the other, without connections. The inputs of the resulting block diagram are the inputs of `A` and `B`. The outputs of the resulting block diagram are the outputs of `A` and `B`.

Parallel composition is an associative operation: `(A,(B,C))` and `((A,B),C)` are equivalent. When no parentheses are used, as in `A,B,C,D`, FAUST uses right associativity

and therefore builds internally the expression `(A,(B,(C,D)))`. This organization is important to know when using pattern-matching techniques on parallel compositions.

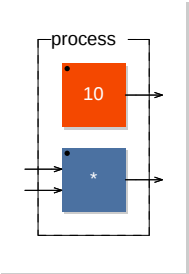


Figure 3.4: Example of parallel composition `(10,*)`

Sequential Composition The *sequential composition* `A:B` (Figure 3.5) expects:

$$\text{outputs}(A) = \text{inputs}(B) \tag{3.1}$$

It connects each output of A to the corresponding input of B:

$$A[i] \rightarrow [i]B \tag{3.2}$$

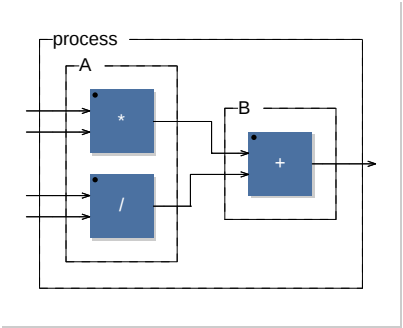


Figure 3.5: Example of sequential composition `((*,/):+)`

Sequential composition is an associative operation: `(A:(B:C))` and `((A:B):C)` are equivalent. When no parentheses are used, like in `A:B:C:D`, FAUST uses right associativity and therefore builds internally the expression `(A:(B:(C:D)))`.

Split Composition The *split composition* `A<:B` (Figure 3.6) operator is used to distribute the outputs of A to the inputs of B.

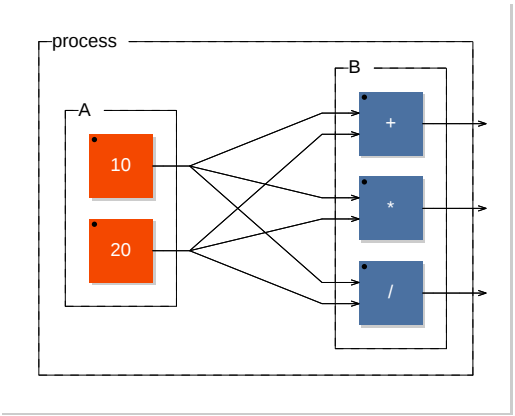


Figure 3.6: Example of split composition `((10,20)<: (+,*,/))`

For the operation to be valid, the number of inputs of B must be a multiple of the number of outputs of A:

$$\text{outputs}(A).k = \text{inputs}(B) \tag{3.3}$$

Each input i of B is connected to the output $i \bmod k$ of A:

$$A[i \bmod k] \rightarrow [i]B \tag{3.4}$$

Merge Composition The *merge composition* `A:>B` (Figure 3.7) is the dual of the *split composition*. The number of outputs of A must be a multiple of the number of inputs of B:

$$\text{outputs}(A) = k.\text{inputs}(B) \tag{3.5}$$

Each output i of A is connected to the input $i \bmod k$ of B:

$$A[i] \rightarrow [i \bmod k]B \tag{3.6}$$

The k incoming signals of an input of B are summed together.

Recursive Composition The *recursive composition* `A~B` (Figure 3.8) is used to create cycles in the block diagram in order to express recursive computations. It is the most complex operation in terms of connections.

To be applicable it requires that:

$$\text{outputs}(A) \geq \text{inputs}(B) \text{ and } \text{inputs}(A) \geq \text{outputs}(B) \tag{3.7}$$

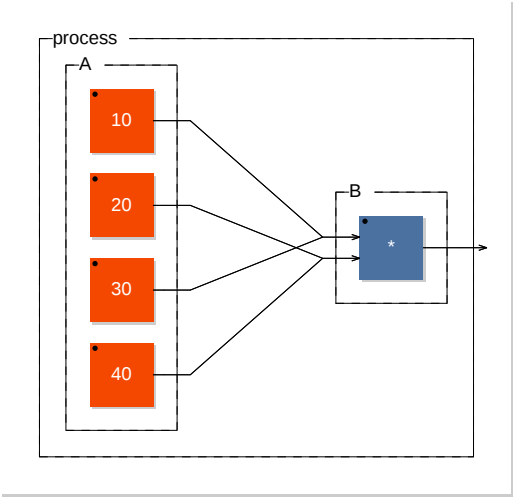


Figure 3.7: Example of merge composition `((10,20,30,40):> *)`

Each input of B is connected to the corresponding output of A via an implicit 1-sample delay:

$$A[i] \xrightarrow{Z^{-1}} [i]B \tag{3.8}$$

and each output of B is connected to the corresponding input of A:

$$B[i] \rightarrow [i]A \tag{3.9}$$

The inputs of the resulting block diagram are the remaining unconnected inputs of A. The outputs are all the outputs of A.

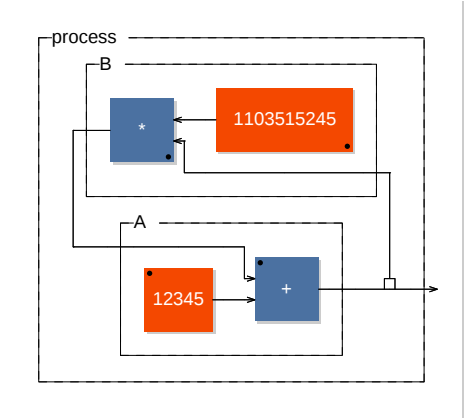
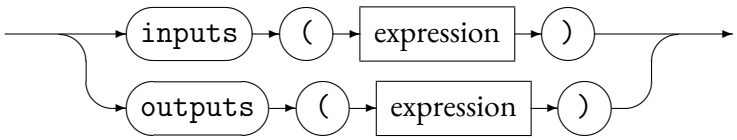


Figure 3.8: Example of recursive composition `+(12345)~ *(1103515245)`

Inputs and outputs of an expression

These two constructions can be used to know at compile time the number of inputs and outputs of any FAUST expression.

insouts



They are useful to define higher-order functions and to build algorithmically complex block diagrams. Here is an example to automatically reverse the order of the outputs of an expression.

```
Xo(expr) = expr <: par(i,n,selector(n-i-1,n))
         with { n=outputs(expr); };
```

And the inputs of an expression :

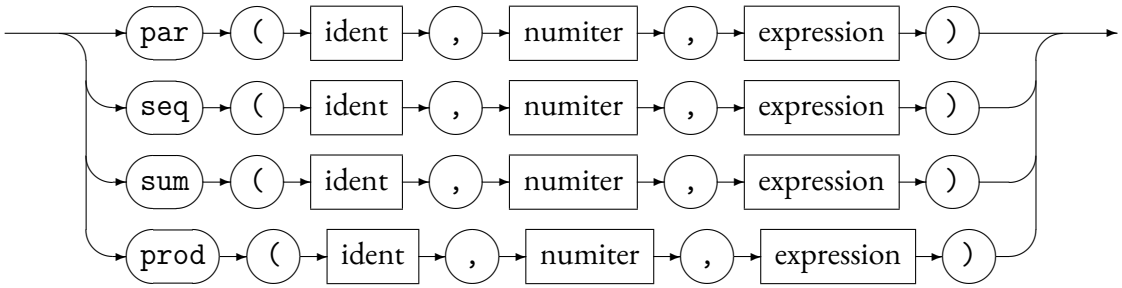
```
Xi(expr) = bus(n) <: par(i,n,selector(n-i-1,n)) : expr
         with { n=inputs(expr); };
```

For example `Xi(-)` will reverse the order of the two inputs of the substraction.

Iterations

Iterations are analogous to `for(...)` loops and provide a convenient way to automate some complex block-diagram constructions.

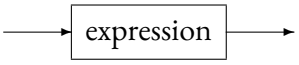
diagiteration



The following example shows the usage of `seq` to create a 10-bands filter:

```
process  =  seq(i, 10,
                vgroup("band %i",
                    bandfilter( 1000*(1+i) )
                )
            );
```

numiter



The number of iterations must be a constant expression.

3.4.3 Infix Notation

Infix notation is commonly used in mathematics. It consists in placing the operand between the arguments as in $2 + 3$

Besides its algebra-based core syntax, Faust provides some syntax extensions, in particular the familiar *infix notation*. For example if you want to multiply two numbers, let say `2` and `3`, you can write directly `2*3` instead of the equivalent core-syntax expression `2,3:*`.

The *infix notation* is not limited to numbers or numerical expressions. Arbitrary expressions `A` and `B` can be used, provided that `A,B` has exactly two outputs. For example you can write `_/2`. It is equivalent to `_,2:/` which divides the incoming signal by `2`.

Examples of equivalences are given table 3.4.

`2-3`	≡	`2,3 : -`
`2*3`	≡	`2,3 : *`
`_@7`	≡	`_,7 : @`
`_/2`	≡	`_,2 : /`
`A<B`	≡	`A,B : <`

Table 3.4: Infix and core syntax equivalences

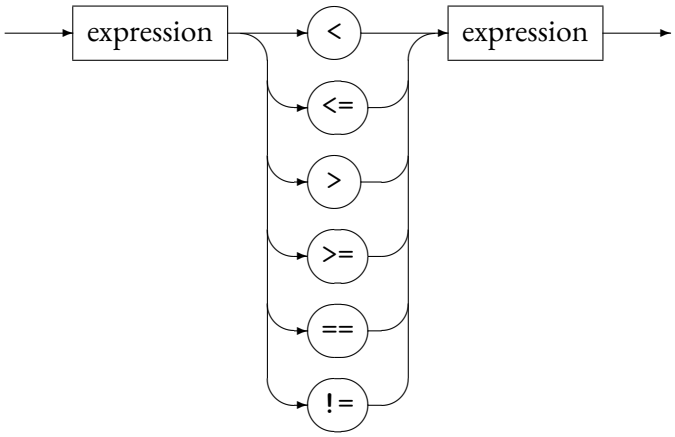
In case of doubts on the meaning of an infix expression, for example `_*_`, it is useful to translate it to its core syntax equivalent, here `_,_:*`, which is equivalent to `*`.

Built-in primitives that can be used in infix notation are called *infix operators* and are listed in this section. Please note that a more detailed description of these operators is available section 3.5.

Comparison Operators

Comparison operators compare two signals and produce a signal that is 1 when the comparison is true and 0 when the comparison is false.

comparison



The priority and associativity of the comparison operators is given table 3.5.

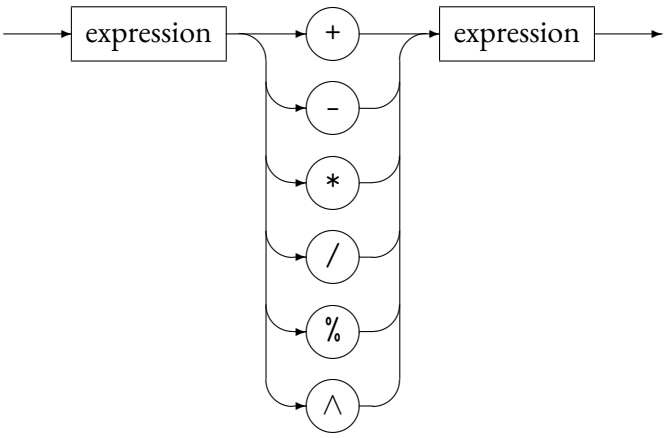
Syntax	Pri.	Assoc.	Description
expression < *expression*	5	left	less than
expression <= *expression*	5	left	less or equal
expression != *expression*	5	left	different
expression >= *expression*	5	left	greater or equal
expression > *expression*	5	left	greater than

Table 3.5: Comparison operators priorities in infix expressions

Math Operators

Math operators combine two signals and produce a resulting signal by applying a numerical operation on each sample.

math



The priority and associativity of the math operators is given table 3.6.

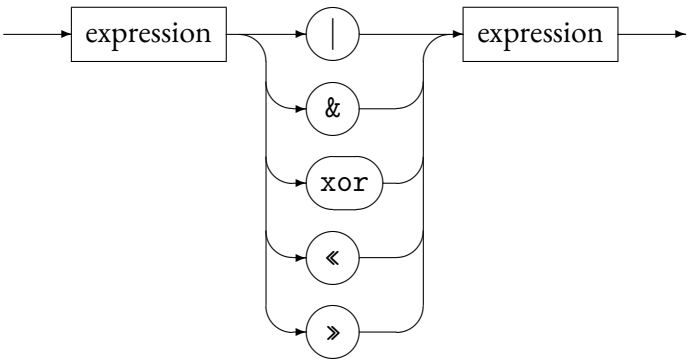
Syntax	Pri.	Assoc.	Description
expression + *expression*	6	left	addition
expression - *expression*	6	left	subtraction
expression * *expression*	7	left	multiplication
expression / *expression*	7	left	division
expression % *expression*	7	left	modulo
expression ^ *expression*	8	left	power

Table 3.6: Math operators priorities in infix expressions

Bitwise operators

Bitwise operators combine two signals and produce a resulting signal by applying a bitwise operation on each sample.

bitwise



The priority and associativity of the bitwise operators is given table 3.7.

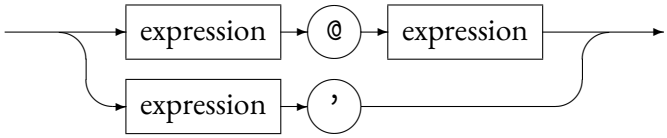
Syntax	Pri.	Assoc.	Description
expression `|` *expression*	6	left	bitwise or
expression `&` *expression*	7	left	bitwise and
expression `xor` *expression*	7	left	bitwise xor
expression `«` *expression*	7	left	bitwise left shift
expression `»` *expression*	7	left	bitwise right shift

Table 3.7: Bitwise operators priorities in infix expressions

Delay operators

Delay operators combine two signals and produce a resulting signal by applying a bitwise operation on each sample.

delay



The delay operator `@` allows to delay left handside expression by the amount defined by the right handside expression. The unary operator `'` delays the left handside expression by one sample.

The priority and associativity of the delay operators is given table 3.8.

Syntax	Pri.	Assoc.	Description
expression `@` *expression*	9	left	variable delay
expression `'`	10	left	one-sample delay

Table 3.8: Delay operators priorities in infix expressions

3.4.4 Prefix Notation

Beside *infix notation*, it is also possible to use *prefix notation*. The *prefix notation* is the usual mathematical notation for functions $f(x, y, z, \ldots)$, but extended to *infix operators*.

It consists in first having the operator, for example `/`, followed by its arguments between parentheses: `/(2,3)` (see table 3.9).

`*(2,3)`	≡	`2,3 : *`
`@(_,7)`	≡	`_,7 : @`
`/(_,2)`	≡	`_,2 : /`
`<(A,B)`	≡	`A,B : <`

Table 3.9: Prefix to core syntax translation rules

Partial Application

The *partial application* notation is a variant of *prefix notation* in which not all arguments are given. For instance `/(2)` (divide by 2), `^(3)` (rise to the cube) and `@(512)` (delay by 512 samples) are examples of partial applications where only one argument is given. The result of a partial application is a function that "waits" for the remaining arguments.

When doing partial application with an *infix operator*, it is important to note that the supplied argument is not the first argument, but always the second one, as summarized table 3.10.

`+(C)`	≡	`_,C:*`
`-(C)`	≡	`_,C:-`
`<(C)`	≡	`_,C:<`
`/(C)`	≡	`_,C:/`

Table 3.10: Partial application of infix operators

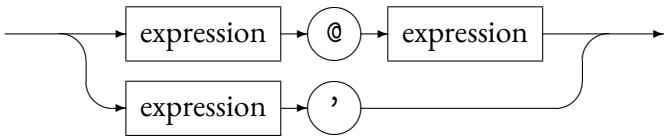
For commutative operations that doesn't matter. But for non-commutative ones, it is more "natural" to fix the second argument. We use divide by 2 (`/(2)`) or rise to the cube (`^(3)`) more often than the other way around.

Please note that this rule only applies to infix operators, not other primitives or functions. If you partially apply a regular function to a single argument, it will correspond to the first parameter.

3.4.5 Time expressions

Time expressions are used to express delays. The notation `X@10` represent the signal `X` delayed by 10 samples. The notation `X'` represent the signal X delayed by one sample and is therefore equivalent to `X@1`.

time



The delay (automatically promoted to *int*) don't have to be fixed, but it must be positive and bounded. The values of a slider are perfectly acceptable as in the following example:

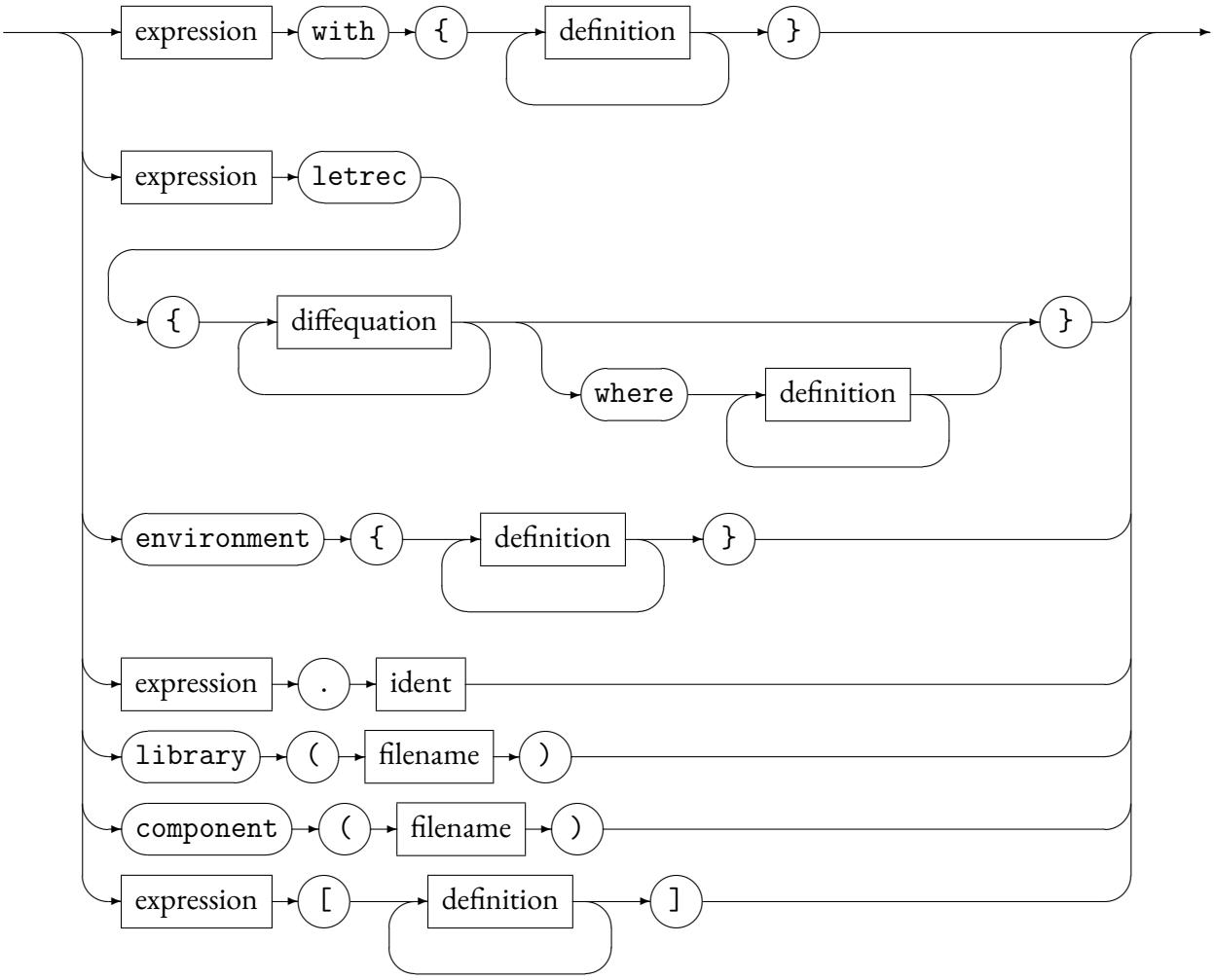
```
process = _ @ hslider("delay",0, 0, 100, 1);
```

3.4.6 Environment expressions

FAUST is a lexically scoped language. The meaning of a FAUST expression is determined by its context of definition (its lexical environment) and not by its context of use.

To keep their original meaning, FAUST expressions are bounded to their lexical environment in structures called *closures*. The following constructions allow to explicitly create and access such environments. Moreover they provide powerful means to reuse existing code and promote modular design.

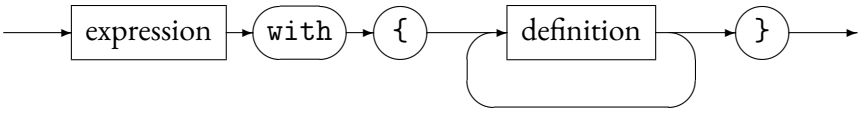
envexp



With

The `with` construction allows to specify a *local environment*, a private list of definition that will be used to evaluate the left hand expression

withexpression



In the following example :

```
pink = f : + ~ g with {
    f(x) = 0.04957526213389*x
         - 0.06305581334498*x'
         + 0.01483220320740*x'';
    g(x) = 1.80116083982126*x
         - 0.80257737639225*x';
};
```

the definitions of `f(x)` and `g(x)` are local to `f : + ~ g`.

Please note that `with` is left associative and has the lowest priority:

- `f : + ~ g with {...}` is equivalent to `(f : + ~ g) with {...}`.
- `f : + ~ g with {...} with {...}` is equivalent to `((f : + ~ g) with {...}) with {...}`.

Letrec

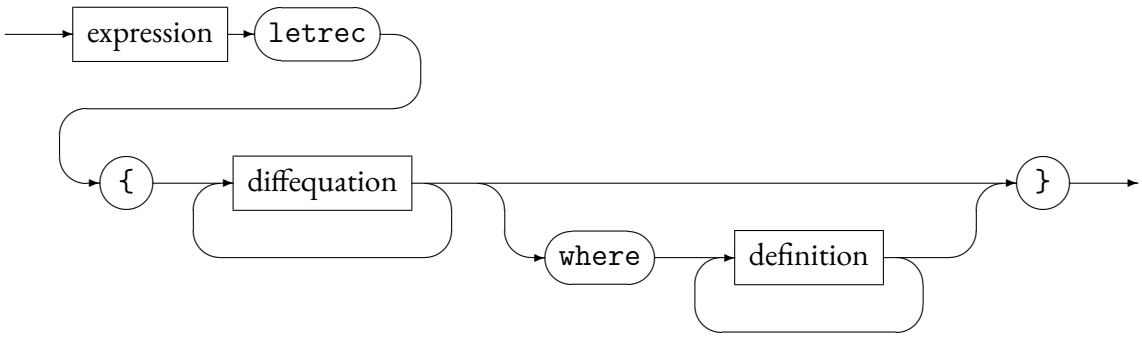
The `letrec` construction is somehow similar to `with`, but for *difference equations* instead of regular definitions. It allows to easily express groups of mutually recursive signals, for example:

$$\begin{aligned} x(t) &= y(t-1) + 10; \\ y(t) &= x(t-1) - 1; \end{aligned}$$

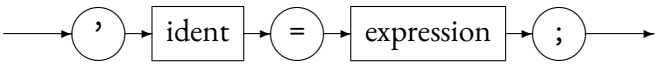
as `E letrec { 'x = y+10; 'y = x-1; }`

The syntax is defined by the following rules:

letrecexpression



diffequation



Please remarks the special notation `'x=y+10` instead of `x=y'+10`. It makes syntactically impossible to write non-sensical equations like `x=x+1`.

Here is a more involved example. Let say we want to define an envelop generator with an attack time, a release time and a gate signal. A possible definition is the following:

```
ar(a,r,g) = v
  letrec {
    'n = (n+1) * (g<=g');
    'v = max(0, v + (n<a)/a - (n>=a)/r) * (g<=g');
  };
```

With the following semantics for $n(t)$ and $v(t)$:

$$\begin{aligned} n(t) &= (n(t-1)+1) * (g(t) <= g(t-1)) \\ v(t) &= max(0, v(t-1) + (n(t-1) < a(t))/a(t) \\ &- (n(t-1) >= a(t))/r(t)) * (g(t) <= g(t-1)) \end{aligned}$$

In order to factor some expressions common to several recursive definitions, we can use the clause `where` followed by one or more definitions. These definitions will only be visible to the recursive equations of the `letrec`, but not to the outside world, unlike the recursive definitions themselves.

For instance in the previous example we can factorize `(g<=g)` leading to the following expression:

```
    ar(a,r,g) = v
      letrec {
        'n = (n+1) * c;
        'v = max(0, v + (n<a)/a - (n>=a)/r) * c;
        where
         c = g<=g';
      };
```

Please note that `letrec` is essentially syntactic sugar.

Here is an example of 'letrec':

```
    x,y  letrec {
           x = defx;
           y = defy;
```

```
        z = defz;
      where
        f = deff;
        g = defg;
    };
```

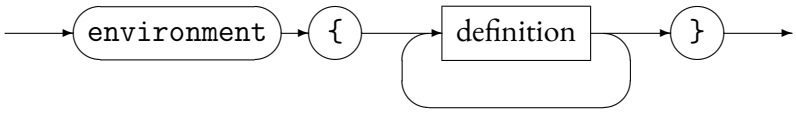
and its translation as done internally by the compiler:

```
  x,y  with {
       x = BODY : _,!,!;
       y = BODY : !,_,!;
       z = BODY : !,!,_;
       BODY = \(x,y,z).((defx,defy,defz) with {f=deff;
          g=defg;}) ~ (_,_,_);
    };
```

Environment

The `environment` construction allows to create an explicit environment. It is like a `with`, but without the left hand expression. It is a convenient way to group together related definitions, to isolate groups of definitions and to create a name space hierarchy.

environment



In the following example an `environment` construction is used to group together some constant definitions :

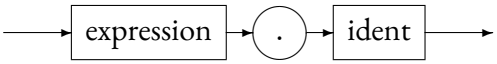
```
constant = environment {
    pi = 3.14159;
    e = 2.718;
    ...
};
```

The `.` construction allows to access the definitions of an environment (see next paragraph).

Access

Definitions inside an environment can be accessed using the '.' construction.

access



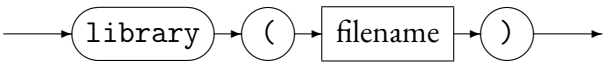
For example `constant.pi` refers to the definition of `pi` in the above `constant` environment.

Please note that environment don't have to be named. We could have written directly `environment{pi = 3.14159; e = 2.718; ... }.pi`

Library

The `library` construct allows to create an environment by reading the definitions from a file.

library



For example `library("miscfilter.lib")` represents the environment obtained by reading the file "miscfilter.lib". It works like `import("miscfilter.lib")` but all the read definitions are stored in a new separate lexical environment. Individual definitions can be accessed as described in the previous paragraph. For example `library("miscfilter.lib").lowpass` denotes the function `lowpass` as defined in the file `"miscfilter.lib"`.

To avoid name conflicts when importing libraries it is recommended to prefer `library` to `import`. So instead of :

```
import("miscfilter.lib");
  ...
...lowpass....
    ...
};
```

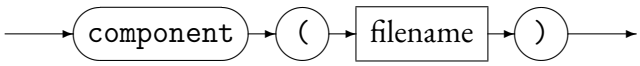
the following will ensure an absence of conflicts :

```
fl = library("miscfilter.lib");
  ...
...fl.lowpass....
    ...
};
```

Component

The `component(...)` construction allows to reuse a full FAUST program as a simple expression.

component



For example `component("freeverb.dsp")` denotes the signal processor defined in file "freeverb.dsp".

Components can be used within expressions like in:

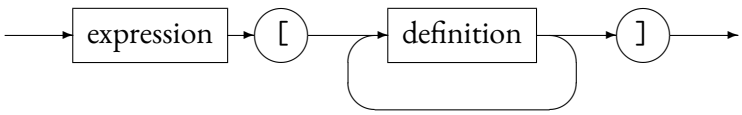
```
...component("karplus32.dsp"):component("freeverb.dsp")
   ...
```

Please note that `component("freeverb.dsp")` is equivalent to `library("freeverb.dsp").process`.

Explicit substitution

Explicit substitution can be used to customize a component or any expression with a lexical environment by replacing some of its internal definitions, without having to modify it.

explicitsubst



For example we can create a customized version of `component("freeverb.dsp")`, with a different definition of `foo(x)`, by writing :

```
...component("freeverb.dsp")[foo(x) = ...;]...
};
```

Combined with the `environment` construct, *explicit substitution* can be used to express a form of *default parameter values* or *keyword arguments* for function definitions:

```
import("stdfaust.lib");

// Filter parameters in an environment
fPar = environment {
    freq = 1000;
    Q = 0.707;
    gain = 1.0;
};

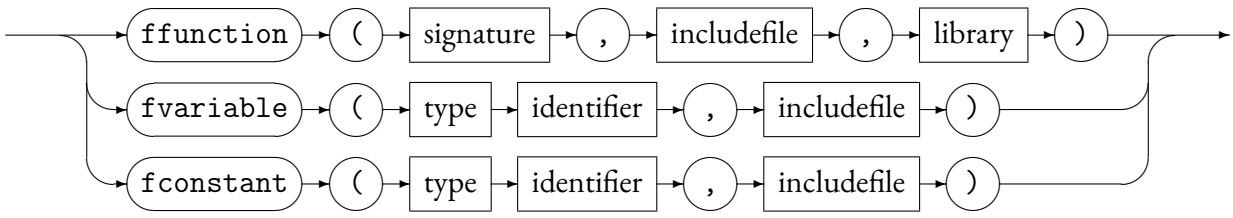
// Filter parametrized with an environment
filter(p) = fi.resonlp(p.freq, p.Q, p.gain);

// Usage examples
// all defaults
process = no.noise : filter(fPar);
 // override freq
process = no.noise : filter(fPar[freq = 2000;]);
 // override freq and Q
process = no.noise : filter(fPar[freq = 200; Q = 2.0;]);
```

3.4.7 Foreign expressions

Reference to external (foreign) C *functions*, *variables* and *constants* can be introduced using the *foreign expressions* mechanism.

foreignexp



Foreign function declaration

An external C function is declared by indicating its name and signature as well as the required include file. The file `"maths.lib"` of the FAUST distribution contains several

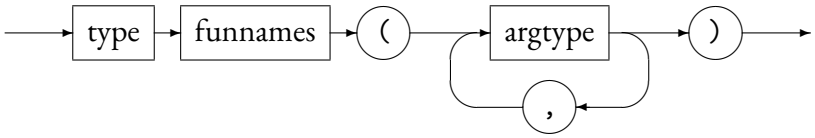
foreign function definitions, for example the inverse hyperbolic sine function `asinh` is defined as follows

```
asinh = ffunction(float asinhf|asinh|asinhl|asinfx(
   float), <math.h>, "");
```

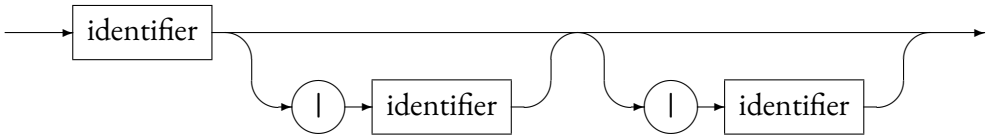
The signature part of a foreign function, `float asinhf|asinh|asinhl|asinfx(float )` in our previous example, describes the prototype of the C function: its return type, function names and list of parameter types. Because the name of the foreign function can possibly depend on the floating point precision in use (float, double, quad and fixed-point), it is possible to give a different function name for each floating point precision using a signature with up to four function names.

In our example, the `asinh` function is called `asinhf` in single precision, `asinh` in double precision, `asinhl` in quad precision, and `asinhfx` in fixed-point precision. This is why the four names are provided in the signature.

signature



funnames

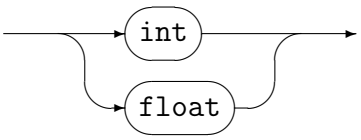


Foreign functions generally expect a precise type: (*int* or *float*) for their parameters. Note that currently only numerical functions involving scalar parameters are allowed. No vectors, tables or data structures can be passed as parameters or returned. During the compilation if the type of an argument is not the same as the type of the parameter, it is automatically converted to the expected one.

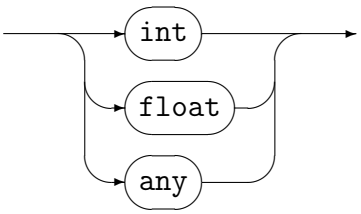
Some foreign functions are polymorphic and can accept either *int* or *float* arguments. In this case, the polymorphism can be indicated by using the type *any* instead or int or float. Here is as an example the C function *sizeof* that returns the size of its argument:

```
sizeof = ffunction(int sizeof(any), "","");
```

type



argtype



Foreign functions with input parameters are considered pure math functions. They are therefore considered free of side effects and called only when their parameters change (that is at the rate of the fastest parameter).

Exceptions are functions with no input parameters. A typical example is the C `rand()` function. In this case the compiler generates code to call the function at sample rate.

Foreign variables and constants

External variables and constants can also be declared with a similar syntax. In the same `"maths.lib"` file, the definition of the sampling rate constant `SR` and the definition of the block-size variable `BS` can be found :

```
SR     = min(192000.0,
             max(1.0,
               fconstant(int fSamplingFreq, <math.h>)));
BS     = fvariable(int count, <math.h>);
```

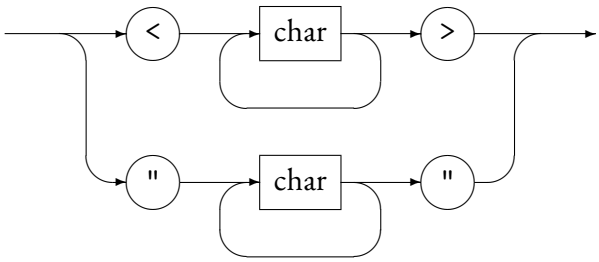
Foreign constants are not supposed to vary. Therefore expressions involving only foreign constants are computed once, during the initialization period.

Foreign variables are considered to vary at block speed. This means that expressions depending of external variables are computed every block.

Include file

In declaring foreign functions one has also to specify the include file. It allows the FAUST compiler to add the corresponding `#include...` in the generated code.

includefile



Library file

In declaring foreign functions one can possibly specify the library where the actual code is located. It allows the FAUST compiler to (possibly) automatically link the library. Note that this feature is only used with the LLVM backend in 'libfaust' dynamic library model.

Availability of Foreign Functions, Variables, and Constants

Foreign functions, variables, and constants can only be used in FAUST when the target backend provides a mechanism to access their definitions and compiled code. In practice, this means:

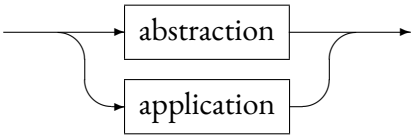
- **C/C++ backends**: full support, since external symbols can be resolved at link time.
- **LLVM backend**: partial support, as external calls can be lowered to LLVM instructions, or using libraries compiled to LLVM bitcode and linked in the final binary, but with more restrictions.
- **Other backends (e.g., Julia, WebAssembly, etc.)**: foreign expressions are generally not supported, and any attempt to use them will cause compilation to fail.

This limitation arises because many backends operate in isolated or sandboxed environments, where external code or symbols cannot be safely linked. Developers should therefore ensure that their DSP code either avoids foreign expressions or is explicitly targeted to backends that support them.

3.4.8 Applications and Abstractions

Abstractions and *applications* are fundamental programming constructions directly inspired by the Lambda-Calculus. These constructions provide powerful ways to describe and transform block-diagrams algorithmically.

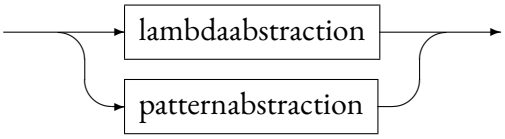
progexp



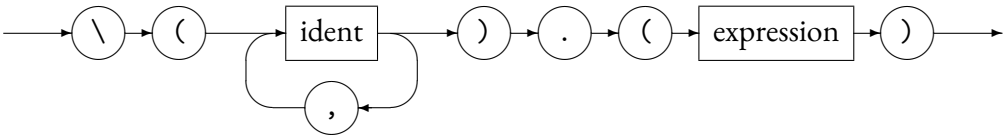
Abstractions

Abstractions correspond to functions definitions and allow to generalize a block-diagram by *making variable* some of its parts.

abstraction



lambdaabstraction



Let's say you want to transform a stereo reverb, `freeverb` for instance, into a mono effect. You can write the following expression:

```
_ <: freeverb :> _
```

The incoming mono signal is splitted to feed the two input channels of the reverb, while the two output channels of the reverb are mixed together to produce the resulting mono output.

Imagine now that you are interested in transforming other stereo effects. It can be interesting to generalize this principle by making `freeverb` a variable:

```
\(freeverb).(_ <: freeverb :> _)
```

The resulting abstraction can then be applied to transform other effects. Note that if `freeverb` is a perfectly valid variable name, a more neutral name would probably be easier to read like:

```
\(fx).(_ <: fx :> _)
```

Moreover it could be convenient to give a name to this abstraction:

```
mono = \(fx).(_ <: fx :> _);
```

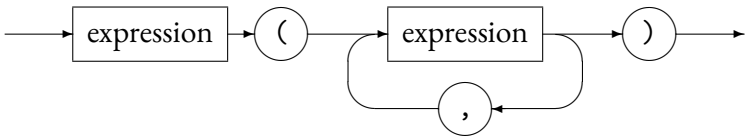
Or even use a more traditional, but equivalent, notation:

```
mono(fx) = _ <: fx :> _;
```

Applications

Applications correspond to function calls and allow to replace the variable parts of an abstraction with the specified arguments.

application



For example you can apply the previous abstraction to transform your stereo harmonizer:

```
mono(harmonizer)
```

The compiler will start by replacing `mono` by its definition:

```
\(fx).(_ <: fx :> _)(harmonizer)
```

Whenever the FAUST compiler find an application of an abstraction it replaces the *variable part* with the argument. The resulting expression is as expected:

Replacing the *variable part* with the argument is called β-reduction in Lambda-Calculus

```
(_ <: harmonizer :> _)
```

Note that the arguments given to the primitive or function in applications are reduced to their *block normal form* (that is the flat equivalent block) before the actual application. Thus if the number of outputs of the argument block does not mach the needed number of arguments, the application will be treated as *partial application* and the missing arguments will be replaced by one or several `_` (to complete the number of missing arguments).

Unapplied abstractions

Usually, lambda abstractions are supposed to be applied on arguments, using beta-reduction in Lambda-Calculus. Functional languages generally treat them as first-class values[1] which give these languages high-order programming capabilities.

Another way of looking at abstractions in Faust is as a means of routing or placing blocks that are given as parameters. For example, the following abstraction `repeat(fx)= fx : fx;` could be used to duplicate an effect and route input signals to be successively processed by that effect:

```
import("stdfaust.lib");
repeat(fx) = fx : fx;
process = repeat(dm.zita_light);
```

In Faust, a proper semantic has been given to *unapplied abstractions*: when a lambda-abstraction is not applied to parameters, it indicates *how to route input signals*. This is a convenient way to work with signals by *explicitly naming them*, to be used in the lambda abstraction body *with their parameter name*.

For instance a stereo crossing block written in the core syntax:

```
process = _,_ <: !,_,_,!;
```

can be simply defined as:

```
process = \(x,y).(y,x);
```

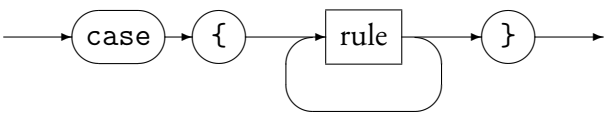
which is actually equivalent to:

```
process(x,y) = y,x;
```

Pattern Matching

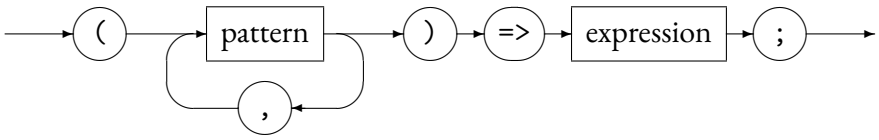
Pattern matching rules provide an effective way to analyze and transform block-diagrams algorithmically.

patternabstraction

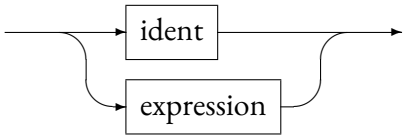


[1] https://en.wikipedia.org/wiki/First-class_function

Rule



Pattern



For example `case{ (x:y)=> y:x; (x)=> x; }` contains two rules. The first one will match a sequential expression and invert the two part. The second one will match all remaining expressions and leave it untouched. Therefore the application:

```
case{(x:y) => y:x; (x) => x;}(freeverb:harmonizer)
```

will produce:

```
    (harmonizer:freeverb)
```

Please note that patterns are evaluated before the pattern matching operation. Therefore only variables that appear free in the pattern are binding variables during pattern matching.

3.4.9 Equivalent expressions

When programming in Faust, you may reuse the same expression in multiple places within your code. If the compiler can prove that these expressions are used in identical contexts, it will automatically share the computation between the different occurrences. This is particularly useful for expressions involving User Interface (UI) widgets like buttons or sliders. In these cases, equivalence is determined by the widget's pathname, which is constructed by concatenating the labels of all surrounding groups. If the pathnames are identical, the compiler will treat the expressions as equivalent, allowing the same control (e.g., a button or slider) to affect different parts of the program.

To share UI widgets deeply nested within the hierarchical structure, you may need to "move them up" in the hierarchy by adjusting their pathnames. This can be done using syntax like the following:

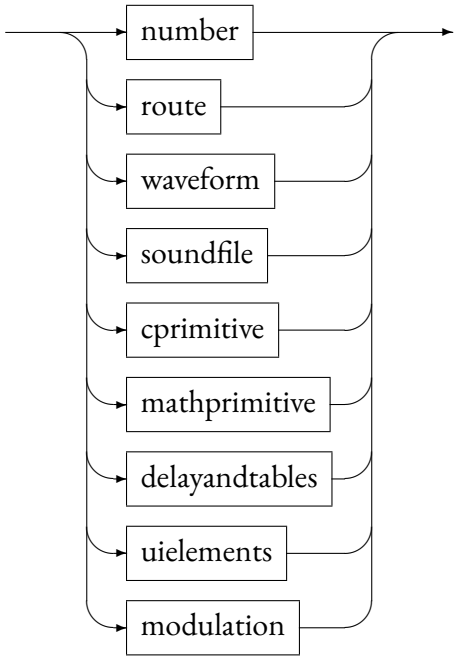
```
    hslider("../volume",...)
```

By doing so, the widget's pathname is made syntactically equivalent across different parts of the program, enabling shared control.

3.5 Primitives

The primitive signal processing operations represent the built-in functionalities of FAUST, that is the atomic operations on signals provided by the language. All these primitives denote *signal processors*, functions transforming *input signals* into *output signals*.

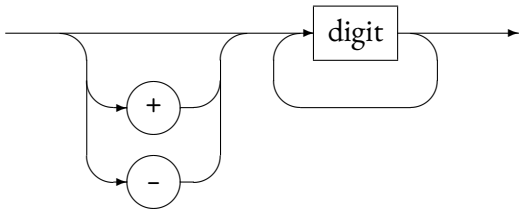
primitive



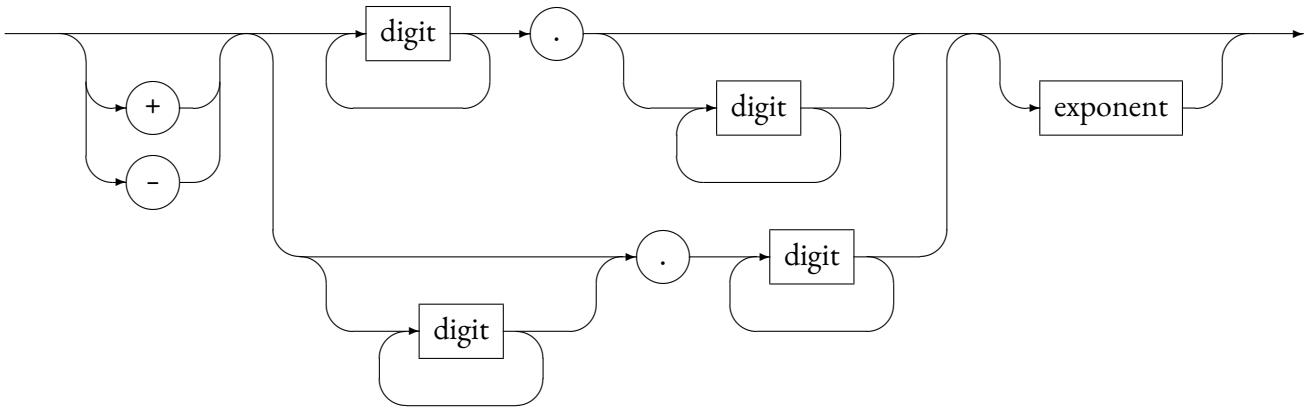
3.5.1 Numbers

FAUST considers two types of numbers: *integers* and *floats*. Integers are implemented as signed 32-bit integers, and floats are implemented with single, double, or extended precision depending on the compiler options. Floats are available in decimal or scientific notation.

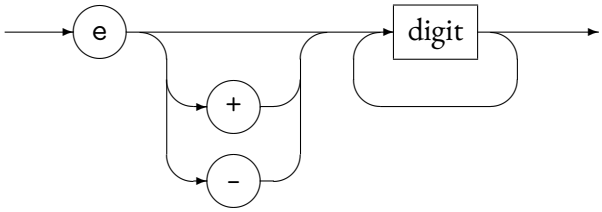
int



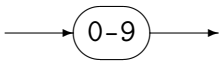
float



exponent



digit



Like any other FAUST expression, numbers are signal processors. For example the number 0.95 is a signal processor of type $\mathbb{S}^0 \to \mathbb{S}^1$ that transforms an empty tuple of signals $()$ into a 1-tuple of signals (y) such that $\forall t \in \mathbb{N}, y(t) = 0.95$.

Operations on *integer* numbers follow the standard C semantics for `+, -, *` operations and can overflow if the result cannot be represented as a 32-bit integer. The `/` operation is treated separately and casts both of its arguments to floats before performing the division, and thus the result takes the float type.

3.5.2 Route Primitive

The `route` primitive facilitates the routing of signals in FAUST. It has the following syntax:

```
route(A,B,a,b,c,d,...)
route(A,B,(a,b),(c,d),...)
```

where:

- `A` is the number of input signals, as an integer *constant numerical expression*, automatically promoted to *int*
- `B` is the number of output signals, as an integer *constant numerical expression*, automatically promoted to *int*
- `a,b` or `(a,b)` is an input/output pair, as integers *constant numerical expressions*, automatically promoted to *int*

Inputs are numbered from `1` to `A` and outputs are numbered from `1` to `B`. There can be any number of input/output pairs after the declaration of `A` and `B`.

For example, crossing two signals can be carried out with:

```
process = route(2,2,1,2,2,1);
```

In that case, `route` has 2 inputs and 2 outputs. The first input (1) is connected to the second output (2) and the second input (2) is connected to the first output (1).

Note that parentheses can optionally be used to define a pair, so the previous example can also be written as:

```
process = route(2,2,(1,2),(2,1));
```

More complex expressions can be written using algorithmic constructions, like the following one to cross N signals:

```
// cross 10 signals:
// input 0 -> output 10,
// input 1 -> output 9,
// ...,
// input 9 -> output 0

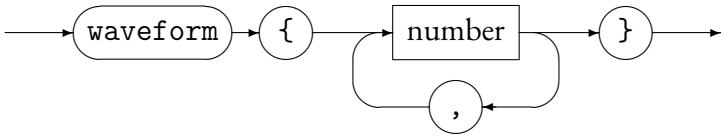
N = 10;
r = route(N,N,par(i,N,(i+1,N-i)));

process = r;
```

3.5.3 Waveform Primitive

A waveform is a fixed periodic signal defined by a list of samples as literal numbers. A waveform has two outputs. The first output is constant and indicates the size (number of samples) of the period. The second output is the periodic signal itself.

waveform



For example `waveform {0,1,2,3}` produces two outputs, the constant signal 4 and the periodic signal `0,1,2,3,0,1,2,3,0,1`....

Please note that `waveform` works nicely with `rdtable`. Its first output, known at compile time, gives the size of the table, while the second signal gives the content of the table. Here is an example:

```
process = waveform {10,20,30,40,50,60,70}, %(7)~+(3) :
   rdtable;
```

3.5.4 Soundfile Primitive

The `soundfile("label[url:\{path1';'path2';'path3';}]", n)'` primitive allows access to a list of externally defined sound resources, described as a label followed by the list of their filenames or complete paths (possibly using the '`%i`' syntax, as in the label part). The simplified syntax `soundfile("label[url:path]", n)` or `soundfile("label", n)` (where the label is used as the soundfile path) allows a single file to be used. All sound resources are concatenated in a single data structure, and each item can be accessed and used independently.

A soundfile has:

- two inputs: the sound number (as an integer between 0 and 255, automatically promoted to *int*), and the read index in the sound (automatically promoted to *int*). **Note that when the read index is higher or equal to the sample length, the soundfile primitive will return the last sample.**
- two fixed outputs: the first one is the length in samples of the currently accessed sound, the second one is the nominal sample rate in Hz of the currently accessed sound

- `n` additional outputs for the sound channels themselves, as an integer *constant numerical expression*

If more outputs than the actual number of channels in the soundfile are used, the sound channels will be automatically duplicated up to the wanted number of outputs: output channel c reads the real channel c modulo the number of channels, with no limit on the number of outputs (so for instance if a stereo sound is used with four output channels, the same group of two channels will be duplicated).

If the soundfile cannot be loaded for whatever reason, a default sound with one channel, a length of 1024 frames and null outputs (with samples of value 0) will be used. Note also that soundfiles are entirely loaded in memory by the architecture file, so that the read index signal can access any sample.

Specialized architecture files are responsible for loading the actual soundfile. The `SoundUI` C++ class located in the `faust/gui/SoundUI.h` file implements the `void addSoundfile(label, url, sf_zone)` method, which loads the actual soundfiles using the *libsndfile* library, or possibly specific audio file loading code (in the case of the JUCE framework, for instance), and sets up the `sf_zone` sound memory pointers.

Note that a specific architecture file can choose to access and use sound resources created by other means (that is, not directly loaded from a soundfile). For instance, a mapping between labels and sound resources defined in memory could be used, with additional code responsible for actually setting up all sound memory pointers when `void addSoundfile(label, url, sf_zone)` is called by the `buildUserInterface` mechanism.

3.5.5 C-equivalent primitives

Most FAUST primitives are analogous to their C counterparts but adapted to signal processing. For example `+` is a function of type $\mathbb{S}^2 \to \mathbb{S}^1$ that transforms a pair of signals (x_1, x_2) into a 1-tuple of signals (y) such that $\forall t \in \mathbb{N}, y(t) = x_1(t) + x_2(t)$. The function `-` has type $\mathbb{S}^2 \to \mathbb{S}^1$ and transforms a pair of signals (x_1, x_2) into a 1-tuple of signals (y) such that $\forall t \in \mathbb{N}, y(t) = x_1(t) - x_2(t)$.

Warning: unlike other programming languages the unary operator `-` only exists in limited form in FAUST

Please be aware that the unary `-` exists only in a limited form. It can be used with numbers (`-0.5`) and variables (`-myvar`), but not with expressions surrounded by parentheses, because in this case it represents a partial application. For instance `-(a * b)` is a partial application. It is syntactic sugar for `_,(a * b): -`. If you want to negate a complex term in parentheses, you'll have to use `0 - (a * b)` instead.

The primitives may use the `int` type for their arguments, but will automatically use the `float` type when the actual computation requires it. For instance `1/2` using `int` type

arguments will correctly result in `0.5` in `float` type. Logical and shift primitives use the `int` type.

Syntax	Type	Description
n	$\mathbb{S}^0 \to \mathbb{S}^1$	integer number: $y(t) = n$
$n.m$	$\mathbb{S}^0 \to \mathbb{S}^1$	floating point number: $y(t) = n.m$
`_`	$\mathbb{S}^1 \to \mathbb{S}^1$	identity function: $y(t) = x(t)$
`!`	$\mathbb{S}^1 \to \mathbb{S}^0$	cut function: $\forall x \in \mathbb{S}, (x) \to ()$
`int`	$\mathbb{S}^1 \to \mathbb{S}^1$	cast into an int signal: $y(t) = (int)x(t)$
`float`	$\mathbb{S}^1 \to \mathbb{S}^1$	cast into a float signal: $y(t) = (float)x(t)$
`+`	$\mathbb{S}^2 \to \mathbb{S}^1$	addition: $y(t) = x_1(t) + x_2(t)$
`-`	$\mathbb{S}^2 \to \mathbb{S}^1$	subtraction: $y(t) = x_1(t) - x_2(t)$
`*`	$\mathbb{S}^2 \to \mathbb{S}^1$	multiplication: $y(t) = x_1(t) * x_2(t)$
$\wedge$	$\mathbb{S}^2 \to \mathbb{S}^1$	power: $y(t) = x_1(t)^{x_2(t)}$
`/`	$\mathbb{S}^2 \to \mathbb{S}^1$	division: $y(t) = x_1(t)/x_2(t)$
`%`	$\mathbb{S}^2 \to \mathbb{S}^1$	modulo: $y(t) = x_1(t)\%x_2(t)$
`&`	$\mathbb{S}^2 \to \mathbb{S}^1$	bitwise AND: $y(t) = x_1(t)\&x_2(t)$
`|`	$\mathbb{S}^2 \to \mathbb{S}^1$	bitwise OR: $y(t) = x_1(t)\|x_2(t)$
`xor`	$\mathbb{S}^2 \to \mathbb{S}^1$	bitwise XOR: $y(t) = x_1(t) \wedge x_2(t)$
`<<`	$\mathbb{S}^2 \to \mathbb{S}^1$	arithmetic shift left: $y(t) = x_1(t) << x_2(t)$
`>>`	$\mathbb{S}^2 \to \mathbb{S}^1$	arithmetic shift right: $y(t) = x_1(t) >> x_2(t)$
`<`	$\mathbb{S}^2 \to \mathbb{S}^1$	less than: $y(t) = x_1(t) < x_2(t)$
`<=`	$\mathbb{S}^2 \to \mathbb{S}^1$	less than or equal: $y(t) = x_1(t) <= x_2(t)$
`>`	$\mathbb{S}^2 \to \mathbb{S}^1$	greater than: $y(t) = x_1(t) > x_2(t)$
`>=`	$\mathbb{S}^2 \to \mathbb{S}^1$	greater than or equal: $y(t) = x_1(t) >= x_2(t)$
`==`	$\mathbb{S}^2 \to \mathbb{S}^1$	equal: $y(t) = x_1(t) == x_2(t)$
`!=`	$\mathbb{S}^2 \to \mathbb{S}^1$	not equal: $y(t) = x_1(t)! = x_2(t)$

3.5.6 `math.h`-equivalent primitives

Most of the C `math.h` functions are also built-in as primitives (the others are defined as external functions in file `maths.lib`). The primitives may use the `int` type for their arguments, but will automatically use the `float` type when the actual computation requires it.

Syntax	Type	Description
`acos`	$\mathbb{S}^1 \to \mathbb{S}^1$	arc cosine: $y(t) = \text{acosf}(x(t))$
`asin`	$\mathbb{S}^1 \to \mathbb{S}^1$	arc sine: $y(t) = \text{asinf}(x(t))$
`atan`	$\mathbb{S}^1 \to \mathbb{S}^1$	arc tangent: $y(t) = \text{atanf}(x(t))$
`atan2`	$\mathbb{S}^2 \to \mathbb{S}^1$	arc tangent: $y(t) = \text{atan2f}(x_1(t), x_2(t))$
`cos`	$\mathbb{S}^1 \to \mathbb{S}^1$	cosine: $y(t) = \text{cosf}(x(t))$
`sin`	$\mathbb{S}^1 \to \mathbb{S}^1$	sine: $y(t) = \text{sinf}(x(t))$
`tan`	$\mathbb{S}^1 \to \mathbb{S}^1$	tangent: $y(t) = \text{tanf}(x(t))$
`exp`	$\mathbb{S}^1 \to \mathbb{S}^1$	base-e exponential: $y(t) = \text{expf}(x(t))$
`log`	$\mathbb{S}^1 \to \mathbb{S}^1$	base-e logarithm: $y(t) = \text{logf}(x(t))$
`log10`	$\mathbb{S}^1 \to \mathbb{S}^1$	base-10 logarithm: $y(t) = \text{log10f}(x(t))$
`pow`	$\mathbb{S}^2 \to \mathbb{S}^1$	power: $y(t) = \text{powf}(x_1(t), x_2(t))$
`sqrt`	$\mathbb{S}^1 \to \mathbb{S}^1$	square root: $y(t) = \text{sqrtf}(x(t))$
`abs`	$\mathbb{S}^1 \to \mathbb{S}^1$	absolute value (int): $y(t) = \text{abs}(x(t))$
		absolute value (float): $y(t) = \text{fabsf}(x(t))$
`min`	$\mathbb{S}^2 \to \mathbb{S}^1$	minimum: $y(t) = \min(x_1(t), x_2(t))$
`max`	$\mathbb{S}^2 \to \mathbb{S}^1$	maximum: $y(t) = \max(x_1(t), x_2(t))$
`fmod`	$\mathbb{S}^2 \to \mathbb{S}^1$	float modulo: $y(t) = \text{fmodf}(x_1(t), x_2(t))$
`remainder`	$\mathbb{S}^2 \to \mathbb{S}^1$	float remainder: $y(t) = \text{remainderf}(x_1(t), x_2(t))$
`floor`	$\mathbb{S}^1 \to \mathbb{S}^1$	largest int $\leq$: $y(t) = \text{floorf}(x(t))$
`ceil`	$\mathbb{S}^1 \to \mathbb{S}^1$	smallest int $\geq$: $y(t) = \text{ceilf}(x(t))$
`rint`	$\mathbb{S}^1 \to \mathbb{S}^1$	closest int using the current rounding mode: $y(t) = \text{rintf}(x(t))$
`round`	$\mathbb{S}^1 \to \mathbb{S}^1$	nearest int value, regardless of the current rounding mode: $y(t) = \text{rintf}(x(t$

3.5.7 Delay, Table, Selector primitives

The following primitives allow to define delays, read-only and read-write tables and 2 or 3-ways selectors (see figure 3.9), and some other specific primitives.

Syntax	Type	Description
`mem`	$\mathbb{S}^1 \to \mathbb{S}^1$	1-sample delay: $y(t+1) = x(t), y(0) = 0$
`prefix`	$\mathbb{S}^2 \to \mathbb{S}^1$	1-sample delay: $y(t+1) = x_2(t), y(0) = x_1(0)$
`@`	$\mathbb{S}^2 \to \mathbb{S}^1$	variable delay: $y(t) = x_1(t - x_2(t)), x_1(t < 0) = 0$
`rdtable`	$\mathbb{S}^3 \to \mathbb{S}^1$	read-only table: $y(t) = T[r(t)]$
`rwtable`	$\mathbb{S}^5 \to \mathbb{S}^1$	read-write table: $T[w(t)] = c(t); y(t) = T[r(t)]$
`select2`	$\mathbb{S}^3 \to \mathbb{S}^1$	select between 2 signals: $T[] = \{x_0(t), x_1(t)\}; y(t) = T[s(t)]$
`select3`	$\mathbb{S}^4 \to \mathbb{S}^1$	select between 3 signals: $T[] = \{x_0(t), x_1(t), x_2(t)\}; y(t) = T[s(t)]$
`lowest`	$\mathbb{S}^1 \to \mathbb{S}^1$	$y(t) = \min_{j=0}^{\text{inf}} x(j)$
`highest`	$\mathbb{S}^1 \to \mathbb{S}^1$	$y(t) = \max_{j=0}^{\text{inf}} x(j)$

The size input of *rdtable* and *rwtable* are integer *constant numerical expressions* automatically promoted to *int*, and the read and write indexes are also automatically promoted to *int*. The delay value is automatically promoted to *int*.

3.5.8 User Interface Elements

FAUST user interface widgets allow an abstract description of the user interface from within the FAUST code. This description is independent of any GUI toolkits. It is based on *buttons*, *checkboxes*, *sliders*, etc. that are grouped together vertically and horizontally using appropriate grouping schemes.

All these GUI elements produce signals. A button for example (see figure 3.10) produces a signal which is 1 when the button is pressed and 0 otherwise. These signals can be freely combined with other audio signals.

Syntax	Example
`button(`*str*`)`	`button("play")`
`checkbox(`*str*`)`	`checkbox("mute")`
`vslider(`*str*`,`*cur*`,`*min*`,`*max*`,`*step*`)`	`vslider("vol",50,0,100,1)`
`hslider(`*str*`,`*cur*`,`*min*`,`*max*`,`*step*`)`	`hslider("vol",0.5,0,1,0.01)`
`nentry(`*str*`,`*cur*`,`*min*`,`*max*`,`*step*`)`	`nentry("freq",440,0,8000,1)`
`vgroup(`*str*`,`*block-diagram*`)`	`vgroup("reverb", ...)`
`hgroup(`*str*`,`*block-diagram*`)`	`hgroup("mixer", ...)`
`tgroup(`*str*`,`*block-diagram*`)`	`tgroup("parametric", ...)`
`vbargraph(`*str*`,`*min*`,`*max*`)`	`vbargraph("input",0,100)`
`hbargraph(`*str*`,`*min*`,`*max*`)`	`hbargraph("signal",0,1.0)`
`attach`	`attach(x, vumeter(x))`

All numerical parameters (like *cur*, *min*, *max*, *step*) are *constant numerical expressions*.

Labels

Every user interface widget has a label (a string) that identifies it and informs the user of its purpose. There are three important mechanisms associated with labels (and coded inside the string): *variable parts*, *pathnames* and *metadata*.

Variable parts. Labels can contain variable parts. These variable parts are indicated by the sign '%' followed by the name of a variable. During compilation each label is processed in order to replace the variable parts by the value of the variable. For example `par(i,8,hslider("Voice %i", 0.9, 0, 1, 0.01))` creates 8 different sliders in parallel :

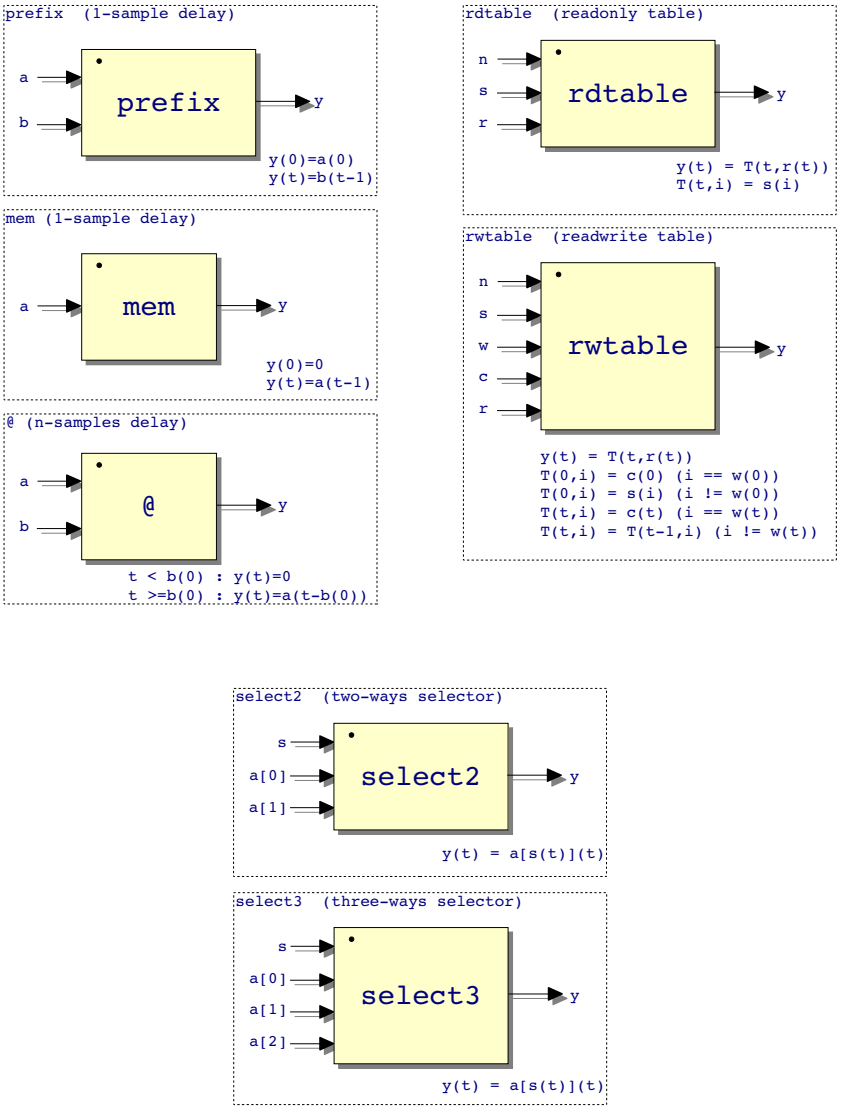


Figure 3.9: Delays, tables and selectors primitives

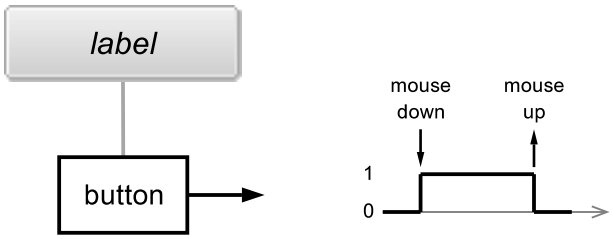


Figure 3.10: User Interface Button

```
hslider("Voice 0", 0.9, 0, 1, 0.01),
hslider("Voice 1", 0.9, 0, 1, 0.01),
...
hslider("Voice 7", 0.9, 0, 1, 0.01).
```

while `par(i,8,hslider("Voice", 0.9, 0, 1, 0.01))` would have created only one slider and duplicated its output 8 times.

The variable part can have an optional format digit. For example `"Voice %2i"` would indicate to use two digits when inserting the value of i in the string.

An escape mechanism is provided. If the sign `%` is followed by itself, it will be included in the resulting string. For example `"feedback (%%)"` will result in `"feedback (%)"`.

The variable name can be enclosed in curly brackets to clearly separate it from the rest of the string, as in `par(i,8,hslider("Voice %{i}", 0.9, 0, 1, 0.01))`.

Pathnames. Thanks to horizontal, vertical and tabs groups, user interfaces have a hierarchical structure analog to a hierarchical file system. Each widget has an associated *pathname* obtained by concatenating the labels of all its surrounding groups with its own label.

In the following example:

```
hgroup("Foo",
    ...
    vgroup("Faa",
        ...
        hslider("volume",...)
        ...
    )
    ...
)
```

the volume slider has pathname `/h:Foo/v:Faa/volume`.

In order to give more flexibility to the design of user interfaces, it is possible to explicitly specify the absolute or relative pathname of a widget directly in its label.

In our previous example the pathname of:

```
hslider("../volume",...)
```

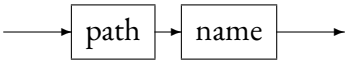
would have been `"/h:Foo/volume"`, while the pathname of:

```
hslider("t:Fii/volume",...)
```

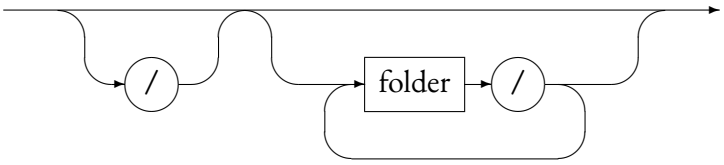
would have been: `"/h:Foo/v:Faa/t:Fii/volume"`.

The grammar for labels with pathnames is the following:

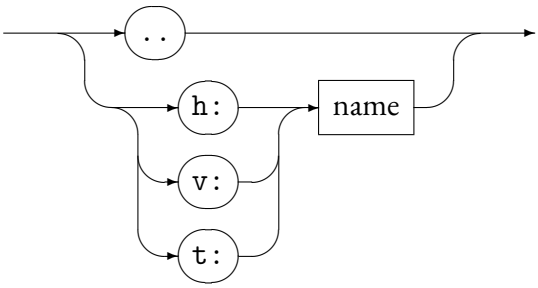
label



path



folder



Metadata Widget labels can contain metadata enclosed in square brackets. These metadata associate a key with a value and are used to provide additional information to the architecture file. They are typically used to improve the look and feel of the user interface. The FAUST code:

```
process = *(hslider("foo [key1: val 1][key2: val 2]",
                    0, 0, 1, 0.1));
```

will produce and the corresponding C++ code:

```
class mydsp : public dsp {
    ...
    virtual void buildUserInterface(UI* interface) {
      interface->openVerticalBox("m");
      interface->declare(&fslider0, "key1", "val 1");
      interface->declare(&fslider0, "key2", "val 2");
      interface->addHorizontalSlider("foo",
        &fslider0, 0.0f, 0.0f, 1.0f, 0.1f);
      interface->closeBox();
    }
...
};
```

All the metadata are removed from the label by the compiler and transformed in calls to the `UI::declare()` method. All these `UI::declare()` calls will always take place before the `UI::AddSomething()` call that creates the User Interface element. This allows the `UI::AddSomething()` method to make full use of the available metadata.

It is the role of the architecture file to decide what to do with these metadata. The `jack-qt.cpp` architecture file for example implements the following:

1. `"...[style:knob]..."` creates a rotating knob instead of a regular slider or nentry.
2. `"...[style:led]..."` in a bargraph's label creates a small LED instead of a full bargraph
3. `"...[unit:dB]..."` in a bargraph's label creates a more realistic bargraph with colors ranging from green to red depending of the level of the value
4. `"...[unit:xx]..."` in a widget postfixes the value displayed with xx
5. `"...[tooltip:bla bla]..."` add a tooltip to the widget
6. `"...[osc:/address min max]..."` Open Sound Control message alias

Moreover starting a label with a number option like in `"[1]..."` provides a convenient means to control the alphabetical order of the widgets.

Attach

The `attach` primitive takes two input signals and produce one output signal which is a copy of the first input. The role of `attach` is to force its second input signal to

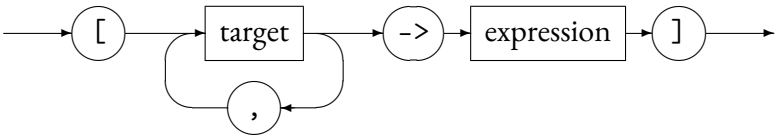
be compiled with the first one. From a mathematical point of view `attach(x,y)` is equivalent to `1*x+0*y`, which is in turn equivalent to `x`, but it tells the compiler not to optimize-out `y`.

To illustrate this role let say that we want to develop a mixer application with a vumeter for each input signals. Such vumeters can be easily coded in FAUST using an envelop detector connected to a bargraph. The problem is that these envelop signals have no role in the output signals. Using `attach(x,vumeter(x))` one can tell the compiler that when `x` is compiled `vumeter(x)` should also be compiled.

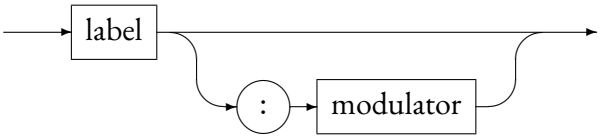
3.5.9 Widget Modulation

Widget modulation acts on the widgets of an existing FAUST expression, but without requiring any manual modifications of the expression's code. This operation is done directly by the compiler, according to a list of *target widgets* and associated *modulators*. Target widgets are specified by their label, as used in the graphical user interface. Modulators are FAUST expressions that describe how to transform the signal produced by widgets. The syntax of a widget modulation is the following:

modulation



target



modulator



Here is a very simple example of widget modulation, assuming freeverb is a fully functional reverb with a `"Wet"` slider:

```
["Wet" -> freeverb]
```

The resulting circuit will have three inputs instead of two. The additional input is for the `"Wet"` widget. It acts on the values produced by the widget inside the freeverb expression.

By default, the additional input signal, and the widget signal are multiplied together. In the following example, an external LFO is connected to this additional input:

```
lfo(10, 0.5), _, _ : ["Wet" -> freeverb]
```

Target Widgets

Target widgets are specified by their label. Of course, this presupposes knowing the names of the widgets. But as these names appear on the user interface, it's easy enough. If several widgets have the same name, adding the names of some (not necessarily all) of the surrounding groups, as in `"h:group/h:subgroup/label"` can help distinguish them.

Multiple targets can be indicated in the same widget modulation expression as in:

```
["Wet", "Damp", "RoomSize" -> freeverb]
```

Modulators

Modulators are FAUST expressions, with exactly one output and at most two inputs that describe how to transform the signals produced by widgets. By default, when nothing is specified, the modulator is a multiplication. This is why our previous example is equivalent to:

```
["Wet": * -> freeverb]
```

Please note that the `':'` sign used here is just a visual separator, it is not the sequential composition operator.

To indicate that the modulation signal should be added, instead of multiplied, one could write:

```
["Wet": + -> freeverb]
```

Multiplications and addition are examples of `2->1` modulators, but two other types are allowed: `0->1` and `1->1`.

Modulators with no inputs Modulators with no inputs `0->1` completely replace the target widget (it won't appear anymore in the user interface). Let's say that we want to remove the `"Damp"` slider and replace it with the constant `0.5`, we can write:

```
    ["Damp": 0.5 -> freeverb]
```

Modulators with one input A `1->1` modulator transforms the signal produced by the target widget without the help of an external input. Our previous example could be written as:

```
["Wet": *(lfo(10, 0.5)) -> freeverb]
```

If `lfo` had its user interface, it would be added to the freeverb interface, at the same level as the `"Wet"` slider.

Modulators with two inputs Modulators with two inputs, like `*` or `+`, are used to combine the signal produced by the target widget with an external signal. The first input is connected to the widget, the second one is connected to the external signal. As we have already seen, our example could be written as:

```
    lfo(10, 0.5), _, _ : ["Wet": * -> freeverb]
```

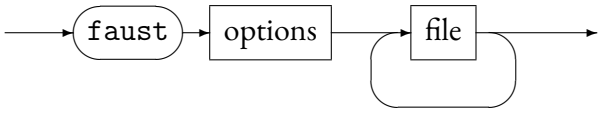
The main difference with the previous case is that if `lfo` had a user interface, it would be added outside of the freeverb interface. Please note that only `2->1` modulators result in additional inputs.

Chapter 4

Invoking the FAUST Compiler

The FAUST compiler is invoked using the `faust` command. It translates FAUST programs into C++ code. The generated code can be wrapped into an optional *architecture file*, allowing you to directly produce a fully operational program.

compiler



For example, `faust noise.dsp` will compile `noise.dsp` and output the corresponding C++ code on the standard output. The option `-o` allows you to choose the output file: `faust noise.dsp -o noise.cpp`. The option `-a` allows you to choose the architecture file: `faust -a alsa-gtk.cpp noise.dsp`.

To compile a FAUST program into an ALSA application on Linux you can use the following commands:

```
faust -a alsa-gtk.cpp noise.dsp -o noise.cpp
g++ -lpthread -lasound `pkg-config --cflags --libs
    gtk+-2.0` noise.cpp -o noise
```

4.1 Structure of the generated code

A FAUST DSP C++ class derives from the base *dsp* class defined below (a similar structure is used for languages other than C++):

```
class dsp {

    public:

        dsp() {}
        virtual ~dsp() {}

        // Returns the number of inputs of the Faust program
        virtual int getNumInputs() = 0;

        // Returns the number of outputs of the Faust program
        virtual int getNumOutputs() = 0;

        // This method can be called to retrieve
        // the UI description of the Faust program
        // and its associated fields
        virtual void buildUserInterface(UI* ui_interface) =
            0;

        // Returns the current sampling rate
        virtual int getSampleRate() = 0;

        // Init methods
        virtual void init(int sample_rate) = 0;
        virtual void instanceInit(int sample_rate) = 0;
        virtual void instanceConstants(int sample_rate) = 0;
        virtual void instanceResetUserInterface() = 0;
        virtual void instanceClear() = 0;

        // Returns a clone of the instance
        virtual dsp* clone() = 0;

        // Retrieve the global metadata of the Faust program
        virtual void metadata(Meta* m) = 0;

        // Compute one audio buffer
        virtual void compute(int count, FAUSTFLOAT** inputs,
            FAUSTFLOAT** outputs) = 0;

        // Compute a time-stamped audio buffer
        virtual void compute(double date_usec, int count,
            FAUSTFLOAT** inputs, FAUSTFLOAT** outputs)
        {
             compute(count, inputs, outputs);
        }
};
```

Here is the class generated with the command `faust noise.dsp`. Methods are filled by the compiler with the actual code.

Several fine-grained initialization methods are available. The `instanceInit` method calls several additional initialization methods. The `instanceConstants` method sets the instance constant state. The `instanceClear` method resets the instance dynamic state (delay lines, etc.). The `instanceResetUserInterface` method resets all control values to their default state. All of those methods can be used individually on an allocated instance to reset part of its state.

The `classInit` static method will initialize static tables that are shared between all instances of the class, and is typically supposed to be called once.

Finally, the `init` method combines class static state and instance initialization.

When using a single instance, calling `init` is the simplest way to do what is needed. When using several instances, all of them can be initialized using `instanceInit`, with a single call to `classInit` to initialize the static shared state.

The `compute` method takes the number of frames to process, and `inputs` and `outputs` buffers as arrays of separate mono channels. Note that by default `inputs` and `outputs` buffers are supposed to be distinct memory zones, so one cannot safely write `compute(count, inputs, inputs)`. The `-inpl` compilation option can be used for that, but only in scalar mode for now.

By default the generated code processes `float` type samples. This can be changed using the `-double` option (or even `-quad` in some backends). The `FAUSTFLOAT` type used in the `compute` method is defined in architecture files, and can be `float` or `double`, depending on the audio driver layer. Sample adaptation may be required between the DSP sample type and the audio driver sample type.

```
class mydsp : public dsp {

  private:
      FAUSTFLOAT fslider0;
      int iRec0[2];
      int fSampleRate;

  public:
     virtual void metadata(Meta* m) {
        m->declare("name", "Noise");
        m->declare("version", "1.1");
        m->declare("author", "Grame");
        m->declare("license", "BSD");
        m->declare("copyright", "(c)GRAME 2009");
    }

    virtual int getNumInputs() { return 0; }
    virtual int getNumOutputs() { return 1; }
    static void classInit(int sample_rate) {
    }
```

```
    virtual void instanceConstants(int sample_rate) {
        fSampleRate = sample_rate;
    }
    virtual void instanceResetUserInterface() {
        fslider0 = 0.5f;
    }
    virtual void instanceClear() {
        for (int i=0; i<2; i++) iRec0[i] = 0;
    }
    virtual void init(int sample_rate) {
        classInit(sample_rate);
        instanceInit(sample_rate);
    }
    virtual void instanceInit(int sample_rate) {
        instanceConstants(sample_rate);
        instanceResetUserInterface();
        instanceClear();
    }
    virtual mydsp* clone() {
        return new mydsp();
    }
    virtual int getSampleRate() {
        return fSampleRate;
    }
    virtual void buildUserInterface(UI* ui_interface) {
        ui_interface->openVerticalBox("Noise");
        ui_interface->declare(&fslider0, "style", "knob");
        ui_interface->addVerticalSlider("Volume", &fslider0,
            0.5f, 0.0f, 1.0f, 0.1f);
        ui_interface->closeBox();
    }
    virtual void compute (int count, FAUSTFLOAT** input,
        FAUSTFLOAT** output) {
        float fSlow0 = (4.656613e-10f * float(fslider0));
        FAUSTFLOAT* output0 = output[0];
        for (int i=0; i<count; i++) {
            iRec0[0] = ((1103515245 * iRec0[1]) + 12345);
            output0[i] = (FAUSTFLOAT)(fSlow0 * iRec0[0]);
            // post processing
            iRec0[1] = iRec0[0];
        }
    }
};
```

4.2 Compilation options

Compilation options are listed in the following table :

Short	Long	Description
-h	-help	print the help message
-v	-version	print version information
-d	-details	print compilation details
-tg	-task-graph	draw a graph of all internal computation loops as a .dot (graphviz) file.
-sg	-signal-graph	draw a graph of all internal signal expressions as a .dot (graphviz) file.
-ps	-postscript	generate block-diagram postscript files
-svg	-svg	generate block-diagram svg files
-blur	-shadow-blur	add a blur to boxes shadows
-sd	-simplify-diagrams	simplify block-diagram before drawing them
-f *n*	-fold *n*	max complexity of svg diagrams before splitting into several files (default 25 boxes)
-mns *n*	-max-name-size *n*	max character size used in svg diagram labels
-sn	-simple-names	use simple names (without arguments) for block-diagram (default max size : 40 chars)
-xml	-xml	generate an additional description file in xml format
-uim	-user-interface-macros	add user interface macro definitions to the C++ code
-flist	-file-list	list all the source files and libraries implied in a compilation
-norm	-normalized-form	prints the internal signals in normalized form and exits
-lb	-left-balanced	generate left-balanced expressions
-mb	-mid-balanced	generate mid-balanced expressions (default)
-rb	-right-balanced	generate right-balanced expressions
-lt	-less-temporaries	generate less temporaries in compiling delays
-mcd *n*	-max-copy-delay *n*	threshold between copy and ring buffer delays (default 16 samples)
-vec	-vectorize	generate easier to vectorize code

continued on next page

Short	Long	Description
`-vs` *n*	`-vec-size` *n*	size of the vector (default 32 samples) when -vec
`-lv` *n*	`-loop-variant` *n*	loop variant [0:fastest (default), 1:simple] when -vec
`-dfs`	`-deepFirstScheduling`	schedule vector loops in deep first order when -vec
`-omp`	`-openMP`	generate parallel code using OpenMP (implies -vec)
`-sch`	`-scheduler`	generate parallel code using threads directly (implies -vec)
`-g`	`-groupTasks`	group sequential tasks together when -omp or -sch is used
`-single`	`-single-precision-floats`	use floats for internal computations (default)
`-double`	`-double-precision-floats`	use doubles for internal computations
`-quad`	`-quad-precision-floats`	use extended for internal computations
`-mdoc`	`-mathdoc`	generates the full mathematical description of a FAUST program
`-mdlang` *l*	`-mathdoc-lang` *l*	choose the language of the mathematical description (*l* = en, fr, ...)
`-stripmdoc`	`-strip-mdoc-tags`	remove documentation tags when printing FAUST listings
`-cn` *name*	`-class-name` *name*	name of the dsp class to be used instead of 'mydsp'
`-t` *time*	`-timeout` *time*	time out of time seconds (default 600) for the compiler to abort
`-a` *file*		architecture file to use
`-o` *file*		C++ output file

Chapter 5

Embedding the FAUST Compiler Using libfaust

The dynamic compilation chain allows developers to embed the FAUST compiler technology directly in their applications or plugins. Thanks to LLVM technology combined with libfaust, the library version of the FAUST compiler, FAUST DSP programs can be compiled and executed on the fly at full speed.

5.1 Dynamic compilation chain

The FAUST compiler uses an intermediate FIR representation (FAUST Imperative Representation), which can be translated to several output languages. The FIR language describes the computation performed on the samples in a generic manner. It contains primitives to read and write variables and arrays, do arithmetic operations, and define the necessary control structures (for and while loops, if structure etc.).

To generate various output languages, several backends have been developed: for C, C++, Java, JavaScript, asm.js, and LLVM IR. The native LLVM based compilation chain is particularly interesting: it provides direct compilation of a DSP source into executable code in memory, bypassing the external compiler requirement.

5.2 LLVM

LLVM (formerly Low Level Virtual Machine) is a compiler infrastructure, designed for compile-time, link-time, run-time optimization of programs written in arbitrary programming languages. Executable code is produced dynamically using a *Just In Time*

compiler from a specific code representation, called LLVM IR. Clang, the LLVM native C/C++/Objective- C compiler is a front-end for LLVM Compiler. It can, for instance, convert a C or C++ source file into LLVM IR code. Domain-specific languages like FAUST can easily target the LLVM IR. This has been done by developing an LLVM IR backend in the FAUST compiler.

5.3 Compiling in memory

The complete chain goes from the FAUST DSP source code, compiled in LLVM IR using the LLVM backend, to finally produce the executable code using the LLVM JIT. All steps take place in memory, getting rid of the classical file based approaches. Pointers to executable functions can be retrieved from the resulting LLVM module and the code directly called with the appropriate parameters.

The FAUST compiler has been packaged as an embeddable library called libfaust, published with an associated API that imitates the concept of object-oriented languages, like C++. Given a FAUST source code (as a file or a string), calling the `createDSPFactoryXXX` function runs the compilation chain (FAUST + LLVM JIT) and generates the *prototype* of the class as a `llvm_dsp_factory` pointer.

```
class llvm_dsp_factory {

 public:

    /* Return Factory name */
    std::string getName();

    /* Return Factory LLVM target */
    std::string getTarget();

    /* Return Factory SHA key */
    std::string getSHAKey();

    /* Return Factory expanded DSP code */
    std::string getDSPCode();

    /* Create a new DSP instance, to be deleted with C++
       'delete' */
    llvm_dsp* createDSPInstance();

    /* Set a custom memory manager to be used when
       creating instances */
```

```
    void setMemoryManager(dsp_memory_manager* manager);

    /* Return the currently set custom memory manager */
    dsp_memory_manager* getMemoryManager();
};
```

Note that the library keeps an internal cache of all allocated factories so that the compilation of the same DSP code—meaning the same source code and the same set of *normalized* (sorted in a canonical order) compilation options—will return the same (reference-counted) factory pointer. You will have to explicitly use `deleteDSPFactory` to properly decrement the reference counter when the factory is no longer needed. You can get a unique SHA1 key of the created factory using its `getSHAKey` method.

Next, the `createDSPInstance` function, corresponding to the `new className` of C++, instantiates a `llvm_dsp` pointer to be used through its interface, connected to the audio chain and controller interfaces. When finished, use `delete` to destroy the dsp instance.

```
class llvm_dsp : public dsp {

 public:

    int getNumInputs();
    int getNumOutputs();

    void buildUserInterface(UI* ui_interface);

    int getSampleRate();

    void init(int sample_rate);
    void instanceInit(int sample_rate);
    void instanceConstants(int sample_rate);
    void instanceResetUserInterface();
    void instanceClear();

    llvm_dsp* clone();

    void metadata(Meta* m);

    void compute(int count, FAUSTFLOAT** inputs,
       FAUSTFLOAT** outputs);
};
```

Since `llvm_dsp` is a subclass of the dsp base class, an object of this type can be used with all already available audio and UI classes, in essence reusing all architecture files already

developed for the static C++ class compilation scheme (like OSCUI, httpdUI interfaces etc.), look at Developing a new architecture file section.

5.4 Saving/restoring the factory

After the DSP factory has been compiled, your application or plugin may want to save/restore it in order to save FAUST to LLVM IR compilation or even JIT compilation time at next use. To get the internal factory compiled code, several functions are available:

- `writeDSPFactoryToIR` allows to get the DSP factory LLVM IR (in textual format) as a string, `writeDSPFactoryToIRFile` allows to save the DSP factory LLVM IR (in textual format) in a file,
- `writeDSPFactoryToBitcode` allows to get the DSP factory LLVM IR (in binary format) as a string, `writeDSPFactoryToBitcodeFile` allows to save the DSP factory LLVM IR (in binary format) in a file,
- `writeDSPFactoryToMachine` allows to get the DSP factory executable machine code as a string, `writeDSPFactoryToMachineFile` allows to save the DSP factory executable machine code in a file.

To re-create a DSP factory from a previously saved code, several functions are available:

- `readDSPFactoryFromIR` allows to create a DSP factory from a string containing the LLVM IR (in textual format), `readDSPFactoryFromIRFile` allows to create a DSP factory from a file containing the LLVM IR (in textual format),
- `readDSPFactoryFromBitcode` allows to create a DSP factory from a string containing the LLVM IR (in binary format), `readDSPFactoryFromBitcodeFile` allows to create a DSP factory from a file containing the LLVM IR (in binary format),
- `readDSPFactoryFromMachine` allows to create a DSP factory from a string containing the executable machine code, `readDSPFactoryFromMachineFile` allows to create a DSP factory from a file containing the executable machine code.

5.5 Additional functions

Some additional functions are available in the libfaust API:

`expandDSPFromString/expandDSPFromFile` creates a self-contained DSP source string where all needed libraries have been included. All compilation options are normalized and included as a comment in the expanded string, `generateAuxFilesFromString/generateAuxFilesFromFile`: from a DSP source string or file, generates auxiliary files: SVG, XML, PS... depending on the argv parameters.

5.6 Using the libfaust library

The libfaust library is part of the FAUST tree. You'll have to compile and install it. Then look at the installed `faust/dsp/llvm-dsp.h` header for a complete description of the API. Note that `faust/dsp/llvm-c-dsp.h` is a pure C version of the same API. The additional functions are available in the `faust/dsp/libfaust.h` header and their C version is in `faust/dsp/libfaust-c.h`.

5.7 Use case examples

The dynamic compilation chain has been used in several projects:

- FaustLive, an integrated IDE for FAUST development
- Faustgen, an external object for Cycling Max/MSP language
- Csound6, see this demo video
- LibAudioStream, a framework to manipulate audio resources through the concept of streams
- Oliver Larkin JUCE framework integration and pMix2 project
- an experimental version of Antescofo
- FaucK: the combination of the Chuck language and FAUST

Chapter 6

Architecture files

A FAUST program describes a *signal processor*, a pure computation that maps *input signals* to *output signals*. It says nothing about audio drivers or GUI toolkits. This missing information is provided by *architecture files*.

An *architecture file* describes how to relate a FAUST program to the external world, in particular the audio drivers and the user interface to be used. This approach allows a single FAUST program to be easily deployed to a large variety of audio standards (Max/MSP externals, PD externals, VST plugins, CoreAudio applications, Jack applications, iPhone, etc.).

The architecture to be used is specified at compile time with the `-a` option. For example `faust -a jack-gtk.cpp foo.dsp` indicates that the Jack GTK architecture should be used when compiling `foo.dsp`.

The main available architecture files are listed in Table 6.1. Since FAUST 0.9.40 some of these architectures are a modular combination of an *audio module* and one or more *user interface modules*. Among these user interface modules, OSCUI provides support for Open Sound Control, allowing FAUST programs to be controlled by OSC messages.

6.1 Audio architecture modules

An *audio architecture module* typically connects a FAUST program to the audio drivers. It is responsible for allocating and releasing the audio channels and for calling the FAUST `dsp::compute` method to handle incoming audio buffers and/or to produce audio output. It is also responsible for presenting the audio as non-interleaved float data, normalized between -1.0 and 1.0.

A FAUST audio architecture module derives an *audio* class defined as below:

File name	Description
alchemy-as.cpp	Flash - ActionScript plugin
ca-qt.cpp	CoreAudio Qt4 standalone application
jack-gtk.cpp	JACK GTK standalone application
jack-qt.cpp	JACK Qt4 standalone application
jack-console.cpp	JACK command line application
jack-internal.cpp	JACK server plugin
alsa-gtk.cpp	ALSA GTK standalone application
alsa-qt.cpp	ALSA Qt4 standalone application
oss-gtk.cpp	OSS GTK standalone application
pa-gtk.cpp	PortAudio GTK standalone application
pa-qt.cpp	PortAudio Qt4 standalone application
max-msp.cpp	Max/MSP external
vst.cpp	VST plugin
vst2p4.cpp	VST 2.4 plugin
vsti-mono.cpp	VSTi mono instrument
vsti-poly.cpp	VSTi polyphonic instrument
ladspa.cpp	LADSPA plugin
q.cpp	Q language plugin
supercollider.cpp	SuperCollider Unit Generator
snd-rt-gtk.cpp	Snd-RT music programming language
csound.cpp	CSOUND opcode
puredata.cpp	PD external
sndfile.cpp	sound file transformation command
bench.cpp	speed benchmark
octave.cpp	Octave plugin
plot.cpp	Command line application
sndfile.cpp	Command line application

Table 6.1: Some of the available architectures.

```
class audio {
 public:
           audio() {}
  virtual ~audio() {}
  virtual bool init(const char*, dsp*) = 0;
  virtual bool start() = 0;
  virtual void stop() = 0;
  virtual void shutdown(shutdown_callback cb, void* arg)  {}
  virtual int getBufferSize() = 0;
  virtual int getSampleRate() = 0;

  virtual int getNumInputs() = 0;
  virtual int getNumOutputs() = 0;

  virtual float getCPULoad() { return 0.f; }
};
```

The API is simple enough to give great flexibility to audio architecture implementations. The `init` method should initialize the audio. Upon `init` exit, the system should be in a safe state to recall the `dsp` object state.

Table 6.2 gives some of the audio architectures currently available for various operating systems.

Audio system	Operating system
Alsa	Linux
CoreAudio	Mac OS X, iOS
JACK	Linux, Mac OS X, Windows
PortAudio	Linux, Mac OS X, Windows
OSC	Linux, Mac OS X, Windows
VST	Mac OS X, Windows
Max/MSP	Mac OS X, Windows
Csound	Linux, Mac OS X, Windows
SuperCollider	Linux, Mac OS X, Windows
PureData	Linux, Mac OS X, Windows
Pure	Linux, Mac OS X, Windows

Table 6.2: Some of FAUST's audio architectures.

6.2 UI architecture modules

A UI architecture module links user actions (via graphic widgets, command line parameters, OSC messages, etc.) with the FAUST program to control. It is responsible for associ-

ating program parameters with user interface elements and updating parameter values according to user actions. This association is triggered by the `dsp::buildUserInterface` call, where the `dsp` asks a UI object to build the DSP module controllers.

Since the interface is basically graphically oriented, the main concepts are *widget* based: a UI architecture module is semantically oriented to handle active widgets, passive widgets, and widget layout.

A FAUST UI architecture module derives an *UI* class (Figure 6.1).

6.2.1 Active widgets

Active widgets are graphical elements that control a parameter value. They are initialized with the widget name and a pointer to the linked value, using the FAUSTFLOAT macro type (defined at compile time as either float or double). The widgets currently considered are `Button`, `CheckButton`, `VerticalSlider`, `HorizontalSlider`, and `NumEntry`.
A GUI architecture must implement a method
`addXxx(const char* name, FAUSTFLOAT* zone, ...)` for each active widget. Additional parameters are available for `Slider` and `NumEntry`: the `init`, `min`, `max` and `step` values.

6.2.2 Passive widgets

Passive widgets are graphical elements that reflect values. Similarly to active widgets, they are initialized with the widget name and a pointer to the linked value. The widgets currently considered are `HorizontalBarGraph` and `VerticalBarGraph`.
A UI architecture must implement a method
`addXxx(const char* name, FAUSTFLOAT* zone, ...)` for each passive widget. Additional parameters are available, depending on the passive widget type.

6.2.3 Widgets layout

Generally, a GUI is hierarchically organized into boxes and/or tab boxes. A UI architecture must support the following methods to set up this hierarchy:

```
openTabBox(const char* label)
openHorizontalBox(const char* label)
openVerticalBox(const char* label)
closeBox(const char* label)
```

Note that all the widgets are added to the current box.

```
#ifndef FAUSTFLOAT
#define FAUSTFLOAT float
#endif

class UI
{
 public:
           UI() {}
  virtual ~UI() {}

     -- widget layouts
  virtual void openTabBox(const char* l)  = 0;
  virtual void openHorizontalBox(const char* l) = 0;
  virtual void openVerticalBox(const char* l) = 0;
  virtual void closeBox() = 0;

   -- active widgets
  virtual void addButton(const char* l, FAUSTFLOAT* z)
     = 0;
  virtual void addCheckButton(const char* l, FAUSTFLOAT* z)
     = 0;

  virtual void addVerticalSlider(const char* l,
          FAUSTFLOAT* z,
          FAUSTFLOAT init, FAUSTFLOAT min,
          FAUSTFLOAT max, FAUSTFLOAT step) = 0;

  virtual void addHorizontalSlider(const char* l,
          FAUSTFLOAT* z,
          FAUSTFLOAT init, FAUSTFLOAT min,
          FAUSTFLOAT max, FAUSTFLOAT step) = 0;

  virtual void addNumEntry(const char* l, FAUSTFLOAT* z,
          FAUSTFLOAT init, FAUSTFLOAT min,
          FAUSTFLOAT max, FAUSTFLOAT step)  = 0;

   -- passive widgets
  virtual void addHorizontalBargraph(const char* l,
                       FAUSTFLOAT* z, FAUSTFLOAT min,
                       FAUSTFLOAT max) = 0;

  virtual void addVerticalBargraph(const char* l,
                       FAUSTFLOAT* z, FAUSTFLOAT min,
                       FAUSTFLOAT max) = 0;

   -- metadata declarations
  virtual void declare(FAUSTFLOAT*, const char*, const char*)
      {}
};
```

Figure 6.1: UI, the root user interface class.

UI	Comment
console	a textual command line UI
GTKUI	a GTK-based GUI
QTGUI	a multi-platform Qt-based GUI
FUI	a file-based UI to store and recall modules states
OSCUI	OSC control (see section 7)
httpdUI	HTTP control (see section 8)
...	...

Table 6.3: Some of the available UI architectures.

6.2.4 Metadata

The FAUST language allows widget labels to contain metadata enclosed in square brackets as key/value pairs. These metadata are handled at GUI level by a `declare` method taking as arguments a pointer to the widget-associated zone, the metadata key, and the value:

```
declare(FAUSTFLOAT* zone, const char* key, const char* value)
```

Here is the table of currently supported general metadata (look at section 7 for OSC-specific metadata and section 9 for MIDI-specific metadata):

Key	Value
tooltip	actual string content
hidden	`0` or `1`
size	actual value
unit	`Hz` or `dB`
scale	`log` or `exp`
style	`knob` or `led` or `numerical`
style	`radio{'label1':v1;'label2':v2...}`
style	`menu{'label1':v1;'label2':v2...}`
acc	`axe curve amin amid amax`
gyr	`axe curve amin amid amax`
screencolor	`red` or `green` or `blue` or `white`

Table 6.4: Supported metadata.

Some typical examples where several metadata are defined could be:

```
nentry("freq [unit:Hz][scale:log][acc:0 0 -30 0 30][style:menu{'white
noise':0;'pink noise':1;'sine':2}][hidden:0]", 0, 20, 100, 1)
```

or:

```
vslider("freq [unit:dB][style:knob][gyr:0 0 -30 0 30]", 0, 20, 100,
1)
```

Note that metadata are not supported in all architecture files. Some of them, like `acc` or `gyr`, only make sense on platforms with accelerometer or gyroscope sensors. The set of metadata may be extended in the future.

6.3 Developing a new architecture file

Developing a new architecture file typically means writing a generic C++ file that will be populated with the actual output of the FAUST compiler, in order to produce a complete C++ file ready to be compiled as a standalone application or plugin.

The architecture to be used is specified at compile time with the `-a` option. It must contain the `<<includeIntrinsic>>` and `<<includeclass>>` lines that will be processed by the FAUST compiler and replaced by the generated C++ class.

Look at the `minimal.cpp` example located in the architecture folder:

```
#include "faust/gui/PrintUI.h"
#include "faust/gui/meta.h"
#include "faust/audio/dummy-audio.h"

using std::max;
using std::min;

//----------------------------------------------------
//  FAUST generated signal processor
//----------------------------------------------------

<<includeIntrinsic>>

<<includeclass>>

int main(int argc, char *argv[])
{
    mydsp DSP;
    PrintUI ui;

    // Activate the UI
    // (here it only prints the control paths)
    DSP.buildUserInterface(&ui);
```

```
    // Allocate the audio driver to render
    // 5 buffers of 512 frames
    dummyaudio audio(5);
    audio.init("Test", &DSP);

    // Render buffers...
    audio.start();

    audio.stop();
}
```

Calling `faust -a minimal.cpp noise.dsp -a noise.cpp` will produce a ready to compile `noise.cpp` file:

```
#include "faust/gui/PrintUI.h"
#include "faust/gui/meta.h"
#include "faust/audio/dummy-audio.h"

using std::max;
using std::min;

//-------------------------------------------------------
//  FAUST generated signal processor
//-------------------------------------------------------

#ifndef FAUSTFLOAT
#define FAUSTFLOAT float
#endif

#ifndef FAUSTCLASS
#define FAUSTCLASS mydsp
#endif

class mydsp : public dsp {
  private:
    FAUSTFLOAT fslider0;
    int iRec0[2];
    int fSamplingFreq;

  public:
    virtual void metadata(Meta* m) {
        m->declare("name", "Noise");
        m->declare("version", "1.1");
        m->declare("author", "Grame");
        m->declare("license", "BSD");
        m->declare("copyright", "(c)GRAME 2009");
    }

    virtual int getNumInputs() { return 0; }
```

```
    virtual int getNumOutputs() { return 1; }
    static void classInit(int samplingFreq) {
    }
    virtual void instanceConstants(int samplingFreq) {
        fSamplingFreq = samplingFreq;
    }
    virtual void instanceResetUserInterface() {
        fslider0 = 0.5f;
    }
    virtual void instanceClear() {
        for (int i=0; i<2; i++) iRec0[i] = 0;
    }
    virtual void init(int samplingFreq) {
        classInit(samplingFreq);
        instanceInit(samplingFreq);
    }
    virtual void instanceInit(int samplingFreq) {
        instanceConstants(samplingFreq);
        instanceResetUserInterface();
        instanceClear();
    }
    virtual mydsp* clone() {
        return new mydsp();
    }
    virtual int getSampleRate() {
        return fSamplingFreq;
    }
    virtual void buildUserInterface(UI* ui_interface) {
        ui_interface->openVerticalBox("Noise");
        ui_interface->declare(&fslider0, "acc", "0 0 -10 0 10
            ");
        ui_interface->declare(&fslider0, "style", "knob");
        ui_interface->addVerticalSlider("Volume", &fslider0,
            0.5f, 0.0f, 1.0f, 0.1f);
        ui_interface->closeBox();
    }
    virtual void compute (int count, FAUSTFLOAT** input,
       FAUSTFLOAT** output) {
        float fSlow0 = (4.656613e-10f * float(fslider0));
        FAUSTFLOAT* output0 = output[0];
        for (int i=0; i<count; i++) {
            iRec0[0] = ((1103515245 * iRec0[1]) + 12345);
            output0[i] = (FAUSTFLOAT)(fSlow0 * iRec0[0]);
            // post processing
            iRec0[1] = iRec0[0];
        }
    }
};
```

```
int main(int argc, char* argv[])
{
    mydsp DSP;
    PrintUI ui;

    // Activate the UI
    // (here that only prints the control paths)
    DSP.buildUserInterface(&ui);

    // Allocate the audio driver to render
    // 5 buffers of 512 frames
    dummyaudio audio(5);
    audio.init("Test", &DSP);

    // Render buffers...
    audio.start();

    audio.stop();
}
```

You can optionally add the `-i` option to inline all `#include "faust/xxx/yyy"` headers (all files starting with "faust"). Then you will have to write a `faust2xxx` script that chains the FAUST compilation step and the C++ compilation step. Look at the scripts in the tools/faust2appls folder for real examples.

Developing the adapted C++ file may require "aggregating" the generated `mydsp` class (a subclass of the `dsp` base class defined in the faust/dsp/dsp.h header) in your specific class, or "subclassing" and extending it. So you will have to write something like:

```
class my_class : public base_interface {

  private:

    mydsp fDSP;

  public:

    my_class()
    {
        // Do something specific
    }

    virtual ~my_class()
    {
        // Do something specific
    }

    // Do something specific
```

```
    void my_compute(int count,
        FAUSTFLOAT** inputs,
        FAUSTFLOAT** outputs,....)
    {
        // Do something specific
        fDSP.compute(count, inputs, outputs);
    }

    // Do something specific
};
```

or:

```
class my_class : public mydsp  {

  private:

    // Do something specific

  public:

    my_class()
    {
        // Do something specific
    }

    virtual ~my_class()
    {
        // Do something specific
    }

    // Do something specific

    void my_compute(int count,
        FAUSTFLOAT** inputs,
        FAUSTFLOAT** outputs,....)
    {
        // Do something specific
        compute(count, inputs, outputs);
    }

    // Do something specific
};
```

This way your architecture file will be adapted to any "shape" of the generated code. That is, depending on whether you generate purely scalar or vector code (using the `-vec` option), or any other option, the generated `mydsp` class will always be correctly inserted in the final C++ file. Look, for instance, at the `csound.cpp` and `unity.cpp` architecture files in the architecture folder for real examples.

Chapter 7

OSC support

Most FAUST architectures provide Open Sound Control (OSC) support[1]. This allows FAUST applications to be remotely controlled from any OSC-capable application, programming language, or hardware device. OSC support can be activated using the `-osc` option when building the application with the appropriate `faust2xxx` command. The following table (Table 7.1) lists the FAUST architectures that provide OSC support.

7.1 A simple example

To illustrate how OSC support works let's define a very simple noise generator with a level control: `noise.dsp`

```
process = library("music.lib").noise
        * hslider("level", 0, 0, 1, 0.01);
```

We are going to compile this example as a standalone JACK/Qt application with OSC support using the command:

```
faust2jaqt -osc noise.dsp
```

When we start the application from the command line:

```
./noise
```

we get various information on the standard output, including:

```
Faust OSC version 0.93 application 'noise' is running
    on UDP ports 5510, 5511, 5512
```

[1]The implementation is based internally on the *oscpack* library by Ross Bencina

Audio system	Environment	OSC support
Linux		
Alsa	GTK, Qt, Console	yes
Jack	GTK, Qt, Console	yes
Netjack	GTK, Qt, Console	yes
PortAudio	GTK, Qt	yes
Mac OS X		
CoreAudio	Qt	yes
Jack	Qt, Console	yes
Netjack	Qt, Console	yes
PortAudio	Qt	yes
Windows		
Jack	Qt, Console	yes
PortAudio	Qt	yes

Table 7.1: FAUST architectures with OSC support.

As we can see, the OSC module makes use of three different UDP ports:

- `5510` is the listening port number: control messages should be addressed to this port.
- `5511` is the output port number: control messages sent by the application and answers to query messages are sent to this port.
- `5512` is the error port number: used for asynchronous error notifications.

Note that if a `declare name "Foo";` line is present in the DSP program, `Foo` will be used as the OSC root name, otherwise the DSP filename will be used instead.

These OSC parameters can be changed from the command line using one of the following options:

- `-port number` sets the port number used by the application to receive messages.
- `-outport number` sets the port number used by the application to transmit messages.
- `-errport number` sets the port number used by the application to transmit error messages.

- `-desthost host` sets the destination host for the messages sent by the application. Note that the destination address can be changed with the first incoming message: the first received packet from another host sets the destination address to that host.

- `-xmit 0|1|2` turns transmission OFF, ALL, or ALIAS (default OFF). When transmission is OFF, input elements can be controlled using their addresses or aliases (if present). When transmission is ALL, input elements can be controlled using their addresses or aliases (if present), user's actions and output elements (bargraph) are transmitted as OSC messages as well as aliases (if present). When transmission is ALIAS, input elements can only be controlled using their aliases, user's actions and output elements (bargraph) are transmitted as aliases only.

- `-xmitfilter path` allows to filter output messages. Note that 'path' can be a regular expression (like "/freeverb/Reverb1/*").

- `-bundle 0|1` activates the bundle mode where all transmitted values are sent in a single message, to be used together with the `-xmit` option.

For example:

```
./noise -xmit 1 -desthost 192.168.1.104 -outport 6000
```

will run noise with transmission mode ON, using 192.168.1.104 on port 6000 as destination.

7.2 Automatic port allocation

In order to address each application individually, only one application can be listening on a single port at one time. Therefore when the default incoming port 5510 is already opened by some other application, an application will automatically try increasing port numbers until it finds an available port. Let's say that we start two applications `noise` and `mixer` on the same machine, here is what we get:

```
$ ./noise &
...
Faust OSC version 0.93 application 'noise' is running
    on UDP ports 5510, 5511, 5512
$ ./mixer
...
Faust OSC version 0.93 application 'mixer' is running
    on UDP ports 5513, 5511, 5512
```

The `mixer` application fails to open the default incoming port 5510 because it is already opened by `noise`. Therefore it tries to find an available port starting from 5513 and open it. Please note that the two outcoming ports 5511 and 5512 are shared by all running applications.

7.3 Discovering OSC applications

`oscsend` *hostname port address types values*: send OpenSound Control message via UDP. *types* is a string, the letters indicates the type of the following values: i=integer, f=float, s=string,...

`oscdump` *port* : receive OpenSound Control messages via UDP and dump to standard output

The commands `oscsend` Send OpenSound Control message via UDP. and `oscdump` from the liblo package provide a convenient mean to experiment with OSC control. For the experiment let's use two additional terminals. The first one will be used to send OSC messages to the noise application using `oscsend`. The second terminal will be used to monitor the messages sent by the application using `oscdump`. We will indicate by `T1$` the command types on terminal T1 and by `T2:` the messages received on terminal T2. To monitor on terminal T2 the OSC messages received on UDP port 5511 we will use `oscdump`:

```
T2$ oscdump 5511
```

Once set we can use the `hello` message to scan UDP ports for FAUST applications. For example:

```
T1$ oscsend localhost 5510 "/*" s hello
```

gives us the root message address, the network and the UDP ports used by the noise application:

```
T2: /noise siii "192.168.1.102" 5510 5511 5512
```

7.4 Discovering the OSC interface of an application

Once we have an application we can discover its OSC interface (the set of OSC messages we can use to control it) by sending the `get` message to the root:

```
T1$ oscsend localhost 5510 /noise s get
```

As an answer of the osc messages understood by the application, a full description is available on terminal T2:

```
T2: /noise sF "xmit" #F
T2: /noise ss "desthost" "127.0.0.1"
T2: /noise si "outport" 5511
T2: /noise si "errport" 5512
T2: /noise/level fff 0.000000 0.000000 1.000000
```

The root of the osc interface is `/noise`. Transmission is OFF, `xmit` is set to false. The destination host for sending messages is `"127.0.0.1"`, the output port is `5511` and the error port is `5512`. The application has only one user interface element: `/noise/level` with current value `0.0`, minimal value `0.0` and maximal value `1.0`.

7.5 Widget's OSC address

Each widget of an application has a unique OSC address obtained by concatenating the labels of its surrounding groups with its own label. Here is, as an example, `mix4.dsp`, a very simplified monophonic audio mixer with four inputs and one output. For each input we have a mute button and a level slider:

There are potential conflicts between widget's labels and the OSC address space. An OSC symbolic name is an ASCII string consisting of a restricted set of printable characters. Therefore to ensure compatibility spaces are replaced by underscores and some other characters (asterisk, comma, forward, question mark, open bracket, close bracket, open curly brace, close curly brace) are replaced by hyphens.

```
input(v) = vgroup("input %v", *(1-checkbox("mute")) :
    *(vslider("level", 0, 0, 1, 0.01)));
process = hgroup("mixer", par(i, 4, input(i)) :> _);
```

If we query this application:

```
T1$ oscsend localhost 5510 "/*" s get
```

We get a full description of its OSC interface on terminal T2:

```
T2: /mixer sF "xmit" #F
T2: /mixer ss "desthost" "127.0.0.1"
T2: /mixer si "outport" 5511
T2: /mixer si "errport" 5512
T2: /mixer/input_0/level fff 0.0000 0.0000 1.0000
T2: /mixer/input_0/mute  fff 0.0000 0.0000 1.0000
T2: /mixer/input_1/level fff 0.0000 0.0000 1.0000
T2: /mixer/input_1/mute  fff 0.0000 0.0000 1.0000
T2: /mixer/input_2/level fff 0.0000 0.0000 1.0000
T2: /mixer/input_2/mute  fff 0.0000 0.0000 1.0000
T2: /mixer/input_3/level fff 0.0000 0.0000 1.0000
T2: /mixer/input_3/mute  fff 0.0000 0.0000 1.0000
```

As we can see each widget has a unique OSC address obtained by concatenating the top level group label "mixer", with the "input" group label and the widget label. Please note that in this operation whites spaces are replaced by underscores and metadata are removed.

All addresses must have a common root. This is the case in our example because there is a unique horizontal group "mixer" containing all widgets. If a common root is missing as in the following code:

```
input(v) = vgroup("input %v", *(1-checkbox("mute")) :
    *(vslider("level", 0, 0, 1, 0.01)));
process = par(i, 4, input(i)) :> _;
```

then a default vertical group is automatically create by the FAUST compiler using the name of the file `mix4` as label:

```
T2: /mix4 sF "xmit" #F
T2: /mix4 ss "desthost" "127.0.0.1"
T2: /mix4 si "outport" 5511
T2: /mix4 si "errport" 5512
T2: /mix4/input_0/level fff 0.0000 0.0000 1.0000
T2: /mix4/input_0/mute  fff 0.0000 0.0000 1.0000
T2: /mix4/input_1/level fff 0.0000 0.0000 1.0000
T2: /mix4/input_1/mute  fff 0.0000 0.0000 1.0000
T2: /mix4/input_2/level fff 0.0000 0.0000 1.0000
T2: /mix4/input_2/mute  fff 0.0000 0.0000 1.0000
T2: /mix4/input_3/level fff 0.0000 0.0000 1.0000
T2: /mix4/input_3/mute  fff 0.0000 0.0000 1.0000
```

7.6 Controlling the application via OSC

We can control any user interface element of the application by sending one of the previously discovered messages. For example to set the noise level of the application to `0.2` we send:

```
T1$ oscsend localhost 5510 /noise/level f 0.2
```

If we now query `/noise/level` we get, as expected, the value `0.2`:

```
T1$ oscsend localhost 5510 /noise/level s get
T2: /noise/level fff 0.2000 0.0000 1.0000
```

7.7 Turning transmission ON

The xmit message at the root level is used to control the realtime transmission of OSC messages corresponding to user interface's actions. For examples:

```
T1$ oscsend localhost 5510 /noise si xmit 1
```

turns transmission in ALL mode. Now if we move the level slider we get a bunch of messages:

```
T2: /noise/level f 0.024000
T2: /noise/level f 0.032000
T2: /noise/level f 0.105000
T2: /noise/level f 0.250000
T2: /noise/level f 0.258000
T2: /noise/level f 0.185000
T2: /noise/level f 0.145000
T2: /noise/level f 0.121000
T2: /noise/level f 0.105000
T2: /noise/level f 0.008000
T2: /noise/level f 0.000000
```

This feature can be typically used for automation to record and replay actions on the user interface, or to remote control from one application to another. It can be turned OFF any time using:

```
T1$ oscsend localhost 5510 /noise si xmit 0
```

Use the ALIAS (xmit = 2) mode if you need restricted access to your program: when ALIAS is mode is used, only aliases of input elements (sliders, buttons...) can be used to control them, and output elements (bargraph) will only emit on their aliases.

7.8 Filtering OSC messages

When the transmission of OSC messages is ON, all the user interface elements are sent through the OSC connection.

```
T2: /harpe/level f 0.024000
T2: /harpe/hand f 0.1
T2: /harpe/level f 0.024000
T2: /harpe/hand f 0.25
T2: /harpe/level f 0.024000
T2: /harpe/hand f 0.44
T2: /noise/level f 0.145000
T2: /harpe/hand f 0.78
T2: /noise/level f 0.145000
T2: /harpe/hand f 0.99
```

We can choose to filter the unwanted parameters (or group of parameters). For example:

```
T1$ oscsend localhost 5510 /harpe si xmit 1
   xmitfilter /harpe/level
```

As a result, we will receive:

```
T2: /harpe/hand f 0.1
T2: /harpe/hand f 0.25
T2: /harpe/hand f 0.44
T2: /harpe/hand f 0.78
```

To reset the filter, send:

```
T1$ oscsend localhost 5510 /harpe si xmit 1
   xmitfilter
```

7.9 Using OSC aliases

Aliases are a convenient mechanism to control a FAUST application from a preexisting set of OSC messages.

Let's say we want to control our noise example with touchOSC on Android. The first step is to configure TouchOSC host to 192.168.1.102 (the host running our noise application) and outgoing port to 5510.

Then we can use oscdump 5510 (after quitting the noise application in order to free port 5510) to visualize the OSC messages sent by TouchOSC. Let's use for that the left slider of simple layout. Here is what we get:

```
T2: /1/fader1 f 0.000000
T2: /1/fader1 f 0.004975
T2: /1/fader1 f 0.004975
T2: /1/fader1 f 0.008125
T2: /1/fader1 f 0.017473
T2: /1/fader1 f 0.032499
T2: /1/fader1 f 0.051032
T2: ...
T2: /1/fader1 f 0.993289
T2: /1/fader1 f 1.000000
```

We can associate this OSC message to the noise level slider by inserting the metadata [osc:/1/fader1 0 1] into the slider's label:

Several osc aliases can be inserted into a single label allowing the same widget to be controlled by several OSC messages.

```
process = library("music.lib").noise * hslider("level
   [osc:/1/fader1 0 1]",0,0,1,0.01);
```

Because here the range of /1/fader1 is 0 to 1 like the level slider we can remove the range mapping information and write simply :

```
process = library("music.lib").noise * hslider("level
   [osc:/1/fader1]", 0, 0, 1, 0.01);
```

TouchOSC can also send accelerometer data by enabling Settings/Options/Accelerometer. Using again oscdump 5510 we can visualize the messages send by TouchOSC:

```
T2: ...
T2: /accxyz fff -0.147842 0.019752 9.694721
T2: /accxyz fff -0.157419 0.016161 9.686341
T2: /accxyz fff -0.167594 0.012570 9.683948
T2: ...
```

As we can see TouchOSC send the x, y and z accelerometers in a single message, as a triplet of values ranging approximatively from -9.81 to 9.81. In order to select the appropriate accelerometer we need to concatenate to /accxyz a suffix /0, /1 or /2. For example /accxyz/0 will correspond to x, /accxyz/1 to y, etc. We also need to define a mapping because the ranges are different:

```
process = library("music.lib").noise * hslider("level[
   osc:/accxyz/0 0 9.81]",0,0,1,0.01);
```

alias	description
[osc:/1/rotary1 0 1]	top left rotary knob
[osc:/1/rotary2 0 1]	middle left rotary knob
[osc:/1/rotary3 0 1]	bottom left rotary knob
[osc:/1/push1 0 1]	bottom left push button
[osc:/1/push2 0 1]	bottom center left push button
[osc:/1/toggle1 0 1]	top center left toggle button
[osc:/1/toggle2 0 1]	middle center left toggle button
[osc:/1/fader1 0 1]	center left vertical fader
[osc:/1/toggle3 0 1]	top center right toggle button
[osc:/1/toggle4 0 1]	middle center right toggle button
[osc:/1/fader2 0 1]	center right vertical toggle button
[osc:/1/rotary4 0 1]	top right rotary knob
[osc:/1/rotary5 0 1]	middle right rotary knob
[osc:/1/rotary6 0 1]	bottom right rotary knob
[osc:/1/push3 0 1]	bottom center right push button
[osc:/1/push4 0 1]	bottom right push button
[osc:/1/fader3 0 1]	bottom horizontal fader
[osc:/accxyz/0 -10 10]	x accelerometer
[osc:/accxyz/1 -10 10]	y accelerometer
[osc:/accxyz/2 -10 10]	z accelerometer

Table 7.2: Examples of OSC message aliases for TouchOSC (layout Mix2).

7.10 OSC cheat sheet

Default ports

`5510`	default listening port
`5511`	default transmission port
`5512`	default error port
`5513...`	alternative listening ports

Command line options

`-port` *n*	set the port number used by the application to receive messages
`-outport` *n*	set the port number used by the application to transmit messages
`-errport` *n*	set the port number used by the application to transmit error messages
`-desthost` *h*	set the destination host for the messages sent by the application
`-xmit 0|1|2`	turn transmission OFF, ALL or ALIAS (default OFF)
`-xmitfilter` *s*	filter the FAUST paths at emission time

Discovery messages

`oscsend` *host port* `"/*" s hello`	discover if any OSC application is listening on port *port*
`oscsend` *host port* `"/*" s get`	query OSC interface of application listening on port *port*
`oscsend` *host port* `"/*" s json`	query JSON description of application listening on port *port*

Control messages

`oscsend` *host port* `"/*" si xmit 0|1|2`	set transmission mode
`oscsend` *host port widget* `s get`	get widget's value
`oscsend` *host port widget* `f` *v*	set widget's value

Alias

`"...[osc:` *address lo hi* `]..."`	alias with *lo*→*min*, *hi*→*max* mapping
`"...[osc:` *address*`]..."`	alias with *min*, *max* clipping

7.11 DSP with polyphonic support

When the DSP code is compiled in polyphonic mode, the generated program will create a more complex hierarchy to possibly access and control individual voices.

The following OSC messages reflect the same DSP code either compiled normally, or in polyphonic mode (only part of the OSC hierarchies are displayed here):

```
// Mono mode

/Organ/vol f -10.0
/Organ/pan f 0.0
```

```
// Polyphonic mode

/Polyphonic/Voices/Organ/pan f 0.0
/Polyphonic/Voices/Organ/vol f -10.0
...
/Polyphonic/Voice1/Organ/vol f -10.0
/Polyphonic/Voice1/Organ/pan f 0.0
...
/Polyphonic/Voice2/Organ/vol f -10.0
/Polyphonic/Voice2/Organ/pan f 0.0
```

Note that to save space on the screen, the `/Polyphonic/VoiceX/xxx` syntax is used when the number of allocated voices is less than 8, then the `/Polyphonic/VX/xxx` syntax is used when more voices are used.

Chapter 8

HTTP support

Like OSC, several FAUST architectures also provide HTTP support. This allows FAUST applications to be remotely controlled from any web browser using specific URLs. Moreover, OSC and HTTPD can be freely combined.

While OSC support is installed by default when FAUST is built, this is not the case for HTTP. That's because it depends on the GNU *libmicrohttpd* library, which is usually not installed by default on the system. An additional `make httpd` step is therefore required when compiling and installing FAUST:

```
make httpd
make
sudo make install
```

Note that `make httpd` will fail if *libmicrohttpd* is not available on the system.

HTTP support can be activated using the `-httpd` option when building the audio application with the appropriate `faust2xxx` command. The following table (Table 8.1) lists the FAUST architectures that provide HTTP support.

8.1 A Simple Example

To illustrate how HTTP support works, let's reuse our previous `mix4.dsp` example, a very simplified monophonic audio mixer with four inputs and one output. For each input we have a mute button and a level slider:

```
input(v) = vgroup("input %v", *(1-checkbox("mute"))
         : *(vslider("level", 0, 0, 1, 0.01)));
process  = hgroup("mixer", par(i, 4, input(i)) :> _);
```

Audio system	Environment	HTTP support
Linux		
Alsa	GTK, Qt, Console	yes
Jack	GTK, Qt, Console	yes
Netjack	GTK, Qt, Console	yes
PortAudio	GTK, Qt	yes
Mac OS X		
CoreAudio	Qt	yes
Jack	Qt, Console	yes
Netjack	Qt, Console	yes
PortAudio	Qt	yes
Windows		
Jack	Qt, Console	yes
PortAudio	Qt	yes

Table 8.1: FAUST architectures with HTTP support.

We are going to compile this example as a standalone JACK/Qt application with HTTP support using the command:

```
faust2jaqt -httpd mix4.dsp
```

The effect of the `-httpd` option is to embed a small web server into the application, whose purpose is to serve an HTML page representing its user interface. This page makes use of JavaScript and SVG and closely resembles the native Qt interface.

When we start the application from the command line:

```
./mix4
```

we get various information on the standard output, including:

```
Faust httpd server version 0.72 is running on TCP port
   5510
```

As we can see, the embedded web server runs by default on TCP port 5510. The entry point is http://localhost:5510. It can be opened from any recent browser and it produces the page shown in Figure 8.1.

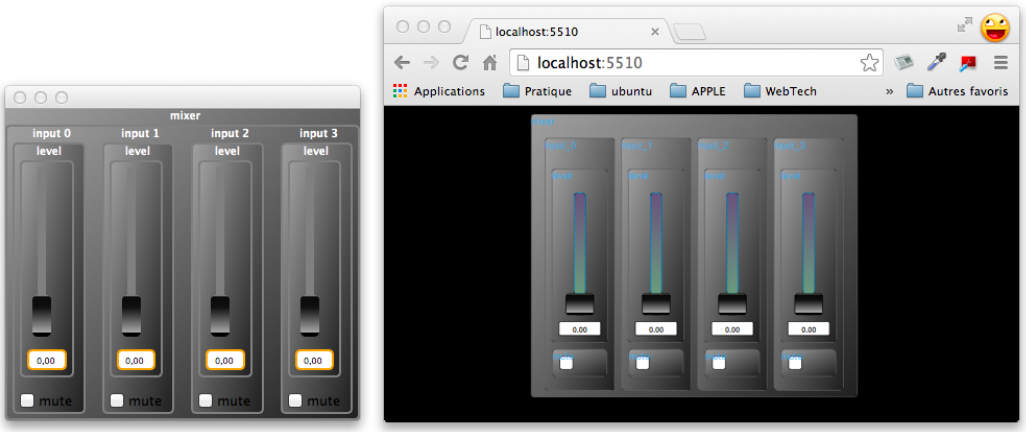


Figure 8.1: User interface of mix4.dsp in a Web browser

8.2 JSON Description of the User Interface

The communication between the application and the web browser is based on several underlying URLs. The first one is http://localhost:5510/JSON, which returns a JSON description of the user interface of the application. This JSON description is used internally by the JavaScript code to build the graphical user interface. Here is (part of) the JSON returned by `mix4`:

```
{
  "name": "mix4",
  "address": "YannAir.local",
  "port": "5511",
  "ui": [
    {
      "type": "hgroup",
      "label": "mixer",
      "items": [
        {
          "type": "vgroup",
          "label": "input_0",
          "items": [
            {
              "type": "vslider",
              "label": "level",
              "address": "/mixer/input_0/level",
              "init": "0", "min": "0", "max": "1",
              "step": "0.01"
            },
```

```
            {
              "type": "checkbox",
              "label": "mute",
              "address": "/mixer/input_0/mute",
              "init": "0", "min": "0", "max": "0",
              "step": "0"
            }
          ]
        },

        ...

      ]
    }
  ]
}
```

8.3 Querying the State of the Application

Each widget has a unique "address" field that can be used to query its value. In our example, the level of input 0 has the address `/mixer/input_0/level`. The address can be used to forge a URL to get the value of the widget: http://localhost:5510/mixer/input_0/level, resulting in:

```
/mixer/input_0/level 0.00000
```

Multiple widgets can be queried at once by using an address higher in the hierarchy. For example, to get the values of the level and the mute state of input 0 we use http://localhost:5510/mixer/input_0, resulting in:

```
/mixer/input_0/level 0.00000
/mixer/input_0/mute  0.00000
```

To get all the values at once we simply use http://localhost:5510/mixer, resulting in:

```
/mixer/input_0/level 0.00000
/mixer/input_0/mute  0.00000
/mixer/input_1/level 0.00000
/mixer/input_1/mute  0.00000
/mixer/input_2/level 0.00000
/mixer/input_2/mute  0.00000
/mixer/input_3/level 0.00000
/mixer/input_3/mute  0.00000
```

8.4 Changing the value of a widget

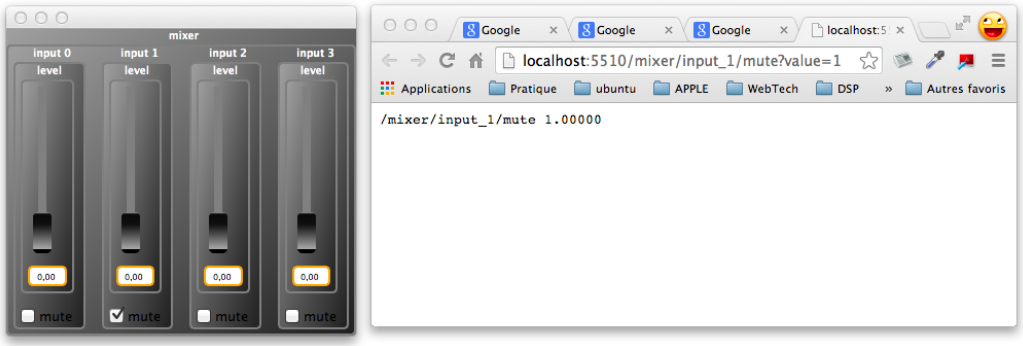


Figure 8.2: Muting input 1 by forging the appropriate URL

Let's say that we want to mute input 1 of our mixer. We can use the URL http://localhost:5510/mixer/input_1/mute?value=1 obtained by concatenating ?value=1 at the end of the widget URL.

All widgets can be controlled similarly. For example, http://localhost:5510/mixer/input_3/level?value=0.7 will set the input 3 level to 0.7.

8.5 Proxy control access to the Web server

A control application may want to access and control the running DSP using its web server, but without using the delivered HTML page in a browser. Since the complete JSON can be retrieved, control applications can be developed purely in C/C++, then build a *proxy* version of the user interface, and set and get parameters using HTTP requests.

This mode can be started dynamically using the *-server URL* parameter. Assuming an application with HTTP support is running remotely on the given URL, the control application will fetch its JSON description, use it to dynamically build the user interface, and allow access to the remote parameters.

8.6 HTTP cheat sheet

Here is a summary of the various URLs used to interact with the application's Web server.

Default ports

`5510`	default TCP port used by the application's Web server
`5511...`	alternative TCP ports

Command-line options

`-port` n	set the TCP port number used by the application's Web server
`-server` URL	start a proxy control application accessing the remote application running on the give

URLs

`http://`*host*`:`*port*	the base URL to be used in proxy control access mode
`http://`*host*`:`*port*`/JSON`	get a JSON description of the user interface
`http://`*host*`:`*port*`/`*address*	get the value of a widget or a group of widgets
`http://`*host*`:`*port*`/`*address*`?value=`v	set the value of a widget to v

JSON

Top level

The json describes the name, host and port of the application and a hierarchy of user interface items:

```
{
  "name": <name>,
  "address": <host>,
  "port": <port>,
  "ui": [ <item> ]
}
```

An `<item>` is either a group (of items) or a widget.

Groups

A group is essentially a list of items with a specific layout:

```
{
    "type": <type>,
    "label": <label>,
    "items": [ <item>, <item>,...]
}
```

The `<type>` defines the layout. It can be either `"vgroup"`, `"hgroup"` or `"tgroup"`

Widgets

```
{
    "type": <type>,
    "label": <label>,
    "address": <address>,
    "meta": [ { "key": "value"},... ],
    "init": <num>,
    "min": <num>,
    "max": <num>,
    "step": <num>
},
```

Widgets are the basic items of the user interface. They can be of different `<type>` : `"button"`, `"checkbox"`, `"nentry"`, `"vslider"`, `"hslider"`, `"vbargraph"` or `"hbargraph"`.

Chapter 9

MIDI support

Like OSC, several FAUST architectures also provide MIDI support. This allows FAUST applications to be controlled from any MIDI device (or to control MIDI devices). MIDI is also the preferred way to control polyphonic instruments.

9.1 Describing MIDI messages in the DSP source code

MIDI control messages are described as metadata in UI elements. They are decoded by a special architecture *MidiUI* class that will parse incoming MIDI messages and update the appropriate control parameters, or send MIDI messages when the UI elements (sliders, buttons...) are moved.

9.2 Description of the possible standard MIDI messages

Below, when a 7-bit MIDI parameter is used to drive a button or checkbox, its maximum value 127 maps to 1 ("on") while its minimum value 0 maps to 0 ("off").

A special `[midi:xxx yyy...]` metadata entry needs to be added to the UI element description. The more usual MIDI messages can be used as described here:

- `[midi:ctrl num]` in a slider or bargraph will map the UI element value to (0, 127) range. When used with a button or checkbox, 1 will be mapped to 127, 0 will be mapped to 0,

- `[midi:keyon pitch]` in a slider or bargraph will register the UI element's state-variable to be driven by MIDI note-on velocity (an integer between 0 and 127) of the specified

key between 0 and 127. When used with a button or checkbox, 1 will be mapped to 127, 0 will be mapped to 0,

- `[midi:keyoff pitch]` in a slider or bargraph will register the UI element's state-variable to be driven by MIDI note-off velocity (an integer between 0 and 127) of the specified key between 0 and 127. When used with a button or checkbox, 1 will be mapped to 127, 0 will be mapped to 0,

- `[midi:key pitch]` in a slider or bargraph will register the UI element's state-variable to be driven by MIDI note-on velocity (an integer between 0 and 127) of the specified key between 0 and 127. When used with a button or checkbox, 1 will be mapped to 127, 0 will be mapped to 0. Note-on and note-off events will be handled,

- `[midi:keypress pitch]` in a slider or bargraph will register the UI element's state-variable to be driven by the MIDI key-pressure (an integer between 0 and 127) from MIDI key,

- `[midi:pgm num]` in a slider or bargraph will map the UI element value to the progchange value, so *progchange* message with the same *num* value will be sent. When used with a button or checkbox, 1 will send the *progchange* message with *num* value, 0 will send nothing,

- `[midi:chanpress num]` in a slider or bargraph will map the UI element value to the chanpress value, so *chanpress* message with the same *num* value will be sent. When used with a button or checkbox, 1 will send the *chanpress* message with *num* value, 0 will send nothing,

- `[midi:pitchwheel]` in a slider or bargraph will map the UI element value to (0,16383) range. When used with a button or checkbox, 1 will be mapped to 16383, 0 will be mapped to 0.

9.3 A Simple Example

Here is an example with a *volume* slider controlled with MIDI ctrlchange 7 messages:

```
//-----------------------------------------------
//  Volume MIDI control in dB
//-----------------------------------------------

import("music.lib");

smooth(c) = *(1-c) : +~*(c);
gain = vslider("Volume [midi:ctrl 7]", 0, -70, +4, 0.1) :
    db2linear : smooth(0.999);
```

```
process = *(gain);
```

A complete testing example named *midi_tester.dsp* is available in the FAUST distribution *examples* folder.

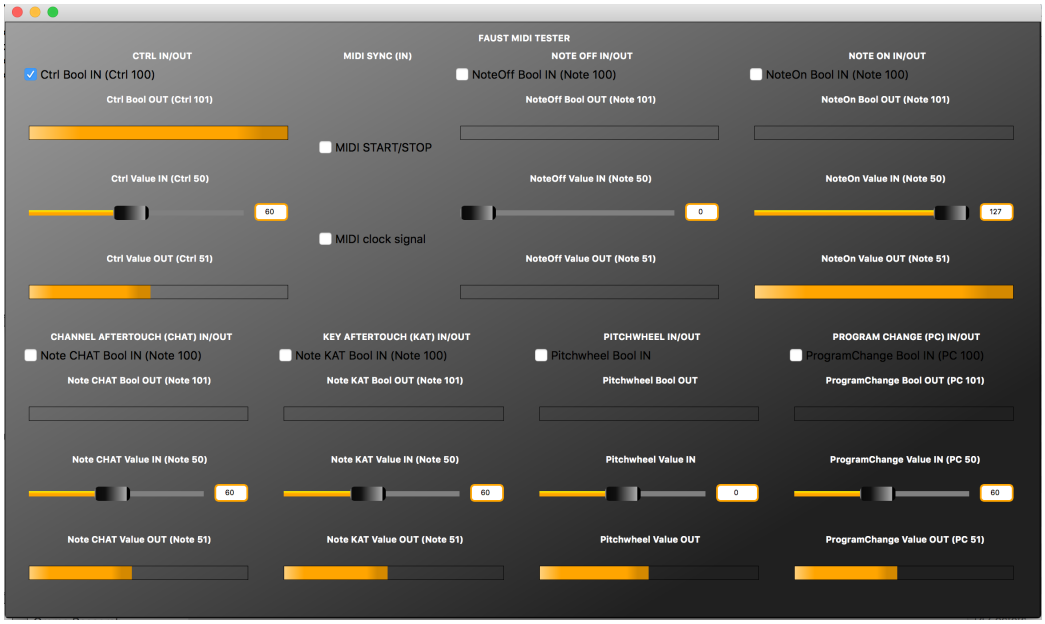


Figure 9.1: MIDI messages testing example

MIDI support can be activated using the `-midi` option when building the audio application with the appropriate `faust2xxx` command. The following table (Table 9.1) lists the FAUST architectures that provide MIDI support.

Audio system	Environment	HTTP support
Linux		
Alsa	Qt	yes
Jack	Qt	yes
Mac OS X		
CoreAudio	Qt	yes
Jack	Qt	yes

Table 9.1: FAUST architectures with MIDI support.

9.4 MIDI Synchronization

MIDI clock-based synchronization can be used to slave a given FAUST program. The following three messages need to be used:

- `[midi:start]` in a button or checkbox will trigger a value of 1 when a *start* MIDI message is received

- `[midi:stop]` in a button or checkbox will trigger a value of 0 when a *stop* MIDI message is received

- `[midi:clock]` in a button or checkbox will deliver a sequence of successive 1 and 0 values each time a *clock* MIDI message is received, seen by FAUST code as a square command signal, to be used to compute higher level information.

A typical FAUST program will then use the MIDI clock stream to compute the BPM information, or for any synchronization need it may have. Here is a simple example of a sine wave generated with a frequency controlled by the MIDI clock stream, starting and stopping when receiving the MIDI start/stop messages:

```
import("music.lib");

// square signal (1/0), changing state at each received
   clock
clocker = checkbox("MIDI clock[midi:clock]");

// ON/OFF button controlled with MIDI start/stop messages
play = checkbox("ON/OFF [midi:start] [midi:stop]");

// detect front
front(x) = x != x';

// count number of peaks during one second
freq(x) = (x-x@SR) : + ~ _;

process = osc(8*freq(front(clocker))) * play;
```

Chapter 10

Polyphonic support

Directly programming polyphonic instruments in FAUST is perfectly possible. It is also necessary when very complex signal interactions between the different voices have to be described[1].

But since all voices would always be computed, this approach could be too CPU-costly for simpler or more limited needs. In this case, describing a single voice in a FAUST DSP program and externally combining several of them with a special *polyphonic instrument aware* architecture file is a better solution. Moreover, this special architecture file takes care of dynamic voice allocation and MIDI message decoding and mapping.

10.1 Polyphony-ready DSP code

By convention FAUST architecture files with polyphonic capabilities expect to find control parameters named *freq*, *gain*, and *gate*. Metadata such as `declare nvoices "8";` can be added to the source code to specify a desired number of voices.

In the case of MIDI control, the *freq* parameter (which should be a frequency) will be automatically computed from MIDI note numbers, *gain* (which should be a value between 0 and 1) from velocity, and *gate* from *keyon/keyoff* events. Thus, gate can be used as a trigger signal for any envelope generator, etc.

10.2 Using the mydsp_poly class

A single voice has to be described by a FAUST DSP program, and the `mydsp_poly` class is then used to combine several voices and create a polyphony-ready DSP:

[1] Like sympathetic string resonance in a physical model of a piano, for instance.

- the *faust/dsp/poly-dsp.h* file contains the definition of the `mydsp_poly` class used to wrap the DSP voice into the polyphonic architecture. This class maintains an array of `dsp`-type objects, manages dynamic voice allocation, decodes and maps MIDI messages, mixes all running voices, and stops a voice when its output level decreases below a given threshold.

- as a subclass of DSP, the `mydsp_poly` class redefines the `buildUserInterface` method. By convention all allocated voices are grouped in a global *Polyphonic* tab group. The first tab contains a *Voices* group, a master-like component used to change parameters on all voices at the same time, with a *Panic* button to stop running voices[2], followed by one tab for each voice. Graphical user interface components will then reflect the multi-voice structure of the new polyphonic DSP (Figure 10.1).



Figure 10.1: Extended multi-voices GUI interface

The resulting polyphonic DSP object can be used as usual, connected with the needed audio driver, and possibly other UI control objects like OSCUI, httpdUI, etc. Having

[2] An internal control grouping mechanism has been defined to automatically dispatch a user interface action done on the master component to all linked voices, except for the *freq*, *gain*, and *gate* controls.

this new UI hierarchical view allows complete OSC control of each single voice and their control parameters, but also all voices using the master component.

The following OSC messages reflect the same DSP code either compiled normally, or in polyphonic mode (only part of the OSC hierarchies are displayed here):

```
// Mono mode

/0x00/0x00/vol f -10.0
/0x00/0x00/pan f 0.0

// Polyphonic mode

/Polyphonic/Voices/0x00/0x00/pan f 0.0
/Polyphonic/Voices/0x00/0x00/vol f -10.0
...
/Polyphonic/Voice1/0x00/0x00/vol f -10.0
/Polyphonic/Voice1/0x00/0x00/pan f 0.0
...
/Polyphonic/Voice2/0x00/0x00/vol f -10.0
/Polyphonic/Voice2/0x00/0x00/pan f 0.0
...
```

The polyphonic instrument allocator takes the DSP to be used for one voice[3], the desired number of voices, the *dynamic voice allocation* state[4], and the *group* state, which controls whether separate voices are displayed (Figure 10.1):

```
    DSP = new mydsp_poly(dsp, 2, true, true);
```

With the following code, note that a polyphonic instrument may be used outside of a MIDI control context, so that all voices will always be running and can be controlled with OSC messages, for instance:

```
    DSP = new mydsp_poly(dsp, 8, false, true);
```

10.3 Controlling the polyphonic instrument

The `mydsp_poly` class is also ready for MIDI control and can react to *keyon/keyoff* and *pitchwheel* messages. Other MIDI control parameters can directly be added in the DSP source code.

[3]The DSP object will be automatically cloned in the mydsp_poly class to create all needed voices.
[4]Voices may always be running, or dynamically started/stopped in case of MIDI control.

10.4 Deploying the polyphonic instrument

Several architecture files and associated scripts have been updated to handle polyphonic instruments:

As an example on macOS, the script `faust2caqt foo.dsp` can be used to create a polyphonic CoreAudio/Qt application. The desired number of voices is either declared via `nvoices` metadata or changed with the `-nvoices num` additional parameter[5]. MIDI control is activated using the `-midi` parameter.

The number of allocated voices can possibly be changed at runtime using the `-nvoices` parameter to change the default value (so using `./foo -nvoices 16` for instance).

Several other scripts have been adapted using the same conventions.

10.5 Polyphonic instrument with a global output effect

Polyphonic instruments may be used with an output effect. Putting that effect in the main FAUST code is not a good idea since it would be instantiated for each voice which would be very inefficient. This is a typical use case for the `dsp_sequencer` class previously presented with the polyphonic DSP connected in sequence with a unique global effect (Figure 10.2).

`faustcaqt inst.dsp -effect effect.dsp` with inst.dsp and effect.dsp in the same folder, and the number of outputs of the instrument matching the number of inputs of the effect, has to be used. A `dsp_sequencer` object will be created to combine the polyphonic instrument in sequence with the single output effect.

Polyphony-ready *faust2xx* scripts will then compile the polyphonic instrument and the effect, combine them in sequence, and create a ready-to-use DSP.

10.5.1 Integrated global output effect

Starting with the 2.5.17 version, a new convention has been defined to directly integrate a global output effect inside the DSP source code itself. The effect has simply to be declared in a `effect = effect_code;` line in the source. Here is a more complete source code example:

```
import("stdfaust.lib");
process = pm.clarinet_ui_MIDI <: _,_;
effect = dm.freeverb_demo;
```

[5]The `-nvoices` parameter takes precedence over the metadata value.

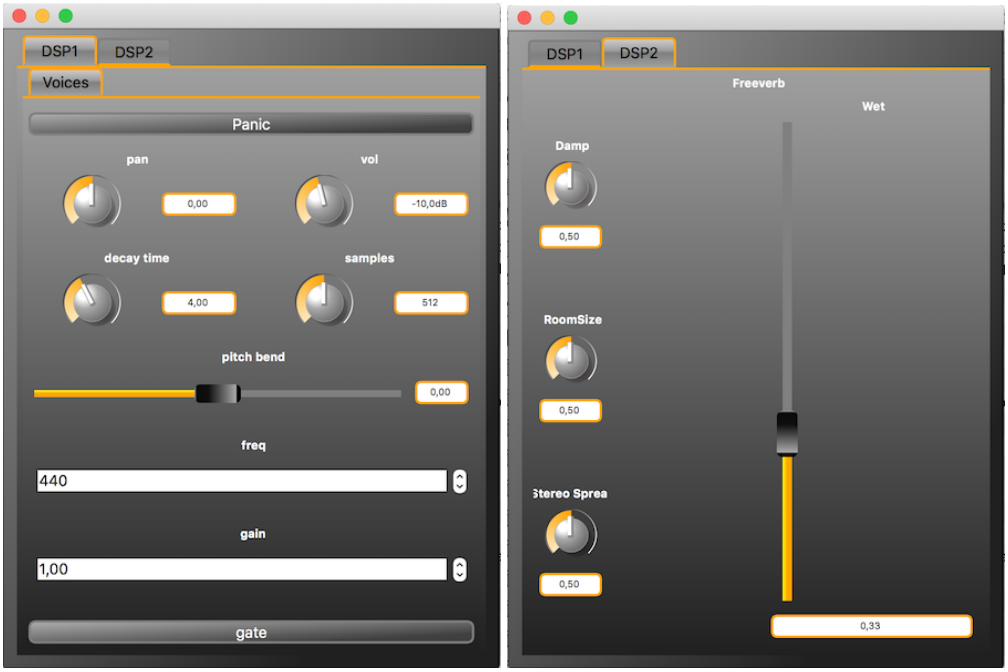


Figure 10.2: Polyphonic instrument with output effect GUI interface: left tab window shows the polyphonic instrument with its *Voices* group only, right tab window shows the output effect.

The architecture script then separates the instrument description itself (the `process = ...` definition) from the effect definition (the `effect = ...` definition), possibly adapts the instrument number of outputs to the effect number of inputs, compiles each part separately, and combines them with the `dsp_sequencer` object.

A new `auto` parameter has been defined for *faust2xx* scripts, as in the `faustcaqt inst.dsp -effect auto` command, for example.

10.5.2 Integrated global output effect and libfaust

For developers using the libfaust library, a helper file named `faust/dsp/poly-dsp-tools.h` is available. It defines an API to automatically create a polyphonic instrument with an output effect, starting from a DSP source file using the `effect = ...` convention. The function `createPolyDSPFactoryFromString` or `createPolyDSPFactoryFromFile` must be used to create the polyphonic DSP factory. Next, the `createPolyDSPInstance` function creates the polyphonic object (a subclass of the `dsp_poly` type) to be used like a regular `dsp` type object.

After the DSP factory has been compiled, your application or plugin may want to save/restore it in order to save FAUST to LLVM IR compilation or even JIT compilation

time at next use. To get the internal factory compiled code, several functions are available:

- `writePolyDSPFactoryToIRFile` allows you to save the polyphonic factory LLVM IR (in textual format) in a file,
- `writePolyDSPFactoryToBitcodeFile` allows you to save the polyphonic factory LLVM IR (in binary format) in a file,
- `writePolyDSPFactoryToMachineFile` allows you to save the polyphonic factory executable machine code in a file.

To re-create a DSP factory from a previously saved code, several functions are available:

- `readPolyDSPFactoryFromIRFile` allows you to create a polyphonic DSP factory from a file containing the LLVM IR (in textual format),
- `readPolyDSPFactoryFromBitcodeFile` allows you to create a polyphonic factory from a file containing the LLVM IR (in binary format),
- `readPolyDSPFactoryFromMachineFile` allows you to create a polyphonic DSP factory from a file containing the executable machine code.

Chapter 11

Controlling the Code Generation

Several options of the FAUST compiler allow you to control the generated C++ code. By default the computations are done sample by sample in a single loop, but the compiler can also generate *vector* and *parallel* code.

11.1 Vector code generation

Modern C++ compilers are able to perform autovectorization, that is, to use SIMD instructions to speed up the code. These instructions can typically operate in parallel on short vectors of four single-precision floating point numbers, leading to a theoretical speed up of $\times 4$. Autovectorization of C/C++ programs is a difficult task. Current compilers are very sensitive to the way the code is arranged. In particular, overly complex loops can prevent autovectorization. The goal of vector code generation is to rearrange the C++ code in a way that makes the autovectorization job of the C++ compiler easier. Instead of generating a single sample computation loop, it splits the computation into several simpler loops that communicate through vectors.

The vector code generation is activated by passing the `--vectorize` (or `-vec`) option to the FAUST compiler. Two additional options are available: `--vec-size <n>` controls the size of the vector (by default 32 samples) and `--loop-variant 0/1` gives some additional control on the loops: `--loop-variant 0` generates fixed-size sub-loops with a final sub-loop that processes the last samples, `--loop-variant 1` generates sub-loops of variable vector size.

To illustrate the difference between scalar code and vector code, let's take the computation of the RMS (Root Mean Square) value of a signal. Here is the FAUST code that

computes the Root Mean Square of a sliding window of 1000 samples:

```
// Root Mean Square of n consecutive samples
RMS(n) = square : mean(n) : sqrt;

// Square of a signal
square(x) = x * x;

// Mean of n consecutive samples of a signal
// (uses fixed point to avoid the accumulation of
// rounding errors)
mean(n) = float2fix : integrate(n) :
          fix2float : /(n);

// Sliding sum of n consecutive samples
integrate(n,x) = x - x@n : +~_;

// Conversion between float and fixed point
float2fix(x) = int(x*(1<<20));
fix2float(x) = float(x)/(1<<20);

// Root Mean Square of 1000 consecutive samples
process = RMS(1000);
```

The compute() method generated in scalar mode is the following:

```
virtual void compute (int count,
                      float** input,
                      float** output)
{
  float* input0 = input[0];
  float* output0 = output[0];
  for (int i=0; i<count; i++) {
    float fTemp0 = input0[i];
    int iTemp1 = int(1048576*fTemp0*fTemp0);
    iVec0[IOTA&1023] = iTemp1;
    iRec0[0] = ((iVec0[IOTA&1023] + iRec0[1])
                    - iVec0[(IOTA-1000)&1023]);
    output0[i] = sqrtf(9.536744e-10f *
                       float(iRec0[0]));
    // post processing
    iRec0[1] = iRec0[0];
    IOTA = IOTA+1;
  }
}
```

The `-vec` option leads to the following reorganization of the code:

```
virtual void compute (int fullcount,
                      float** input,
                      float** output)
{
  int iRec0_tmp[32+4];
  int* iRec0 = &iRec0_tmp[4];
  for (int index=0; index<fullcount; index+=32)
  {
    int count = min (32, fullcount-index);
    float* input0 = &input[0][index];
    float* output0 = &output[0][index];
    for (int i=0; i<4; i++)
      iRec0_tmp[i]=iRec0_perm[i];
    // SECTION : 1
    for (int i=0; i<count; i++) {
      iYec0[(iYec0_idx+i)&2047] =
               int(1048576*input0[i]*input0[i]);
    }
    // SECTION : 2
    for (int i=0; i<count; i++) {
      iRec0[i] = ((iYec0[i] + iRec0[i-1]) -
               iYec0[(iYec0_idx+i-1000)&2047]);
    }
    // SECTION : 3
    for (int i=0; i<count; i++) {
      output0[i] = sqrtf((9.536744e-10f *
                 float(iRec0[i])));
    }
    // SECTION : 4
    iYec0_idx = (iYec0_idx+count)&2047;
    for (int i=0; i<4; i++)
      iRec0_perm[i]=iRec0_tmp[count+i];
  }
}
```

While the second version of the code is more complex, it turns out to be much easier for the C++ compiler to vectorize efficiently. Using Intel icc 11.0, with the exact same compilation options: `-O3 -xHost -ftz -fno-alias -fp-model fast=2`, the scalar version achieves a throughput of 129.144 MB/s, while the vector version reaches 359.548 MB/s, a 2.8× speed up.

Vector code generation is built on top of scalar code generation (see Figure 11.1). Every time an expression needs to be compiled, the compiler checks whether it requires a

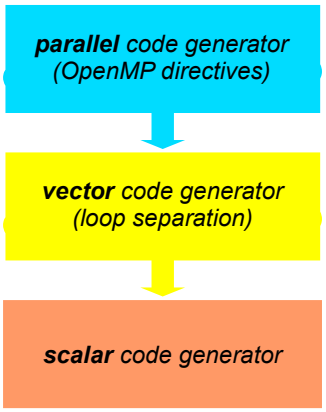


Figure 11.1: FAUST's stack of code generators

separate loop or not. It applies some simple rules for that. Expressions that are shared (and are complex enough) are good candidates to be compiled in a separate loop, as well as recursive expressions and expressions used in delay lines.

The result is a directed graph in which each node is a computation loop (see Figure 11.2). This graph is stored in the klass object and a topological sort is applied to it before printing the code.

11.2 Parallel code generation

Parallel code generation is activated by passing either the `--openMP` (or `-omp`) option or the `--scheduler` (or `-sch`) option. It implies the `-vec` option, as parallel code generation is built on top of vector code generation.

11.2.1 The OpenMP code generator

The `--openMP` (or `-omp`) option given to the FAUST compiler inserts appropriate OpenMP directives in the C++ code. OpenMP (http://www.openmp.org) is a well-established API that is used to explicitly define direct multi-threaded, shared-memory parallelism. It is based on a fork-join model of parallelism (see Figure 11.3). Parallel regions are delimited by `#pragma omp parallel` constructs. At the entrance of a parallel region a team of

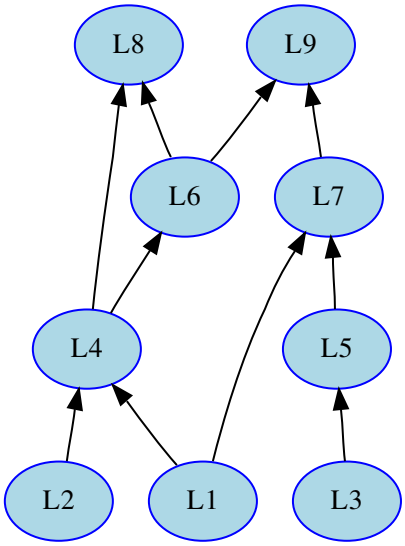


Figure 11.2: The result of the -vec option is a directed acyclic graph (DAG) of small computation loops

parallel threads is activated. The code within a parallel region is executed by each thread of the parallel team until the end of the region.

```
#pragma omp parallel
{
  // the code here is executed simultaneously by
  // every thread of the parallel team
  ...
}
```

To avoid having every thread redundantly do the exact same work, OpenMP provides specific *work-sharing* directives. For example `#pragma omp sections` allows you to break the work into separate, discrete sections, each section being executed by one thread:

```
#pragma omp parallel
{
  #pragma omp sections
  {
    #pragma omp section
    {
      // job 1
```

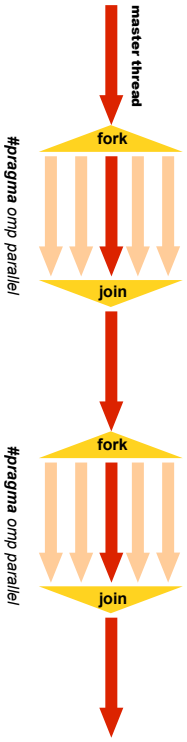


Figure 11.3: OpenMP is based on a fork-join model

```
    }
    #pragma omp section
    {
      // job 2
    }
    ...
  }

  ...
}
```

11.2.2 Adding OpenMP directives

As mentioned earlier, parallel code generation is built on top of vector code generation. The graph of loops produced by the vector code generator is topologically sorted in order to detect the loops that can be computed in parallel. The first set S_0 (loops $L1$, $L2$, and $L3$ in the DAG of Figure 11.2) contains the loops that do not depend on any other

loops. The set S_1 contains the loops that depend only on loops of S_0 (that is, loops $L4$ and $L5$), etc.

As all the loops of a given set S_n can be computed in parallel, the compiler will generate a `sections` construct with a `section` for each loop.

```
#pragma omp sections
{
  #pragma omp section
  for (...) {
    // Loop 1
  }
  #pragma omp section
  for (...) {
    // Loop 2
  }
  ...
}
```

If a given set contains only one loop, then the compiler checks to see whether the loop can be parallelized (no recursive dependencies) or not. If it can be parallelized, it generates:

```
#pragma omp for
for (...) {
 // Loop code
}
```

otherwise it generates a `single` construct so that only one thread executes the loop:

```
#pragma omp single
for (...) {
 // Loop code
}
```

11.2.3 Example of parallel OpenMP code

To illustrate how FAUST uses the OpenMP directives, here is a very simple example: two 1-pole filters in parallel connected to an adder (see Figure 11.4 for the corresponding block-diagram):

```
filter(c) = *(1-c) : + ~ *(c);
process = filter(0.9), filter(0.9) : +;
```

The corresponding compute() method obtained using the `-omp` option is the following:

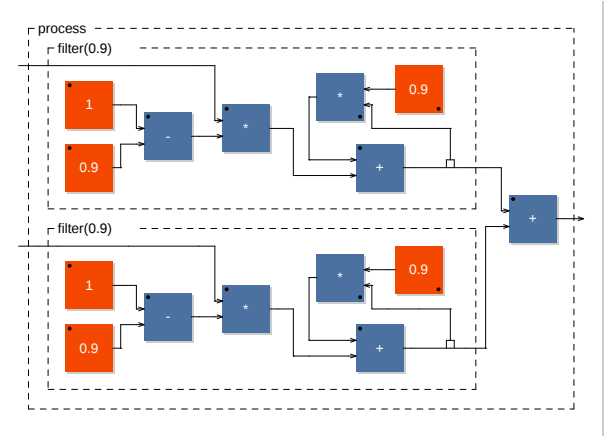


Figure 11.4: Two filters in parallel connected to an adder

```
virtual void compute (int fullcount,
                      float** input,
                      float** output)
{
  float   fRec0_tmp[32+4];
  float   fRec1_tmp[32+4];
  float*  fRec0 = &fRec0_tmp[4];
  float*  fRec1 = &fRec1_tmp[4];
  #pragma omp parallel firstprivate(fRec0,fRec1)
  {
    for (int index = 0; index < fullcount;
                                index += 32)
    {
      int count = min (32, fullcount-index);
      float* input0 = &input[0][index];
      float* input1 = &input[1][index];
      float* output0 = &output[0][index];
      #pragma omp single
      {
        for (int i=0; i<4; i++)
          fRec0_tmp[i]=fRec0_perm[i];
        for (int i=0; i<4; i++)
          fRec1_tmp[i]=fRec1_perm[i];
      }
      // SECTION : 1
      #pragma omp sections
```

```
      {
        #pragma omp section
        for (int i=0; i<count; i++) {
          fRec0[i] = ((0.1f * input1[i])
                    + (0.9f * fRec0[i-1]));
        }
        #pragma omp section
        for (int i=0; i<count; i++) {
          fRec1[i] = ((0.1f * input0[i])
                    + (0.9f * fRec1[i-1]));
        }
      }
      // SECTION : 2
      #pragma omp for
      for (int i=0; i<count; i++) {
        output0[i] = (fRec1[i] + fRec0[i]);
      }
      // SECTION : 3
      #pragma omp single
      {
        for (int i=0; i<4; i++)
          fRec0_perm[i]=fRec0_tmp[count+i];
        for (int i=0; i<4; i++)
          fRec1_perm[i]=fRec1_tmp[count+i];
      }
    }
  }
}
```

This code requires some comments:

1. The parallel construct `#pragma omp parallel` is the fundamental construct that starts parallel execution. The number of parallel threads is generally the number of CPU cores but it can be controlled in several ways.

2. Variables external to the parallel region are shared by default. The pragma `firstprivate (fRec0,fRec1)` indicates that each thread should have its private copy of fReco and fRec1. The reason is that accessing shared variables requires an indirection and is quite inefficient compared to private copies.

3. The top-level loop `for (int index = 0;...)...` is executed by all threads simultaneously. The subsequent work-sharing directives inside the loop indicate how the work must be shared between the threads.

4. Please note that an implied barrier exists at the end of each work-sharing region. All threads must have executed the barrier before any of them can continue.

5. The work-sharing directive `#pragma omp single` indicates that this first section will be executed by only one thread (any of them).

6. The work-sharing directive `#pragma omp sections` indicates that each corresponding `#pragma omp section`, here our two filters, will be executed in parallel.

7. The loop construct `#pragma omp for` specifies that the iterations of the associated loop will be executed in parallel. The iterations of the loop are distributed across the parallel threads. For example, if we have two threads, the first one can compute indices between 0 and count/2 and the other one between count/2 and count.

8. Finally, `#pragma omp single` in section 3 indicates that this last section will be executed by only one thread (any of them).

11.2.4 The scheduler code generator

With the `--scheduler` (or `-sch`) option given to the FAUST compiler, the computation graph is cut into separate computation loops (called "tasks"), and a "Work Stealing Scheduler" is used to activate and execute them following their dependencies. A pool of worker threads is created and each thread uses its own local WSQ (Work Stealing Queue) of tasks. A WSQ is a special queue with a push operation, a "private" LIFO pop operation, and a "public" FIFO pop operation.

Starting from a ready task, each thread follows the dependencies, possibly pushing ready sub-tasks into its own local WSQ. When no more tasks can be activated on a given computation path, the thread pops a task from its local WSQ. If the WSQ is empty, then the thread is allowed to "steal" tasks from another thread's WSQ.

The local LIFO pop operation allows better cache locality, and the FIFO steal pop grabs larger chunks of work to be done. The reason for this is that many work-stealing workloads are divide-and-conquer in nature; stealing one of the oldest tasks implicitly also steals a (potentially) large subtree of computations that will unfold once that piece of work is stolen and run.

Compared to the OpenMP model (`-omp`), the new model is worse for simple FAUST programs and usually starts to behave comparably, or sometimes better, for "complex enough" FAUST programs. In any case, since OpenMP does not behave so well with GCC compilers (only quite recent versions like GCC 4.4 start to show some improvements) and is unusable on macOS in real-time contexts, this new scheduler option has its own value. We plan to improve it by adding a "pipelining" idea in the future.

11.2.5 Example of parallel scheduler code

To illustrate how FAUST generates the scheduler code, here is a very simple example: two 1-pole filters in parallel connected to an adder (see Figure 11.4 for the corresponding block-diagram):

```
filter(c) = *(1-c) : + ~ *(c);
process = filter(0.9), filter(0.9) : +;
```

When the `-sch` option is used, the content of the additional *architecture/scheduler.h* file is inserted in the generated code. It contains code to deal with WSQ and thread management. The `compute()` and `computeThread()` methods are the following:

```
virtual void compute (int fullcount,
                      float** input,
                      float** output)
{
    GetRealTime();
    this->input = input;
    this->output = output;
    StartMeasure();
    for (fIndex = 0; fIndex < fullcount; fIndex += 32) {
        fFullCount = min (32, fullcount-fIndex);
        TaskQueue::Init();
        // Initialize end task
        fGraph.InitTask(1,1);
        // Only initialize tasks with inputs
        fGraph.InitTask(4,2);
        fIsFinished = false;
        fThreadPool.SignalAll(fDynamicNumThreads - 1);
        computeThread(0);
        while (!fThreadPool.IsFinished()) {}
    }
    StopMeasure(fStaticNumThreads,
        fDynamicNumThreads);
}
void computeThread (int cur_thread) {
    float*  fRec0 = &fRec0_tmp[4];
    float*  fRec1 = &fRec1_tmp[4];
    // Init graph state
    {
        TaskQueue taskqueue;
        int tasknum = -1;
        int count = fFullCount;
```

```
        // Init input and output
        FAUSTFLOAT* input0 = &input[0][fIndex];
        FAUSTFLOAT* input1 = &input[1][fIndex];
        FAUSTFLOAT* output0 = &output[0][fIndex];
        int task_list_size = 2;
        int task_list[2] = {2,3};
        taskqueue.InitTaskList(task_list_size, task_list,
            fDynamicNumThreads, cur_thread, tasknum);
        while (!fIsFinished) {
            switch (tasknum) {
                case WORK_STEALING_INDEX: {
                    tasknum = TaskQueue::GetNextTask(
                        cur_thread);
                    break;
                }
                case LAST_TASK_INDEX: {
                    fIsFinished = true;
                    break;
                }
                // SECTION : 1
                case 2: {
                    // LOOP 0x101111680
                    // pre processing
                    for (int i=0; i<4; i++) fRec0_tmp[i]=
                        fRec0_perm[i];
                    // exec code
                    for (int i=0; i<count; i++) {
                        fRec0[i] = ((1.000000e-01f * (
                            float)input1[i]) + (0.9f *
                            fRec0[i-1]));
                    }
                    // post processing
                    for (int i=0; i<4; i++) fRec0_perm[i
                        ]=fRec0_tmp[count+i];

                    fGraph.ActivateOneOutputTask(
                        taskqueue, 4, tasknum);
                    break;
                }
                case 3: {
                    // LOOP 0x1011125e0
                    // pre processing
                    for (int i=0; i<4; i++) fRec1_tmp[i]=
```

```
                        fRec1_perm[i];
                    // exec code
                    for (int i=0; i<count; i++) {
                        fRec1[i] = ((1.000000e-01f * (
                            float)input0[i]) + (0.9f *
                            fRec1[i-1]));
                    }
                    // post processing
                    for (int i=0; i<4; i++) fRec1_perm[i
                        ]=fRec1_tmp[count+i];

                    fGraph.ActivateOneOutputTask(
                        taskqueue, 4, tasknum);
                    break;
                }
                case 4: {
                    // LOOP 0x101111580
                    // exec code
                    for (int i=0; i<count; i++) {
                        output0[i] = (FAUSTFLOAT)(fRec1[i
                            ] + fRec0[i]);
                    }

                    tasknum = LAST_TASK_INDEX;
                    break;
                }
            }
        }
    }
}
```

Chapter 12

Mathematical Documentation

The FAUST compiler provides a mechanism to produce a self-describing documentation of the mathematical semantics of a FAUST program, essentially as a pdf file. The corresponding options are `-mdoc` (short) or `--mathdoc` (long).

12.1 Goals of the mathdoc

There are four main goals, or uses, of this mathematical documentation:

1. to preserve the DSP source code with all the needed libraries, so that the DSP can be compiled with a more recent version of the compiler and produce the same resulting program. This is the way we allow the libraries themselves to evolve (even without maintaining compatibility with older versions), while still allowing an older program to be compiled with newer versions of the compiler;
2. to preserve signal processors, independently from any computer language but only under a mathematical form;
3. to bring some help for debugging tasks, by showing the formulas as they are really computed after the compilation stage;
4. to give a new teaching support, as a bridge between code and formulas for signal processing.

12.2 Installation requirements

- `faust`, of course!

- `svg2pdf` (from the Cairo 2D graphics library), to convert block-diagrams, as LaTeX doesn't eat SVG directly yet...
- `breqn`, a LaTeX package to handle automatic breaking of long equations,
- `pdflatex`, to compile the LaTeX output file.

12.3 Generating the mathdoc

The easiest way to generate the complete mathematical documentation is to call the `faust2mathdoc` script on a FAUST file, as the `-mdoc` option leaves the documentation production unfinished. For example:

```
faust2mathdoc noise.dsp
```

12.3.1 Invoking the -mdoc option

Calling directly `faust -mdoc` does only the first part of the work, generating:

- a top-level directory, suffixed with "`-mdoc`",
- 5 subdirectories (`cpp/`, `pdf/`, `src/`, `svg/`, `tex/`),
- a LaTeX file containing the formulas,
- SVG files for block-diagrams.

At this stage:

- `cpp/` remains empty,
- `pdf/` remains empty,
- `src/` contains all the used FAUST sources (so all needed libraries to have a self-contained DSP),
- `svg/` contains SVG block-diagram files,
- `tex/` contains the generated LaTeX file.

12.3.2 Invoking faust2mathdoc

The `faust2mathdoc` script calls `faust --mathdoc` first, then it finishes the work:

- moving the output C++ file into `cpp/`,
- converting all SVG files into pdf files (you must have `svg2pdf` installed, from the Cairo 2D graphics library),
- launching `pdflatex` on the LaTeX file (you must have both `pdflatex` and the `breqn` package installed),
- moving the resulting pdf file into `pdf/`.

12.3.3 Online examples

To get an idea of the results of this mathematical documentation, which captures the mathematical semantic of FAUST programs, you can look at two pdf files online:

- http://faust.grame.fr/pdf/karplus.pdf (automatic documentation),
- http://faust.grame.fr/pdf/noise.pdf (manual documentation).

You can also generate all *mdoc* pdfs at once, simply invoking the `make mathdoc` command inside the `examples/` directory:

- for each `%.dsp` file, a complete `%-mdoc` directory will be generated,
- a single `allmathpdfs/` directory will gather all the generated pdf files.

12.4 Automatic documentation

By default, when no `<mdoc>` tag can be found in the input FAUST file, the `-mdoc` option automatically generates a LaTeX file with four sections:

1. "**Equations of process**", gathering all formulas needed for `process`,
2. "**Block-diagram schema of process**", showing the top-level block-diagram of `process`,
3. "**Notice of this documentation**", summing up generation and conventions information,
4. "**Complete listing of the input code**", listing all needed input files (including libraries).

12.5 Manual documentation

You can specify the documentation yourself instead of using the automatic mode, with five XML-like tags. That allows you to modify the presentation and to add your own comments, not only on `process`, but also about any expression you'd like to document. Note that as soon as you declare an `<mdoc>` tag inside your FAUST file, the default structure of the automatic mode is ignored, and all the LaTeX content becomes your responsibility.

12.5.1 Six tags

Here are the six specific tags:

- `<mdoc></mdoc>`, to open a documentation field in the FAUST code,
  - `<equation></equation>`, to get equations of a FAUST expression,
  - `<diagram></diagram>`, to get the top-level block-diagram of a FAUST expression,
  - `<metadata></metadata>`, to reference FAUST metadata (cf. declarations), calling the corresponding keyword,
  - `<notice />`, to insert the "adaptive" notice all formulas actually printed,
  - `<listing [attributes] />`, to insert the listing of FAUST files called.

The `<listing />` tag can have up to three boolean attributes (set to `"true"` by default):

- `mdoctags` for `<mdoc>` tags;
- `dependencies` for other files dependencies;
- `distributed` for the distribution of interleaved FAUST code between `<mdoc>` sections.

12.5.2 The mdoc top-level tags

The `<mdoc></mdoc>` tags are the top-level delimiters for FAUST mathematical documentation sections. This means that the four other documentation tags cannot be used outside these pairs (see Section **??**).

In addition to the four inner tags, `<mdoc></mdoc>` tags accept free LaTeX text, including its standard macros (like `\section`, `\emph`, etc.). This allows you to manage the presentation of the resulting tex file directly from within the input FAUST file.

The complete list of the LaTeX packages included by Faust can be found in the file `architecture/latexheader.tex`.

12.5.3 An example of manual mathdoc

```
<mdoc>
\title{<metadata>name</metadata>}
\author{<metadata>author</metadata>}
\date{\today}
\maketitle

\begin{tabular}{ll}
    \hline
    \textbf{name}       & <metadata>name</metadata> \\
    \textbf{version}    & <metadata>version</metadata> \\
    \textbf{author}     & <metadata>author</metadata> \\
    \textbf{license}    & <metadata>license</metadata> \\
    \textbf{copyright}  & <metadata>copyright</metadata>
       \\
    \hline
\end{tabular}
\bigskip
</mdoc>
//
   ----------------------------------------------------------------

// Noise generator and demo file for the Faust math
   documentation
//
   ----------------------------------------------------------------


declare name        "Noise";
declare version     "1.1";
declare author      "Grame";
declare author      "Yghe";
declare license     "BSD";
declare copyright   "(c)GRAME 2009";

<mdoc>
\section{Presentation of the "noise.dsp" Faust program}
This program describes a white noise generator with an
   interactive volume, using a random function.
```

```
\subsection{The random function}
</mdoc>

random  = +(12345)~*(1103515245);

<mdoc>
The \texttt{random} function describes a generator of
   random numbers, which equation follows. You should
   notice hereby the use of an integer arithmetic on 32
   bits, relying on integer wrapping for big numbers.
<equation>random</equation>

\subsection{The noise function}
</mdoc>

noise   = random/2147483647.0;

<mdoc>
The white noise then corresponds to:
<equation>noise</equation>

\subsection{Just add a user interface element to play
   volume!}
</mdoc>

process = noise * vslider("Volume[style:knob]", 0, 0, 1,
   0.1);

<mdoc>
Finally, the sound level of this program is controlled by
    a user slider, which gives the following equation:
<equation>process</equation>

\section{Block-diagram schema of process}
This process is illustrated in Figure 1.
<diagram>process</diagram>

\section{Notice of this documentation}
You might be careful of certain information and naming
   conventions used in this documentation:
<notice />
```

```
\section{Listing of the input code}
The following listing shows the input Faust code, parsed
   to compile this mathematical documentation.
<listing mdoctags="false" dependencies="false"
   distributed="true" />
</mdoc>
```

The following page which gathers the four resulting pages of `noise.pdf` in small size. might give you an idea of the produced documentation.

12.5.4 The -stripmdoc option

As you can see on the resulting file `noisemetadata.pdf` on its pages 3 and 4, the listing of the input code (section 4) contains all the mathdoc text (here colored in grey). As it may be useless in certain cases (see Goals, section 12.1), we provide an option to strip mathdoc contents directly at compilation stage: `-stripmdoc` (short) or `--strip-mdoc-tags` (long).

12.6 Localization of mathdoc files

By default, texts used by the documentator are in English, but you can specify another language (French, German and Italian for the moment), using the `-mdlang` (or `--mathdoc-lang`) option with a two-letters argument (`en`, `fr`, `it`, etc.).

The `faust2mathdoc` script also supports this option, plus a third short form with `-l`:

```
faust2mathdoc -l fr myfaustfile.dsp
```

If you would like to contribute to the localization effort, feel free to translate the mathdoc texts from any of the `mathdoctexts-*.txt` files, that are in the `architecture` directory (`mathdoctexts-fr.txt`, `mathdoctexts-it.txt`, etc.). As these files are dynamically loaded, just adding a new file with an appropriate name should work.

Noise

Grame, Yghe

March 9, 2010

name	Noise
version	1.1
author	Grame, Yghe
license	BSD
copyright	(c)GRAME 2009

```
//-----------------------------------------------------------------
// Noise generator and demo file for the Faust math documentation
//-----------------------------------------------------------------

declare name      "Noise";
declare version   "1.1";
declare author    "Grame";
declare author    "Yghe";
declare license   "BSD";
declare copyright  "(c)GRAME 2009";
```

1 Presentation of the "noise.dsp" Faust program

This program describes a white noise generator with an interactive volume, using a random function.

1.1 The random function

```
random  = +(int(12345))~*(int(1103515245));
```

The `random` function describes a generator of random numbers, which equation follows. You should notice hereby the use of an integer arithmetic on 32 bits, relying on integer wrapping for big numbers.

1. Output signal y such that

$$y(t) = r_1(t)$$

2. Input signal (none)

3. Intermediate signal r_1 such that

$$r_1(t) = 12345 \oplus 1103515245 \odot r_1(t-1)$$

1.2 The noise function

```
noise   = (int(random))/(int(random+1));
```

The white noise then corresponds to:

1. Output signal y such that

$$y(t) = s_1(t)$$

2. Input signal (none)

3. Intermediate signal s_1 such that

$$s_1(t) = \mathrm{int}\,(r_1(t)) \oslash \mathrm{int}\,(1 \oplus r_1(t))$$

1.3 Just add a user interface element to play volume!

```
process = noise * vslider("Volume[style:knob]", 0, 0, 1, 0.1);
```

Endly, the sound level of this program is controlled by a user slider, which gives the following equation:

1. Output signal y such that

$$y(t) = u_{s1}(t) \cdot s_1(t)$$

2. Input signal (none)

3. User-interface input signal u_{s1} such that

$$\text{"Volume"} \quad u_{s1}(t) \in [0,1] \quad \text{(default value} = 0)$$

2 Block-diagram schema of process

This process is illustrated on figure 1.

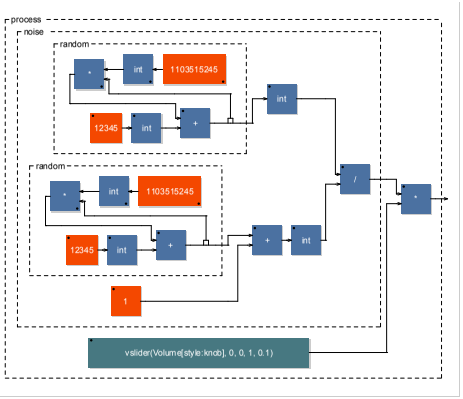


Figure 1: Block diagram of `process`

3 Notice of this documentation

You might be careful of certain information and naming conventions used in this documentation:

- This document was generated using Faust version 0.9.13 on March 09, 2010.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the f_S sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal s in S is a function mapping[1] times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from S^n to S^m, where $n, m \in \mathbb{N}$. See the Faust manual for additional information (http://faust.grame.fr).
- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.
- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E; each graph is put in a box labeled by I. Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- $\forall\, x \in \mathbb{R}$,
$$\mathrm{int}(x) = \begin{cases} \lfloor x \rfloor & \text{if } x > 0 \\ \lceil x \rceil & \text{if } x < 0 \\ 0 & \text{if } x = 0 \end{cases} .$$
- This document uses the following integer operations:

operation	*name*	*semantics*
$i \oplus j$	integer addition	normalize$(i + j)$, in $\mathbb{Z}$
$i \odot j$	integer multiplication	normalize$(i \cdot j)$, in $\mathbb{Z}$
$i \oslash j$	integer division	normalize$(\mathrm{int}(i/j))$, in $\mathbb{Q}$

  Integer operations in Faust are inspired by the semantics of operations on the n-bit two's complement representation of integer numbers; they are internal composition laws on the subset $[\,-2^{n-1}, 2^{n-1}-1\,]$ of $\mathbb{Z}$, with $n = 32$. For any integer binary operation $\times$ on $\mathbb{Z}$, the $\otimes$ operation is defined as: $i \otimes j = \mathrm{normalize}(i \times j)$, with
$$\mathrm{normalize}(i) = i - N \cdot \mathrm{sign}(i) \cdot \left\lfloor \frac{|i| + N/2 + (\mathrm{sign}(i)-1)/2}{N} \right\rfloor ,$$
  where $N = 2^n$ and $\mathrm{sign}(i) = 0$ if $i = 0$ and $i/|i|$ otherwise. Unary integer operations are defined likewise.
- The `noisemetadata-mdoc/` directory may also include the following subdirectories:
  - `cpp/` for Faust compiled code;
  - `pdf/` which contains this document;
  - `src/` for all Faust sources used (even libraries);
  - `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (http://www.w3.org/Graphics/SVG/);
  - `tex/` for the LaTeX source of this document.

[1] Faust assumes that $\forall\, s \in S, \forall\, t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

4 Listing of the input code

The following listing shows the input Faust code, parsed to compile this mathematical documentation.

Listing 1: noisemetadata.dsp

```
//-----------------------------------------------------------------
// Noise generator and demo file for the Faust math documentation
//-----------------------------------------------------------------

declare name      "Noise";
declare version   "1.1";
declare author    "Grame";
declare author    "Yghe";
declare license   "BSD";
declare copyright  "(c)GRAME 2009";


random  = +(int(12345))~*(int(1103515245));


noise   = (int(random))/(int(random+1));


process = noise * vslider("Volume[style:knob]", 0, 0, 1, 0.1);
```

12.7 Summary of the mathdoc generation steps

1. First, to get the full mathematical documentation done on your faust file, call `faust2mathdoc myfaustfile.dsp`.
2. Then, open the pdf file `myfaustfile-mdoc/pdf/myfaustfile.pdf`.
3. That's all !

Chapter 13

Acknowledgments

Many people contribute to the FAUST project by providing code for the compiler, architecture files, libraries, examples, documentation, scripts, bug reports, ideas, etc. We would like in particular to thank:

- Fons Adriaensen
- Karim Barkati
- Jérôme Barthélemy
- Tim Blechmann
- Tiziano Bole
- Alain Bonardi
- Thomas Charbonnel
- Raffaele Ciavarella
- Julien Colafrancesco
- Damien Cramet
- Sarah Denoux
- Étienne Gaudrin
- Olivier Guillerminet
- Pierre Guillot
- Albert Gräf
- Pierre Jouvelot
- Stefan Kersten

- Victor Lazzarini
- Matthieu Leberre
- Mathieu Leroi
- Fernando Lopez-Lezcano
- Kjetil Matheussen
- Hermann Meyer
- Romain Michon
- Rémy Muller
- Eliott Paris
- Reza Payami
- Laurent Pottier
- Sampo Savolainen
- Nicolas Scaringella
- Anne Sedes
- Priyanka Shekar
- Stephen Sinclair
- Travis Skare
- Julius Smith
- Mike Solomon
- Michael Wilson

as well as our colleagues at GRAME:

- Dominique Fober
- Christophe Lebreton
- Stéphane Letz
- Romain Michon

We would also like to thank the following institutions for their financial support:

- the French Ministry of Culture
- the Rhône-Alpes Region
- the City of Lyon
- the French National Research Agency (ANR)